

药材热风烘干过程的热-质耦合模型与收缩域烘干时长预测

摘 要

药材热风烘干的时长取决于内部传热传质阻力。本文以圆柱形药材径向截面为对象，建立温度场与含水率场耦合的一维非线性抛物方程组，按预热、变物性、达标时长与失水收缩四个层次递进求解。

针对问题一，按傅里叶导热与菲克扩散定律写出柱坐标守恒关系，表面取**第三类边界**（ $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ ），轴心取对称零通量。常物性下温度可独立求解，扩散系数仍随含水率逐点更新。采用**全隐式有限体积法**离散、Thomas 消元推进至 1800 s，得中心 33.5755°C 与 2.5500 kg/kg 、表面 36.7856°C 与 1.5103 kg/kg 。热扩散率与扩散系数之比约 34，「热先于质」得到定量刻画；以 Bessel 级数叠加 Duhamel 卷积独立对照，全程最大温度偏差 0.24 mK ，末时刻径向最大偏差 0.15 mK (0.00015°C)。

针对问题二，四条经验式使密度、比热与导热系数随含水率变化，扩散系数同时含 $e^{-0.45/C}$ 与活化能因子 e^{-3850/T_K} ，两场经物性双向耦合。采用**保序算子分裂**并以 Picard 迭代复核：先由旧场算热物性解温度，再用新温度算扩散系数解含水率，最多 3 次收敛。3 h 时中心已升至 49.8495°C 接近烘房温度，表面温度为 49.9664°C ，中心含水率仍为 1.7662 kg/kg 、表面 1.0081 kg/kg ，表明温度场先达准稳态，其后由内部扩散阻力控制。

针对问题三，定义 t_{end} 为最大含水率首次达到阈值 $C_{\text{th}} = 0.15$ 的时刻，取逐点最大值而非体平均（后者会在芯部尚未达标时提前跨过阈值），控制点为含水率最高的轴心，并在输出亚步间线性插值定位。求得**烘干时长** 57.4716 h ，达标时各位置含水率由中心 0.1500 递降至表面 0.0527 ；干燥速率自首个输出时刻起单调衰减，全程无恒速段。

针对问题四，外边界随失水内移，由附件 2 全部 145 个半径数据经单调化与 PCHIP 保形插值给出 $R(t)$ 。干基含水率是物质点上的场；在均匀仿射收缩下取物质坐标 $\xi = r/R(t)$ （等价于 $\eta = (r/R)^2$ ），计算域固定且主闭合不再含表观对流。正式配置 $N = 1280$ 、 $\Delta t = 0.25 \text{ s}$ ， $\chi = 1$ 主闭合求得**时长** 51.0869 h 、末态半径 1.2000 cm ； $\chi = 0$ 替代闭合为 52.6476 h ，两者相差 1.5607 h (3.0550%)。

四问离散守恒残差均不超过 1.6×10^{-14} ；扩散系数前因子是最强不确定度来源。忽略潜热会使时长预测偏短（约 10%）。

关键字： 热-质耦合 有限体积法 移动边界 物质坐标 烘干时长预测

一、问题重述

1.1 问题背景

中药材采收后含水量高，必须及时干燥到安全水分以下才能贮藏与流通。热风烘干是产地初加工中最常用的方式：热空气向表面供热并带走析出水分，水分则由芯部向表面扩散。热量传播快、水分迁移慢，芯部往往在表面明显干燥之后很久才达到安全水分。干燥不足有霉变风险，过度延长则增加能耗并可能损失热敏性有效成分^[1]。真实药材失水时还会收缩^[2]，且密度、比热容、导热系数与水分扩散系数随含水率变化，扩散系数还依赖温度，传热与传质因此必须联立求解。事先预测内部温度与水分的分布并给出可靠烘干时长，直接决定工艺参数与设备排产。

1.2 问题的提出

本题以圆柱形药材为对象。药材初始半径 2 cm、长度 25 cm，初始温度 28 °C、初始干基含水率 2.55 kg/kg，被送入烘房干燥。烘房内热空气的温度与含湿量随时间变化，由附件给出的实测时序描述；药材表面与空气之间的对流换热系数和对流传质系数为已知常数。四个子问题沿同一物理过程逐步放开限制条件：前两问要求刻画给定时段内的场分布，后两问要求给出达到安全水分所需的烘干时长，其中最后一问还需考虑药材的实际收缩。

问题一考察预热阶段。此时物性可按常数处理，水分扩散系数只随局部含水率变化。需要建立药材内部的温度与水分输运关系，求出前 1800 s 内若干时刻沿半径若干位置的温度与含水率，并给出全过程的完整场数据。

问题二把物性的真实依赖关系全部放开。密度、比热容与导热系数随含水率变化，水分扩散系数同时随含水率和温度变化，温度场与水分场因此互相牵动。需要在这一耦合条件下重新刻画整个烘干过程，并给出前 3 h 内的温度与含水率分布。

问题三在问题二的模型上追加干燥完成的判定。当药材内部含水率降到 0.15 kg/kg 以下即视为烘干完成，需要给出达到该状态所需的时长，并给出全过程中若干时刻的含水率分布。

问题四进一步考虑药材的径向收缩。半径随时间的实测数据由附件给出，物性关系也相应更换。求解域不再固定，需要在随时间缩小的区域上重建模型，重新给出烘干时长与含水率分布，并说明收缩这一因素究竟把烘干时长改变了多少。

二、问题分析

四问共享同一个物理对象与同一套输运机理，差异只在物性关系、是否需要终止判断、求解域是否随时间变化。据此把公共部分前置为统一的连续模型与离散方法，各问只陈述本问新增的内容，其间的依赖次序见图 1。

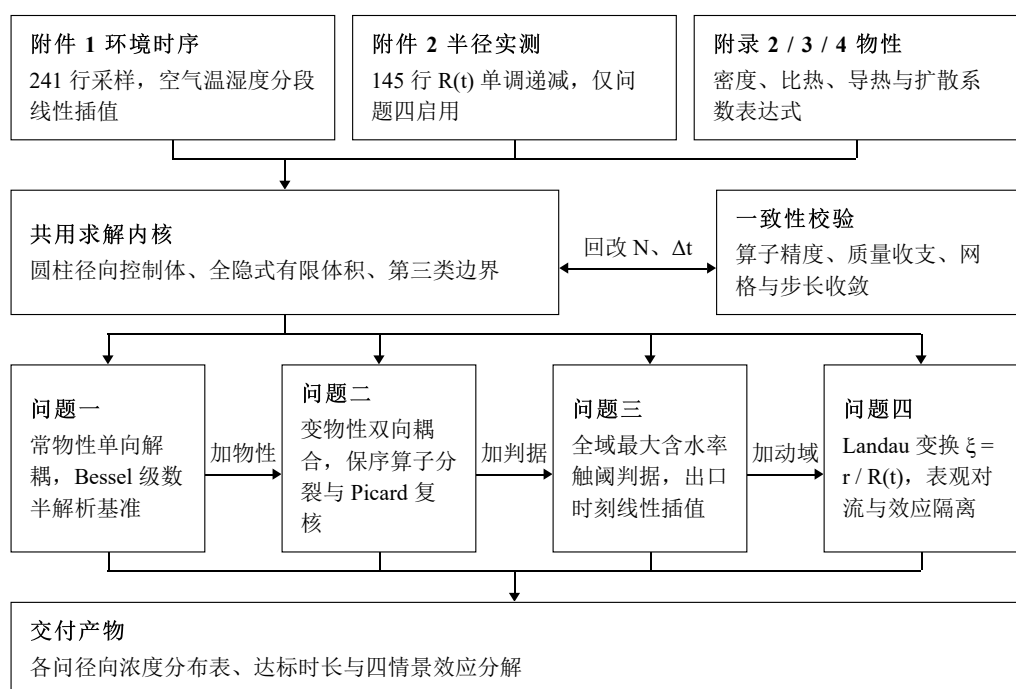


图 1 全文建模路线

图中三份题给材料汇入同一求解内核，内核与一致性校验之间是双向边：算子精度、离散守恒残差和网格与步长收敛一旦不达标，回改的是控制体数与时间步长，而不是物性参数。四问在图中呈链式而非并列排布，正是因为每一问只新增一项机制。

2.1 问题一的分析

本问是给定环境驱动下的瞬态输运正问题。热扩散时间尺度明显短于质扩散，且表面与内部阻力均不可忽略，因此采用柱坐标分布参数模型与第三类边界；具体倍数与毕奥数留到问题一的结果与半解析校核。若改用集中参数，径向差异会被抹去。

本问的另一价值在于可校核性。常物性下温度场退化为线性问题，可用 Bessel 级数配合 Duhamel 叠加得到半解析解。这是全题唯一可与数值解独立比对的外部数学基准，为后三问的同一套离散格式提供信心。

2.2 问题二的分析

本问在问题一基础上放开物性依赖，任务类型不变而数学类型改变。密度、比热容、导热系数随含水率变化，使能量方程的系数由水分场决定；水分扩散系数中含绝对温度的指数因子，使水分方程的系数由温度场决定。两个方向的依赖同时存在，构成双向耦合环，不能先算完整条温度时程再算水分。图 2 把这一结构与问题一的解耦情形并置。

附录 2 下两条耦合边同时断开，温度可独立求解；附录 3 与附录 4 下必须同步推进。求解上要同时处理「物性用哪一层场」与「耦合是否收敛」，次序见公共模型。

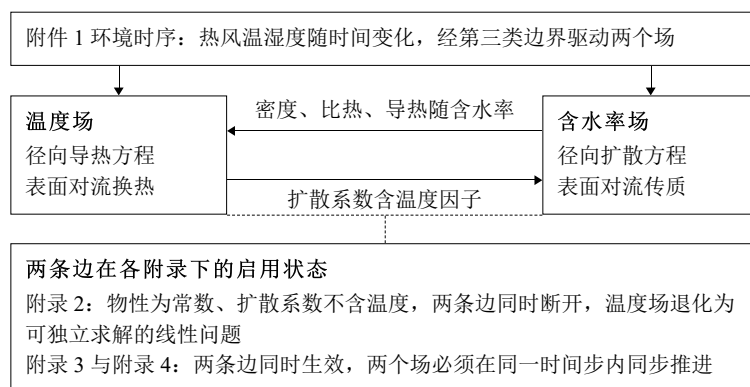


图2 温度场与含水率场的耦合结构

2.3 问题三的分析

本问是在问题二的模型上追加一个事件型终止条件，模型结构不需改动。关键是判据本身如何定义。题目要求药材内部含水率降到阈值以下，而干燥过程中含水率沿半径始终不均匀，中心最高、表面最低。若以体平均含水率判定，会在芯部尚未达标时提前宣布完成；只有以逐点最大值判定，才能保证药材内所有位置都已达标。因此本问把达标时刻定义为逐点最大含水率首次达到阈值的时刻，并需要在输出步之间做插值定位，否则时长精度受输出间隔限制。达标时刻的量化差别有多大，在结果部分用同一次求解的两种判据读数直接对照。

2.4 问题四的分析

本问同时改变了两件事：物性关系由附录3换成附录4，求解域由固定半径换成随时间收缩。干基含水率首先是物质点上的场。直接在物理坐标上离散会导致每步重新生成网格并插值，既损失守恒性又难以判断误差来源。本问在仿射收缩下取物质坐标 $\xi = r/R(t)$ （等价于 $\eta = (r/R)^2$ ），把收缩的物理域映射成固定的计算域，主闭合不再含表观对流；若改用保留表观对流的替代闭合，所得时长用于敏感性比较，而不视为同一守恒律的坐标变换验证。

更需要警惕的是归因。既然两件事同时改变，直接比较问题四与问题三的时长并把差异说成「收缩使干燥加快」是典型的混淆归因。因此本问构造单因素改变的情景链，把净差拆成换物性与加收缩两项，各自量化后再合成。此外，附录4给出的密度关系与附件2给出的收缩数据在严格干物质守恒下并不完全相容，这一不相容度需要如实报告为题给数据的诊断量，而不能反过来修正实测半径。

三、模型假设

(1) 药材内部的温度与水分只沿径向变化，忽略轴向与周向的不均匀。药材长度 25 cm 与直径 4 cm 之比为 6.25，两端面面积仅占总换热面积的 7.4%；按扩散时间 L^2/D 估计，轴向输运所需时间约为径向的 39 倍，在本题关心的时间范围内轴向梯度不足以改变径向结果。

(2) 药材视为均质连续介质，水分以有效扩散的方式迁移，不区分液态水与水蒸气的输运路径。题目直接给出有效水分扩散系数的表达式，即已把毛细流动与气相扩散归并到同一个系数中，这是干燥文献中处理多孔物料的常用简化^[3]。

(3) 能量方程不含水分蒸发的潜热汇。题目未给出汽化潜热与相变发生位置，无法确定该汇项的大小与分布。后文采用近似汽化潜热作结构性诊断，以评估该省略对时长的影响方向与量级。

(4) 烘房空气温度与含湿量在 4 h 后采用稳态外推值 50.0000 °C 与 0.0500 kg/kg，不作线性外推。附件 1 的温度在约 2.95 h 后已进入平台，继续线性外推会得到无物理依据的持续升温。

(5) 药材收缩只发生在径向，长度不变，且收缩不可逆。题目仅给出半径随时间的实测数据，未给出长度变化；失水造成的骨架塌缩在恒温干燥中不会自行回复，故半径取为单调不增。

(6) 表面对流换热系数与对流传质系数在四个子问题中取同一组给定值，不随温度、含水率或表面收缩而变化。题目在给出这两个系数时未附带修正关系，按题面条件统一使用。

四、符号说明

下表收录贯穿全文、直接进入核心方程与判据的符号。开关 χ 的 1/0 含义见问题四运动学闭合。

主要符号说明

符号	含义	单位	取值或范围
r	到药材轴心的径向距离	m	$[0, R]$
t	时间	s	$[0, t_{\text{end}}]$
$T(r, t)$	药材内部温度	°C	$[28, 50.2]$
$C(r, t)$	药材内部干基含水率	kg/kg	$[0.0526, 2.55]$
T_K	绝对温度 $T + 273.15$	K	$[301.15, 323.32]$
w	湿基水分分率 $C/(C + 1)$	—	$[0.050, 0.718]$

符号	含义	单位	取值或范围
ρ	药材密度	$\text{kg}\cdot\text{m}^{-3}$	随物性关系而定
c_p	定压比热容	$\text{J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$	随物性关系而定
k	导热系数	$\text{W}\cdot\text{m}^{-1}\cdot\text{K}^{-1}$	随物性关系而定
D	水分有效扩散系数	$\text{m}^2\cdot\text{s}^{-1}$	1.6×10^{-10} – 1.4×10^{-8}
h	表面对流换热系数	$\text{W}\cdot\text{m}^{-2}\cdot\text{K}^{-1}$	25.0
h_m	表面对流传质系数	$\text{m}\cdot\text{s}^{-1}$	8×10^{-7}
$T_{\text{air}}(t)$	烘房空气温度	$^{\circ}\text{C}$	[28.000, 50.246]
$C_{\text{air}}(t)$	烘房空气含湿量	kg/kg	[0.01963, 0.05025]
$R(t)$	药材半径	m	$0.0200 \rightarrow 0.01198$
$\dot{R}(t)$	半径收缩速率	$\text{m}\cdot\text{s}^{-1}$	≤ 0
ξ	归一化径向坐标 $r/R(t)$	—	[0, 1]
η	归一化物质坐标 $(r/R(t))^2$	—	[0, 1]
v_s	固相仿射速度 $r\dot{R}/R$	$\text{m}\cdot\text{s}^{-1}$	≤ 0
χ	运动学闭合开关	—	{0, 1} (1 主闭合, 0 替代)
C_{th}	达标含水率阈值	kg/kg	0.15
t_{end}	烘干结束时间	h	见各问结果
α	热扩散率 $k/(\rho c_p)$	$\text{m}^2\cdot\text{s}^{-1}$	1.689×10^{-7}
N	径向网格区间数	—	160, 1280, 2560
Δt	时间步长	s	0.125, 0.25

五、热-质耦合输运的公共模型与离散方法

本章建立四问共用的连续模型、边界条件、环境驱动与数值方法。各问只在物性关系、终止条件和求解域三处与本章不同，后续章节只陈述这些差异，不重复整套推导。

5.1 系统边界与守恒关系

取药材横截面的径向一维区域 $r \in [0, R]$ 为研究对象， R 在前三问为常数、在问题四随时间变化。以单位轴向长度的圆环控制体为对象写守恒关系。热量方面，控制体内焓的变化率等于经内外两个圆柱面净流入的导热量；水分方面，干基含水率的变化率等

于净流入的扩散水量。圆柱几何使两个圆环面的面积不等（内侧面积小、外侧面积大），这一差异在微分形式中表现为 $1/r$ 因子。按傅里叶导热定律与菲克扩散定律闭合通量^[3, 4]，得到

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right), \quad (1)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right). \quad (2)$$

式 (1) 中 $T(r, t)$ 为药材内部温度， ρ 、 c_p 、 k 分别为密度、比热容与导热系数；式 (2) 中 $C(r, t)$ 为干基含水率， D 为水分有效扩散系数。本文依据题给经验扩散关系，将干基含水率 C 作为有效扩散变量，建立 Fick 型有效扩散模型；该形式与以湿基质量分数为变量的写法不同，后者会额外引入密度的时间导数，但不声称在一般收缩介质中天然等价于严格局部质量守恒。两式的系数都可能依赖 C 与 T ，因此这是一组非线性抛物型方程，其耦合强度完全由所用物性关系决定。

5.2 边界条件与初始条件

药材表面与热空气之间既有对流换热又有对流传质，两者都属第三类边界：表面通量与药材表面值和空气侧值之差成正比。表面处

$$-k(C) \frac{\partial T}{\partial r} \Big|_{r=R} = h [T(R, t) - T_{\text{air}}(t)], \quad -D(C, T) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m [C(R, t) - C_{\text{air}}(t)], \quad (3)$$

其中 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 为题给的对流换热与对流传质系数。按第三类边界建模而不用第一类，理由见问题一的分析^[4, 5]。轴心处由几何对称性给出零通量条件

$$\frac{\partial T}{\partial r} \Big|_{r=0} = 0, \quad \frac{\partial C}{\partial r} \Big|_{r=0} = 0. \quad (4)$$

式 (4) 并非附加假设，而是径向对称场在轴心可微的必然结果。初始时刻药材温度与含水率均匀

$$T(r, 0) = T_0 = 28^\circ \text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}, \quad R(0) = R_0 = 2 \text{ cm}. \quad (5)$$

5.3 环境驱动的处理

式 (3) 中的 $T_{\text{air}}(t)$ 与 $C_{\text{air}}(t)$ 来自附件 1 的实测时序，全部 241 个采样点按分段线性插值，不作两点线性化。附件 1 仅覆盖前 4 h，其后按平台末值保持 $T_{\text{air}} = 50.0000^\circ \text{C}$ 、

$C_{\text{air}} = 0.0500 \text{ kg/kg}$; 50.165°C 与 0.04986 kg/kg 仅为末采样点。实测段与外推段的衔接见图 3。

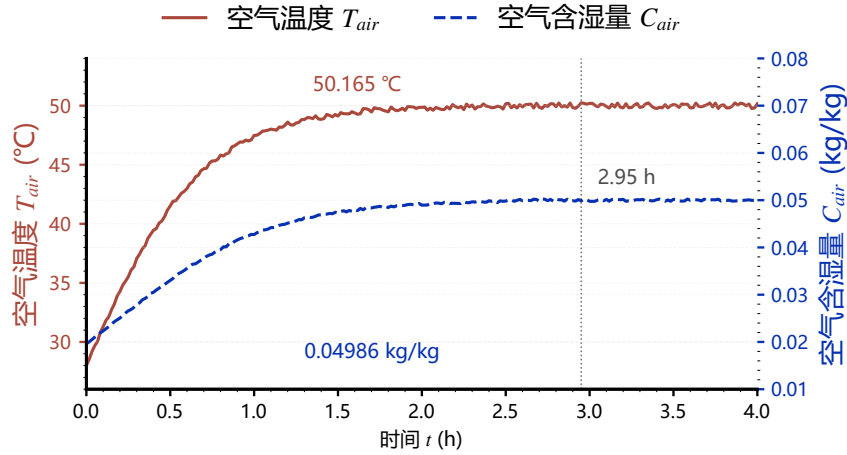


图 3 附件 1 干燥室工况的双轴时程

进入平台后，长时程求解的边界驱动不再随环境变化，内部扩散阻力成为继续失水的限制因素。

5.4 物性关系与由此决定的数学类型

三个附录给出三套物性关系，分别用于问题一、问题二至三、问题四。记湿基水分分率 $w = C/(C + 1)$ 、绝对温度 $T_K = T + 273.15$ 。问题一采用常物性

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D(C) = 7 \times 10^{-9} e^{-0.89/C}, \quad (6)$$

问题二与问题三采用

$$\begin{aligned} \rho &= 650 + 128C, \quad c_p = 1450 + 2736w, \quad k = 0.21 + 0.38w, \\ D &= 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T_K}, \end{aligned} \quad (7)$$

问题四采用

$$\begin{aligned} \rho &= 760 + 90C, \quad c_p = 1850 + 2150w, \quad k = 0.12 + 0.20w, \\ D &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}. \end{aligned} \quad (8)$$

三式中扩散系数的指数因子 $e^{-a/C}$ 表示含水率越低、水分迁移越困难， e^{-3850/T_K} 是典型的活化能形式，表示升温促进扩散^[6]。指数中的温度必须取绝对温度：若误用摄氏读数，在 28°C 附近该因子会相差十余个数量级。

式 (6) 与后两式的区别决定了问题的数学类型。常物性下式 (1) 的系数与 C 无关，且 D 不含温度。在问题一的物性关系下，温度方程与水分方程完全解耦：温度场可独立求解，水分场仅通过 $D(C)$ 与自身含水率发生非线性关联，无需调用温度场结果。式 (7)

与式 (8) 下两条依赖同时接通，两场必须联立推进。图 4 比较后两套关系中扩散系数在实际工作区间上的分布。

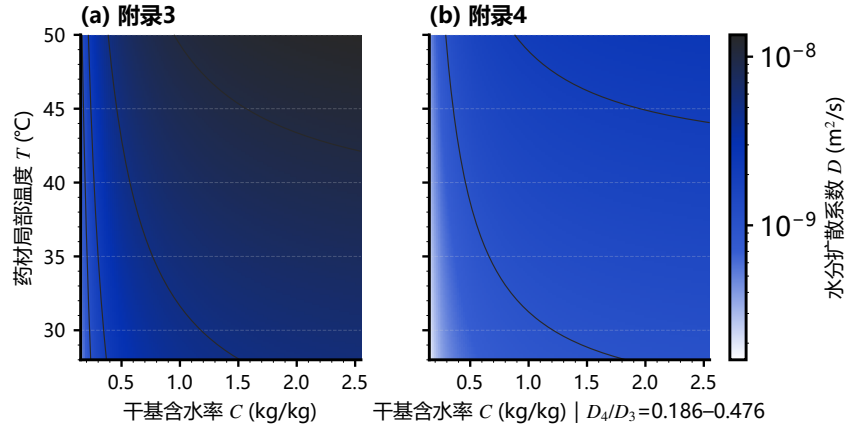


图 4 附录 3 与附录 4 的水分扩散系数分布对比

在 $C \in [0.15, 2.55]$ 、 $T \in [28, 50]$ °C 上，附录 3 的扩散系数覆盖 3.35×10^{-10} – 1.35×10^{-8} m²/s，附录 4 覆盖 1.59×10^{-10} – 2.50×10^{-9} m²/s，比值介于 0.186–0.476。附录 4 的迁移能力系统性地低于附录 3，最多低约五倍：若只换物性而不引入收缩，时长必然显著延长。

5.5 有限体积离散

式 (1) 与式 (2) 的系数随解变化，不存在通用的解析解，须数值求解。本文采用有限体积法而非有限差分：有限体积法从控制体积分形式出发，相邻控制体共用界面通量，离散层面自动满足守恒，圆柱几何的 $1/r$ 因子由界面面积 $2\pi r_{i\pm 1/2}$ 自然承担，不需要额外处理奇点^[7,8]。这一性质在本题尤其重要，因为烘干时长由累积失水量决定，守恒误差会直接积累成时长误差。

将 $[0, R]$ 等分为 N 个径向区间，设置 $N + 1$ 个节点 r_i ($i = 0, \dots, N$)，并以节点为中心构造有限体积控制体；轴心与表面采用半宽控制体。三类控制体的单位长度体积分别为

$$V_0 = \pi \left(\frac{\Delta r}{2} \right)^2, \quad V_i = 2\pi r_i \Delta r \quad (1 \leq i \leq N-1), \quad V_N = \pi (R - r_{N-1/2}) (R + r_{N-1/2}). \quad (9)$$

式 (9) 的三种写法互不相同：轴心控制体是实心小圆盘，内部控制体含向心汇聚因子 r_i ，表面控制体是半宽圆环。若照搬直角坐标下的等体积划分，中心区域的体积会被系统性高估，从而低估中心的升温与失水速率。三类控制体在柱截面上的实际几何见图 5。

图中虚线弧为控制体界面、实心圆点为结点。表面控制体的外向通量由第三类边界 $q_N = 2\pi R \beta (\varphi_{\text{air}} - \varphi_N)$ 给出，其中 φ 泛指 T 或 C 、 β 泛指 $h/(\rho c_p)$ 或 h_m ；轴心控制体

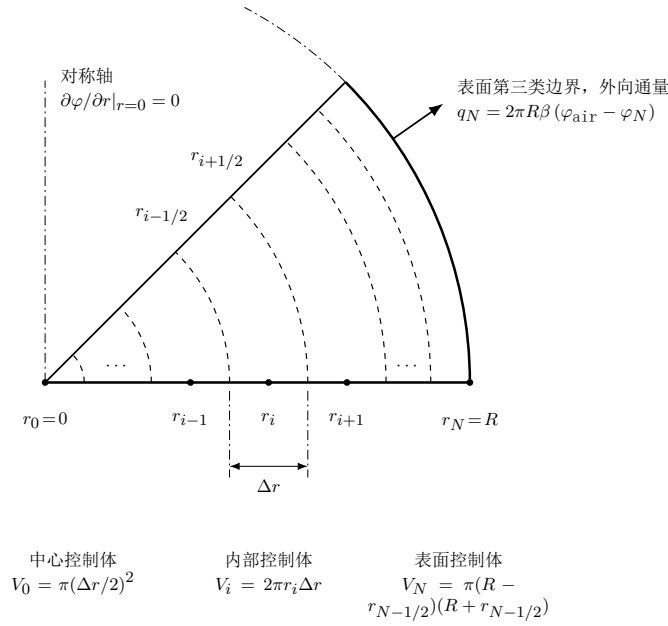


图5 圆柱径向有限体积的控制体划分

则由式 (4) 的对称条件关闭，不需要虚拟结点。

5.6 时间推进、界面物性与耦合处理

时间方向采用全隐式后向 Euler 格式。对通用标量 φ ，在控制体 i 上积分后得到

$$V_i \frac{\varphi_i^{n+1} - \varphi_i^n}{\Delta t} = 2\pi r_{i+1/2} \Gamma_{i+1/2} \frac{\varphi_{i+1}^{n+1} - \varphi_i^{n+1}}{\Delta r} - 2\pi r_{i-1/2} \Gamma_{i-1/2} \frac{\varphi_i^{n+1} - \varphi_{i-1}^{n+1}}{\Delta r}, \quad (10)$$

其中 Γ 为对应的输运系数。显式格式受 $\Delta t \leq \Delta r^2/(2\alpha)$ 限制，本题热扩散率量级下上限约 0.7 s，数十小时积分不可行，故对每次冻结系数后的扩散子问题采用后向 Euler 全隐式离散；该子问题无条件稳定，整体非线性耦合的收敛性由随后的 Picard 迭代判据控制。离散后形成三对角方程组，用 Thomas 算法以 $O(N)$ 求解^[9]。界面系数取算术平均 $\Gamma_{i+1/2} = \frac{1}{2}(\Gamma_i + \Gamma_{i+1})$ ；装配系数满足 $a_i, c_i \leq 0$ 且 $b_i > |a_i| + |c_i|$ ，离散矩阵具有对角占优结构；正式计算未出现非物理振荡，所得温度和含水率剖面保持预期的径向分布特征。

两个场的耦合按固定次序的算子分裂处理。一个时间步内依次完成：（1）由旧层的 (C^n, T^n) 计算 ρ 、 c_p 、 k ；（2）隐式求解温度得 T^{n+1} ；（3）由 (C^n, T^{n+1}) 计算扩散系数；（4）隐式求解含水率得 C^{n+1} ；（5）对新含水率场施加下界以防非物理负值。这一次序不可交换：若在求解温度之前计算扩散系数，该系数就退化为由旧温度决定，温度对传质的促进作用被人为削弱。随后以 Picard 迭代复核——用新层场重算物性并重解，直至相邻两次迭代的场变化按绝对值低于 $10^{-6} \text{ }^\circ\text{C}$ （温度）与 10^{-8} kg/kg （含水率）——以消除分裂本身引入的滞后^[10]。终止判据只在 Picard 收敛之后检查，避免用尚未收敛的中间场触发停机判定。四问的实际计算中 Picard 迭代最多 3 次即达到上述收敛标准，说

明所选时间步长下分裂误差很小；下界截断在四问中均未被触发，即含水率场自始至终保持为正。

交付计算按各问取正式配置：问题一为 $N = 160$ 、 $\Delta t = 0.125$ s，问题二与问题三为同一次 $N = 2560$ 、 $\Delta t = 0.25$ s 连续积分，问题四为 $N = 1280$ 、 $\Delta t = 0.25$ s。这一组网格与步长的选取依据是收敛性检验，具体证据在数值可信度一章给出。

六、问题一：常物性下预热阶段的热-质输运模型与半解析校核

6.1 本问模型

本问沿用公共模型一章的式 (1)–(5)，物性取式 (6)。由于 ρ 、 c_p 、 k 均为常数，式 (1) 化为线性热传导方程

$$\frac{\partial T}{\partial t} = \alpha \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right), \quad \alpha = \frac{k}{\rho c_p} = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}, \quad (11)$$

而水分方程保留非线性

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right), \quad D(C) = 7 \times 10^{-9} \text{ e}^{-0.89/C}. \quad (12)$$

式 (11) 不含 C 、式 (12) 不含 T ，温度场可独立求解。初始含水率下 $D(C_0) = 4.94 \times 10^{-9} \text{ m}^2/\text{s}$ ，与 α 相差 34 倍，预示两个场在同一时间窗内的推进深度将有量级差别。沿用正式配置，本问取 $N = 160$ 、 $\Delta t = 0.125$ s，推进至 1800 s。

6.2 温度场与含水率场的结果

按题目要求的时刻与位置整理结果，见表 1 与表 2。

表 1 预热阶段药材内部温度 (°C)

时间 (s) \ 距中心距离 (cm)	0	0.5	1	1.5	2
100	28.0001	28.0004	28.0041	28.0328	28.1801
300	28.0409	28.0636	28.1515	28.3682	28.8489
600	28.4535	28.5362	28.8041	29.3161	30.1653
900	29.3245	29.4585	29.8757	30.6163	31.7305
1200	30.5429	30.7100	31.2225	32.1127	33.4278
1500	31.9959	32.1869	32.7662	33.7464	35.1203
1800	33.5755	33.7721	34.3643	35.3622	36.7856

表 2 预热阶段药材内部水分浓度 (kg/kg)

时间 (s) \ 距中心距离 (cm)	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2476
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5353	1.8771
900	2.5500	2.5500	2.5497	2.5045	1.7548
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

两表呈现的格局截然不同。温度在 1800 s 内各位置都有明显上升，中心由 28 °C 升至 33.5755 °C、表面升至 36.7856 °C，且同一时刻始终满足表面高于中心的单调序；含水率则只有表面有大幅下降（由 2.55 降至 1.5103 kg/kg）， $r = 1.5$ cm 处降幅 0.1745 kg/kg，而中心处四位小数下仍显示为 2.5500 kg/kg。这一对照即式 (11) 与式 (12) 中输运系数相差 34 倍的直接体现。表面含水率在前 100 s 内即降至 2.2476 kg/kg，说明表面失水几乎从加热一开始就发生，但它并未立即传递到内部。

径向剖面与测点时程合见图 6 与图 7。

图 6 两个面板共用一条时刻色带：温度剖面整体抬升且梯度遍及全域，含水率仅在外层弯折，影响深度不足半径的四分之一。图 7 左轴保持空气高于表面、表面高于中心，与第三类边界方向一致；若表面追平空气温度，则与 $Bi_h = 1.389$ 矛盾。右轴中心含水率在 1800 s 内近乎水平（终值 2.54999 kg/kg），中心水分基本未被动用。

6.3 半解析级数解的独立校核

式 (11) 是常系数线性方程，配合第三类边界与均匀初值，可用分离变量法求解。对阶跃型环境温度，特征值 λ_n 由

$$\lambda_n J_1(\lambda_n) = Bi_h J_0(\lambda_n), \quad Bi_h = \frac{hR}{k} = 1.389 \quad (13)$$

确定， J_0 、 J_1 为零阶与一阶第一类 Bessel 函数，该本征关系是无限长圆柱对流换热问题的经典结果^[11]。实际环境温度并非阶跃而是随时间变化，故按 Duhamel 原理把 $T_{\text{air}}(t)$ 的分段线性时程分解为一系列阶跃响应的叠加，得到

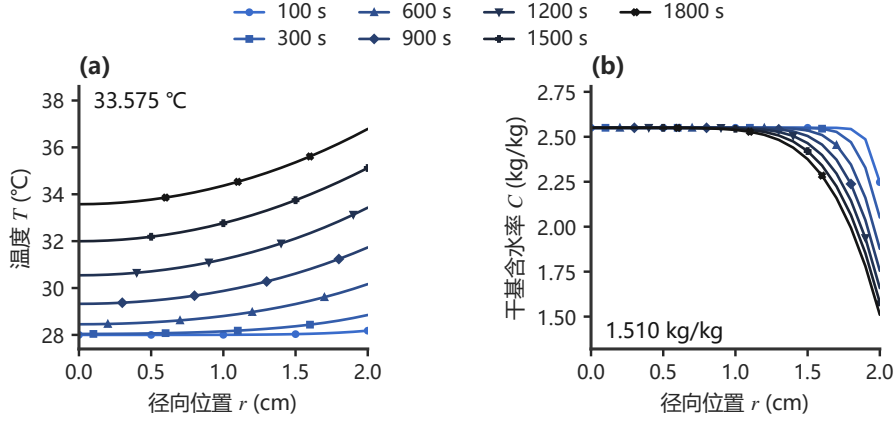


图 6 问题一温度与含水率的径向剖面

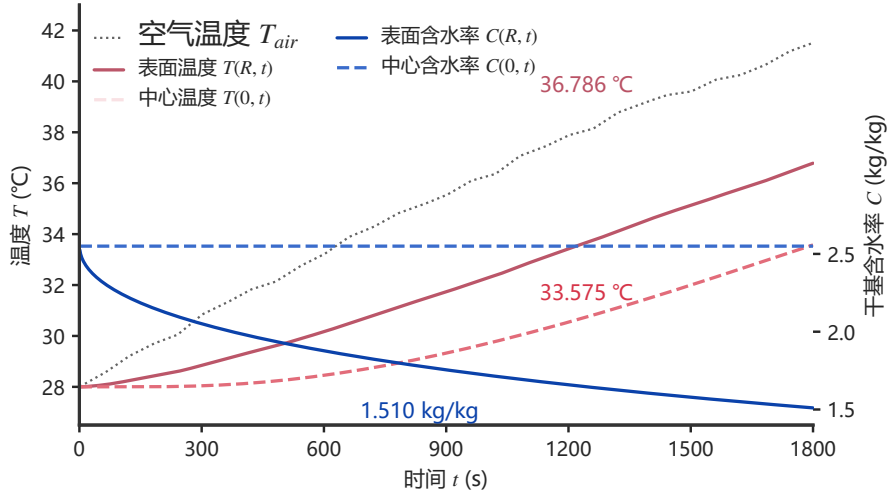


图 7 问题一中心与表面测点的时程对照

$$T(r, t) = T_{\text{air}}(0) + \int_0^t \frac{dT_{\text{air}}(\tau)}{d\tau} \Phi(r, t - \tau) d\tau, \quad \Phi(r, s) = 1 - \sum_{n=1}^{\infty} A_n J_0\left(\lambda_n \frac{r}{R}\right) e^{-\alpha \lambda_n^2 s / R^2}, \quad (14)$$

其中 A_n 为由初值与正交性确定的展开系数， Φ 为单位阶跃响应。式 (14) 与公共模型一章的离散格式完全无关：两者若吻合，则系数装配、边界处理与时间推进都得到了独立确认。本文取前 30 个特征根，并对分段线性的环境时程逐段精确叠加。校核结果与网格、步长收敛证据一并给出，见图 8。

面板 (a) 中数值解与级数解的曲线逐点重合，面板 (b) 给出两者之差：中心与表面两条测点曲线在 1800 s 全程内的最大偏差为 0.24 mK，相对于表面 8.786 °C 的温升约为 0.0027%；若只看末时刻沿半径的偏差，最大值为 0.15 mK 即 0.00015 °C，约为温升的 0.0017%。两个量级都与时间离散的截断误差相当，说明偏差来自 Δt 而非模型实现有误。面板 (c)(d) 给出网格与时间步加密证据：正式结果采用 $N = 160$ 、 $\Delta t = 0.125$ s，其中时间步加密固定 $N = 40$ ，空间加密固定 $\Delta t = 0.125$ s。

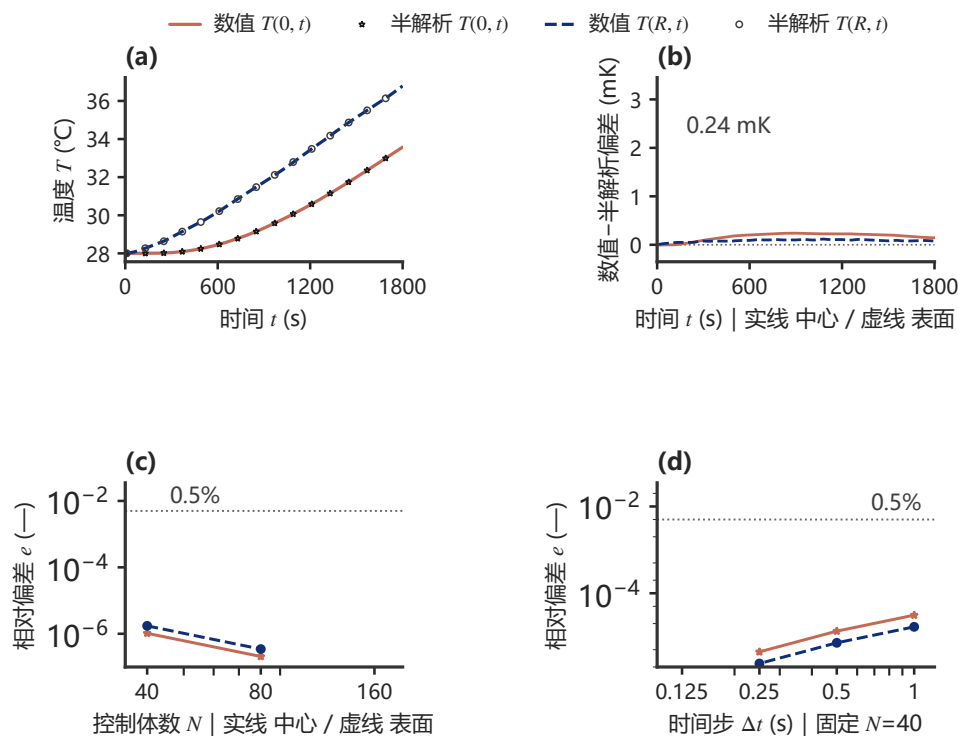


图 8 问题一数值解的四项校核证据

需要说明的是，这一校核只覆盖温度场，因为水分方程含非线性扩散系数，不存在同类级数解。水分场的可信度依靠离散守恒残差与网格收敛证据支撑，详见数值可信度一章。本问的离散守恒相对残差为 7.41×10^{-16} ，已接近双精度浮点的机器精度。该指标用于检验有限体积离散方程的数值一致性，不等同于真实系统总水分质量守恒误差。按题目要求，1800 s 内每隔 1 s、距中心每隔 0.1 cm 的温度与含水率完整结果以双表形式保存于结果文件中，数值保留四位小数；位置坐标由正式网格结点值线性插值至 0.1 cm 间隔。

七、问题二：变物性双向耦合的全过程烘干模型

7.1 相对问题一的改动

本问保留公共模型的控制方程、边界与初值，以及问题一的固定域 $R = 2$ cm，只把物性关系由式 (6) 换成式 (7)。两场因此必须按公共模型的算子分裂与迭代复核联立推进。3 h 只是报表窗口：沿用正式配置，同一次求解一直推进到含水率达标，终止时刻由下一章给出。

7.2 前 3 小时的温度与含水率分布

按题目要求的时刻与位置整理，见表 3 与表 4。

表 3 全过程模型下药材内部温度 (°C)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2
0.5	32.1896	32.3825	32.9663	33.9611	35.4133
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8466	45.9347	46.1928	46.6049	47.1399
2.0	48.4501	48.4879	48.5983	48.7734	49.0032
2.5	49.4669	49.4791	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 全过程模型下药材内部水分浓度 (kg/kg)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5255	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1167
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

温度方面，中心由 0.5 h 的 32.1896 °C 升至 3 h 的 49.8495 °C，已逼近烘房平台温度 50.0000 °C；同一时刻中心与表面之差由 3.22 °C 收窄到 0.12 °C。也就是说药材在约 3 h 后基本进入等温状态，此后温度不再是限制因素。含水率方面，3 h 时中心仍有 1.7662 kg/kg，距 0.15 kg/kg 的目标相去甚远，而表面已降至 1.0081 kg/kg。两者对比给出一个重要判断：干燥的限制环节不是供热，而是水分从芯部迁移到表面的内部阻力。这也解释了题目所述烘干需持续两三天量级。

与问题一相比，中心含水率在 3 h 内下降了 0.78 kg/kg，而问题一在 0.5 h 内几乎未动。本问剖面与同一时刻的两问增量对照合见图 9 与图 10。

图 9 中温度剖面约 2 h 后趋于平坦，含水率由表层弯折转为整体倾斜。图 10 取 $t = 1800$ s 比较温升与含水率降幅：温升差在数摄氏度，中心含水率降幅相差近一个量级 (1.0×10^{-5} 对 6.6×10^{-5} kg/kg)，表面则反转（问题一 1.039 对问题二 0.901 kg/kg）。物性更换主要改写了内部输运，而非表面传质： h_m 相同，问题二内部补水更快，表面浓度维持更高。

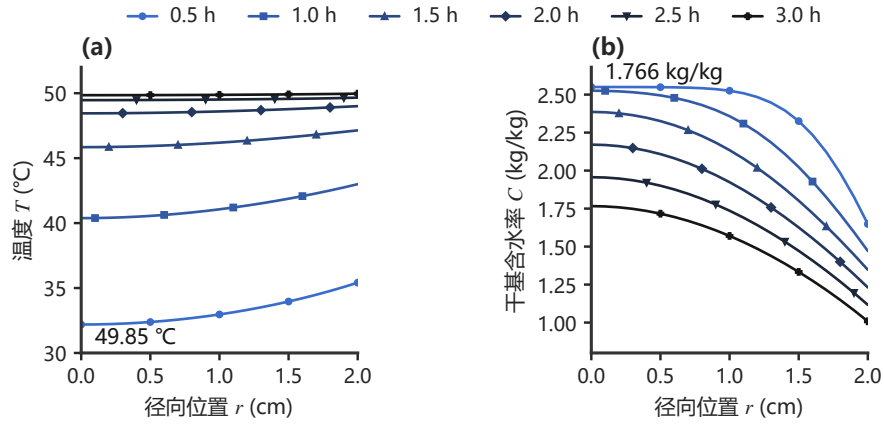


图9 问题二温度与含水率的径向剖面

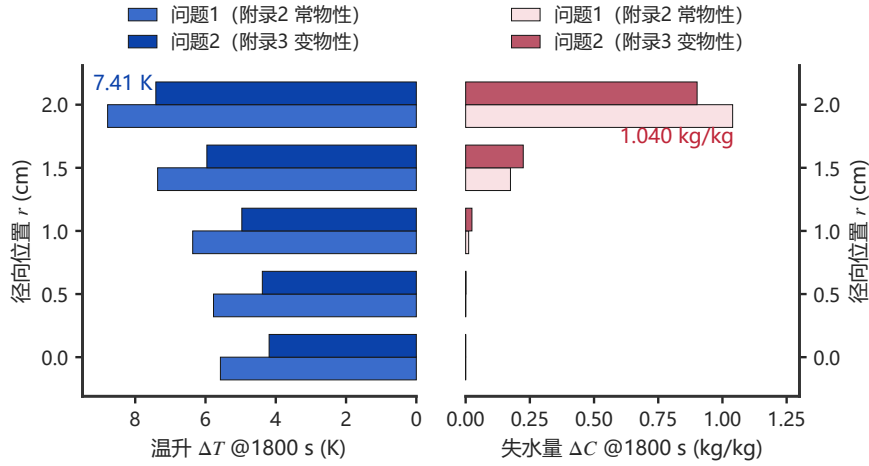


图10 问题一与问题二在 $t = 1800$ s 的增量对比

7.3 耦合关系的实际表现

前文的判断需要落到物性本身。将全程节点场按含水率分箱：随含水率由 2.53 降到 0.13 kg/kg，密度由 973.6 降到 666.5 kg/m³、比热容由 3410 降到 1760 J/(kg·K)、导热系数由 0.482 降到 0.253 W/(m·K)，三者同向下降。扩散系数的最低值 5.56×10^{-10} m²/s 出现在最低含水率箱，说明含水率的影响占优：即使后期温度已升至平台值，扩散系数仍随失水而下降。这正是干燥后期速率持续减慢、达标时间被显著拉长的机制来源。热风干燥试验中由失水曲线反演有效扩散系数与活化能，所得量级与本文所用经验式一致^[12]。

按题目要求，前 3 h 每隔 1 s、距中心每隔 0.1 cm 的温度与含水率完整结果保存于结果文件，共 10800 个时间行；问题二与问题三沿用同一次正式配置连续积分。

八、问题三：逐点达标判据下的烘干时长确定

8.1 相对问题二的改动：达标判据的数学定义

本问的模型与问题二完全相同，唯一新增的是终止条件。判据的定义方式直接决定答案，因此需要单独论证。

干燥过程中含水率沿半径始终不均匀。由表 4 可见同一时刻中心最高、表面最低，两者可相差数倍。要保证「药材内部含水率降到 0.15 kg/kg 以下」，必须要求最难干的位置也达标，故把达标时刻定义为逐点最大含水率首次达到阈值的时刻

$$t_{\text{end}} = \inf \left\{ t > 0 \mid \max_{0 \leq r \leq R} C(r, t) \leq C_{\text{th}} \right\}, \quad C_{\text{th}} = 0.15 \text{ kg/kg}. \quad (15)$$

与式 (15) 相对的是以体平均含水率判定的做法

$$\bar{C}(t) = \frac{\int_0^R C(r, t) 2\pi r \, dr}{\pi R^2}, \quad (16)$$

式 (16) 把径向分布压成一个数，必然在芯部尚未达标时就先跨过阈值，从而给出偏短的时长。本文只把 $\bar{C}$ 作为对照量输出，不用作判据。由式 (4) 的对称条件与解的单调性，含水率最大值总出现在轴心，故式 (15) 的控制点即 $r = 0$ 。

数值实现上， $C_{\text{max}}(t)$ 只在离散输出步上可得，而阈值一般落在两个输出步之间。为不使时长精度受输出间隔限制，在跨越阈值的相邻两步之间对 C_{max} 作线性插值

$$t_{\text{end}} \approx t^n + \Delta t \frac{C_{\text{max}}(t^n) - C_{\text{th}}}{C_{\text{max}}(t^n) - C_{\text{max}}(t^{n+1})}, \quad (17)$$

式 (17) 要求 $C_{\text{max}}(t^n) \geq C_{\text{th}} > C_{\text{max}}(t^{n+1})$ ，即定位的是首次跨越而非任意一次低于阈值的时刻。

8.2 烘干时长与含水率分布

沿用问题二同一次连续积分的正式配置 $N = 2560$ 、 $\Delta t = 0.25 \text{ s}$ ，得到烘干时长

$$t_{\text{end}} = 57.4716 \text{ h} \approx 2.39 \text{ d}, \quad (18)$$

该时刻的逐点最大含水率为 0.150000 kg/kg，恰好落在阈值上，控制点位于轴心。式 (18) 与题目所述「烘干过程一般持续 2–3 天」的量级一致。各时刻的含水率分布见表 5。

表 5 烘干过程中药材内部水分浓度 (kg/kg)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
烘干结束时间 57.4716 h	0.1500	0.1477	0.1402	0.1249	0.0527

表中每一行都保持中心含水率最高、表面最低的次序，与式 (15) 取轴心为控制点相符。前 12 h 中心含水率由 2.55 降至 0.4561 kg/kg，减少了八成；而从 12 h 到 57.4716 h 的四十余小时只再减少 0.30 kg/kg。也就是说约五分之四的水分在前四分之一的时间内被除去，剩下的时间几乎全部花在把中心含水率从 0.45 压到 0.15 kg/kg 上。末行还显示表面含水率已降至 0.0527 kg/kg，远低于阈值，中心与表面相差近三倍——这正是不能用平均量判定的直接证据。

判据差别与干燥速率阶段合见图 11 与图 12。

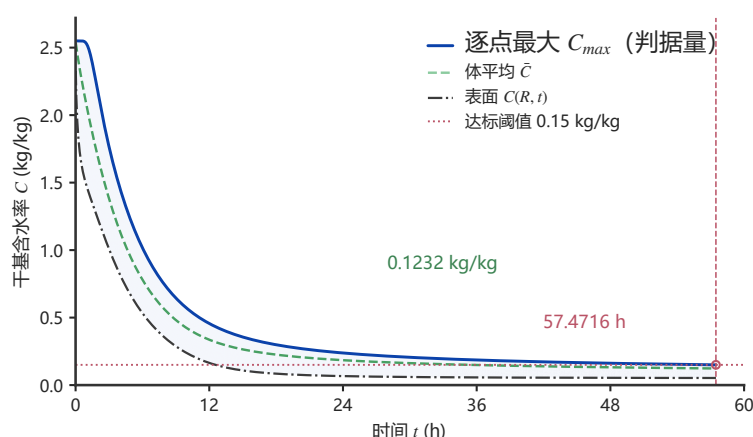


图 11 问题三干燥曲线与达标时刻定位

图 11 的渐变带为同一时刻由表面到芯部的含水率范围。 $C_{\max}$ 触及阈值时体平均只有 0.1232 kg/kg，已低于阈值约 18%；若改用式 (16)，会在芯部尚有约 0.18 kg/kg 时宣布

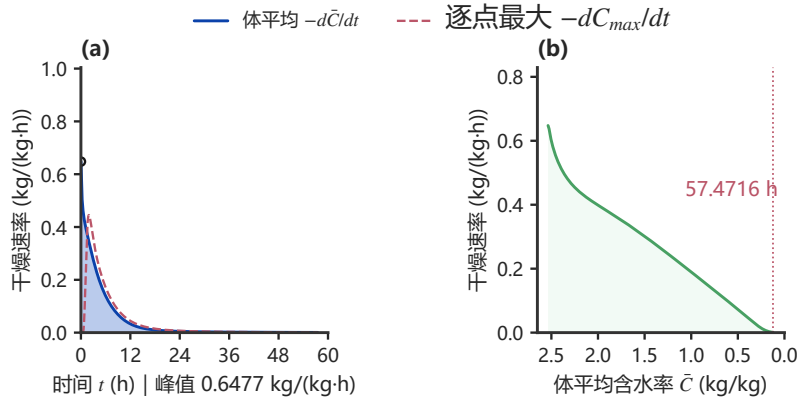


图 12 问题三干燥速率的两种读法

完成。标注时刻 57.4716 h 与式 (18) 一致，且与问题二来自同一次连续积分。图 12(a) 的速率峰值 0.6477 kg/(kg·h) 出现在首个输出时刻 $t = 0.0167$ h，此后单调衰减，无升速段；(b) 按 Krischer 方式对 $\bar{C}$ 作图，无水平段，即无恒速段。全过程由内部扩散控制^[6]。

按题目要求，全过程每隔 60 s、距中心每隔 0.1 cm 的含水率完整结果保存于结果文件，覆盖从初始时刻到 t_{end} 的整个过程。

九、问题四：收缩移动边界模型与收缩效应的定量隔离

9.1 相对问题三的两处改动

本问相对问题三改变了两件事：物性关系由式 (7) 换成式 (8)，求解域由固定半径换成随时间收缩的 $r \in [0, R(t)]$ 。达标判据式 (15) 与插值定位式 (17) 保持不变，边界条件式 (3) 的形式不变但作用位置随 $R(t)$ 移动。两处改动中，物性只是系数替换，域的变化才是本质困难。

干基含水率 C 以干物质质量为基准。药材收缩时，固体骨架携带结合水一起向内运动，因此 C 首先是物质点上的场。若直接在物理坐标上离散，每步都要重新生成网格并插值，既破坏离散守恒，又难以区分误差来源。坐标变换的目的是让计算坐标跟着物质点走，把动域化为定域。

9.2 物质坐标与归一化坐标

引入归一化径向坐标

$$\xi = \frac{r}{R(t)} \in [0, 1], \quad (19)$$

并记

$$\eta = (r/R(t))^2 = \xi^2 \in [0, 1]. \quad (20)$$

均匀收缩假设下，固相速度取仿射场

$$v_s(r, t) = r \dot{R}(t)/R(t), \quad (21)$$

它是使 ξ （因而也使 η ）对每个干物质质点保持不变的径向速度。因此，定 ξ 求导与定 η 求导给出同一物质导数。式 (19) 把随时间收缩的物理域映成端点恒为 0 与 1 的固定计算域，这一处理是收缩干燥建模的通行做法^[2, 13-16]。

若先把前三问的经验扩散写在实验室坐标 r 上，再变到 ξ ，定 ξ 求导与定 r 求导并不相同，两者相差一项

$$\left. \frac{\partial \varphi}{\partial t} \right|_r = \left. \frac{\partial \varphi}{\partial t} \right|_\xi - \frac{\xi \dot{R}(t)}{R(t)} \frac{\partial \varphi}{\partial \xi}, \quad (22)$$

其中 φ 泛指 T 或 C 、 $\dot{R} = dR/dt \leq 0$ 。把式 (22) 代入式 (1) 与式 (2)，得到计算域上的方程

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{R^2} \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(\xi k \frac{\partial T}{\partial \xi} \right) + (1 - \chi) \rho c_p \frac{\xi \dot{R}}{R} \frac{\partial T}{\partial \xi}, \quad (23)$$

$$\frac{\partial C}{\partial t} = \frac{1}{R^2} \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(\xi D \frac{\partial C}{\partial \xi} \right) + (1 - \chi) \frac{\xi \dot{R}}{R} \frac{\partial C}{\partial \xi}, \quad (24)$$

表面边界条件相应变为

$$-\left. \frac{k}{R} \frac{\partial T}{\partial \xi} \right|_{\xi=1} = h[T(1, t) - T_{\text{air}}], \quad -\left. \frac{D}{R} \frac{\partial C}{\partial \xi} \right|_{\xi=1} = h_m[C(1, t) - C_{\text{air}}]. \quad (25)$$

式 (23) 与式 (24) 中扩散项前多出 $1/R^2$ 因子，它表示同一无量纲梯度在更细的半径上对应更陡的物理梯度——这正是收缩缩短扩散路径的数学体现。末项是由边界移动引起的表观对流项，物理上并非真实流动，而是坐标随时间变形的结果。式 (25) 中的 $1/R$ 同样来自 $r = R(t)\xi$ 的坐标变换：同一无量纲空间梯度对应更大的物理空间梯度；同时表面积-体积几何关系随半径缩小而变化，使表面交换相对于内部储存的作用增强。

本文将 $\chi = 1$ 与 $\chi = 0$ 视为两种运动学/输运闭合，而非同一守恒律的简单坐标表示。 $\chi = 1$ 时物质点与计算坐标同步内缩，对流项系数为 0，式 (24) 即与收缩骨架随动假设一致的主闭合。由 $\eta = \xi^2$ 换元，空间导数满足 $\partial C / \partial \xi = 2\xi \partial C / \partial \eta$ ，于是

$$\frac{\partial C}{\partial t} = \frac{4}{R(t)^2} \frac{\partial}{\partial \eta} \left(\eta D(C, T) \frac{\partial C}{\partial \eta} \right), \quad (26)$$

表面边界化为

$$-\left. \frac{2D}{R} \frac{\partial C}{\partial \eta} \right|_{\eta=1} = h_m[C(1, t) - C_{\text{air}}]. \quad (27)$$

能量方程同理，只把 D 换成 k 、左端乘 ρc_p 。式 (26) 与 $\chi = 1$ 时的式 (24) 是同一条方程，并不是另一套水分守恒形式；离散实现仍在均匀 ξ 网格上装配，因为仿射收缩下每个 ξ_i 已经跟着物质点走。

$\chi = 0$ 保留表观对流，对应另一种把经验扩散写在实验室坐标后再变换到 ξ 的替代闭合。正式结果采用与收缩固体骨架随动假设一致的 $\chi = 1$ 闭合：附件 2 的外轮廓半径用于给定移动边界，内部骨架位移按仿射场式 (21) 闭合。 $\chi = 0$ 仅用于考察预测时长对运输闭合方式的敏感性，两者之差单独报告，而不合并成单一数字。映射的几何含义见图 13。

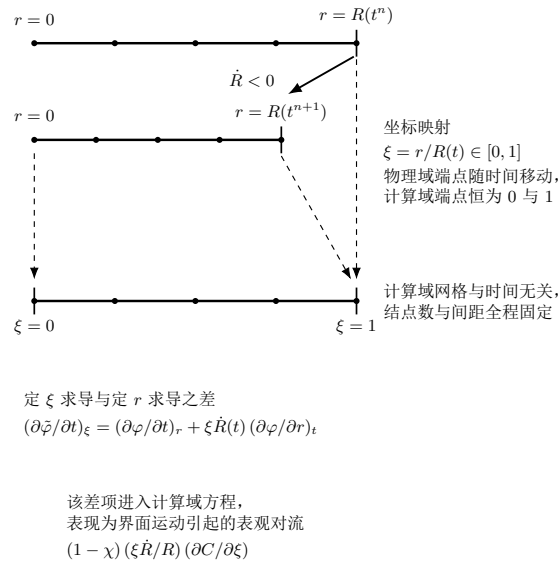


图 13 归一化坐标把收缩物理域映为固定计算域

$\chi = 0$ 时表观对流须用一阶迎风： $\dot{R} < 0$ 使对流指向 ξ 减小的方向；表面附近网格 Péclet 数可超过 2，中心差分会破坏对角占优。

9.3 半径时程的构造

式 (23)–(24) 需要 $R(t)$ 与 $\dot{R}(t)$ 。附件 2 给出 145 个半径实测点，全部参与插值。由假设 5 收缩不可逆，先对实测序列作累积取最小值处理以保证单调不增，再用保形分段三次插值构造连续曲线，其解析导数即 $\dot{R}(t)$ 。选择保形插值而非普通三次样条的理由是后者可能在数据点之间产生过冲，使 $R(t)$ 局部上升、 $\dot{R}$ 变号，与收缩不可逆矛盾^[17]。对实测半径采用保形 PCHIP 插值， $\dot{R}(t)$ 由插值函数求导，见图 14。

半径由 2.000 cm 降至 1.198 cm，末态体积约为初始的 36%。单调化前后最大偏差仅 2.2×10^{-16} cm，实测本身已经单调； $\dot{R}$ 绝对值在早期最大，收缩影响主要在前期。

9.4 烘干时长与含水率分布

在物质坐标 ($\chi = 1$) 下求解，正式配置取 $N = 1280$ 、 $\Delta t = 0.25$ s，得到烘干时长

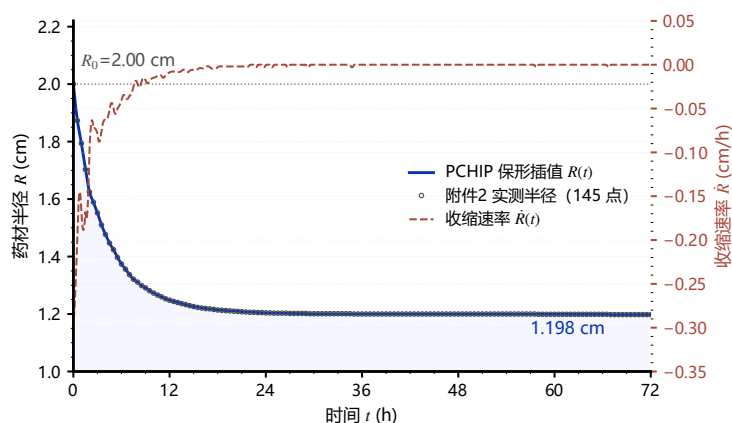


图 14 附件 2 收缩半径的实测散点、插值与收缩速率

$$t_{\text{end}} = 51.0869 \text{ h} \approx 2.13 \text{ d}, \quad (28)$$

该时刻逐点最大含水率为 0.1500 kg/kg ，末态半径 $R(t_{\text{end}}) = 1.2000 \text{ cm}$ 。由于 51.0869 h 落在附件 2 数据覆盖范围之内，半径全程由实测插值给出，不涉及数据用尽后的外推。各时刻含水率分布见表 6。

表 6 考虑收缩时药材内部水分浓度 (kg/kg)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2	药材表面
6	1.7188	1.5370	1.0221	—	—	0.4205
12	0.7374	0.6544	0.4082	—	—	0.1669
18	0.4086	0.3686	0.2397	—	—	0.0892
24	0.2851	0.2610	0.1790	—	—	0.0673
30	0.2263	0.2094	0.1492	—	—	0.0595
36	0.1928	0.1796	0.1316	—	—	0.0560
42	0.1712	0.1604	0.1200	—	—	0.0541
48	0.1561	0.1468	0.1116	—	—	0.0530
烘干结束时间 51.0869 h	0.1500	0.1413	0.1081	—	—	0.0526

表中 1.5 cm 与 2 cm 两列自第一个输出时刻起即为空，这不是缺算而是几何事实： $t = 6 \text{ h}$ 时半径已收缩至 1.374 cm ，这两个位置已落在药材之外，不存在含水率可言。按题目表格要求，末列单独给出随时间移动的药材表面处的值。把本表与表 5 对照可以看到， 6 h 时中心含水率为 1.7188 kg/kg ，明显高于问题三同时刻的 1.0170 kg/kg ，即前期收缩情形反而干得更慢，因为附录 4 的扩散能力更低；但到 18 h 两者已接近（ 0.4086 对 0.2988 ），最终本问反而先达标。这一交叉说明两处改动的作用方向相反，必须分离。

收缩域内的场演化与分阶段剖面合见图 15 与图 16。

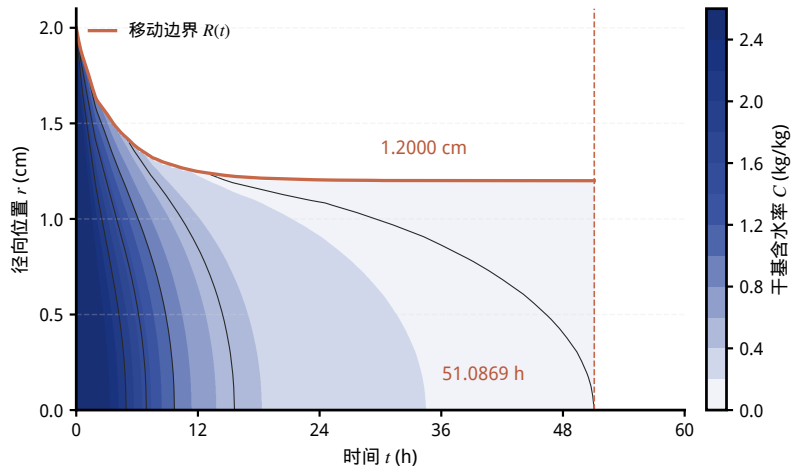


图 15 问题四收缩域内含水率的时空分布与移动边界

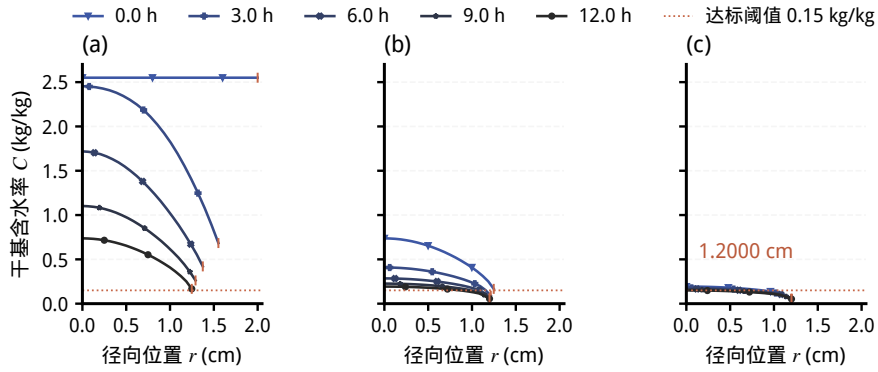


图 16 问题四含水率剖面的三阶段演化

图 15 的等高线绘于物理坐标，外侧留空，红线 $R(t)$ 由 2.000 cm 缩至 1.200 cm。域缩小缩短扩散路径，附录 4 的低扩散系数则相反，两项贡献须由对照实验分离。图 16 横轴为物理半径，右端点即当时表面：(a) 0–12 h 收缩最快而剖面仍陡；(b) 12–36 h 整体下移；(c) 趋于平坦并触及阈值，末态半径 1.200 cm，与图 14 中前期快、后期缓的 $\dot{R}$ 一致。

9.5 收缩效应的定量隔离

式 (28) 比问题三的 57.4716 h 短 6.3847 h (11.1094%)。但本问同时改变了物性与求解域，把这一差值归因于「收缩使干燥加快」是混淆归因。为分离两项作用，在完全相同的离散参数、环境驱动与达标判据下构造情景链，每次只改变一个因素。情景 S_0 为共同网格诊断基准（附录 3 物性、固定域）， S_1 只把物性换成附录 4 而仍保持固定域， S_2 在 S_1 之上再引入收缩即本问的物质坐标主模型。三者之差满足加和恒等式

$$\underbrace{t_{S_2} - t_{S_0}}_{\text{净效应}} = \underbrace{(t_{S_1} - t_{S_0})}_{\text{换物性}} + \underbrace{(t_{S_2} - t_{S_1})}_{\text{加收缩}}. \quad (29)$$

式 (29) 之所以成立是构造使然而非物理定律：三个情景两两之间只差一个因素，差值可以逐段相加。此外设 S_3 为 $\chi = 0$ 的替代闭合， S_4 为以末态半径 1.198 cm 固定求解的情形，后者给出理想化小半径参照。需要特别说明 S_1 的性质：它用附录 4 的慢扩散却不允许收缩，物理上并不自洽，是人为构造的对照情景，其时长不作为任何交付结论。五个情景的结果列于表 7。

表 7 效应隔离情景与烘干时长

情景	物性关系	求解域	t_{end} (h)
S_0	附录 3	固定 $R = 2$ cm（共同网格诊断基准）	57.4693
S_1	附录 4	固定 $R = 2$ cm（对照，非交付）	129.8423
S_2	附录 4	收缩域，物质坐标 $\chi = 1$ （正式主模型）	51.0869
S_3	附录 4	收缩域， $\chi = 0$ （替代闭合）	52.6476
S_4	附录 4	固定 $R = 1.198$ cm（理想化小半径参照）	47.7985

共同网格分解见表 7，满足式 (29)：换物性 +72.3731 h、加收缩 -78.7554 h、净效应 -6.3824 h。正式比较仍用问题三 57.4716 h 对问题四 51.0869 h，提前 6.3847 h (11.1094%)。 S_0 不替代问题三。若只看净差就断言「收缩对烘干时长影响不大」，仍会混淆物性更换与求解域变化。分解的量值构成见图 17。

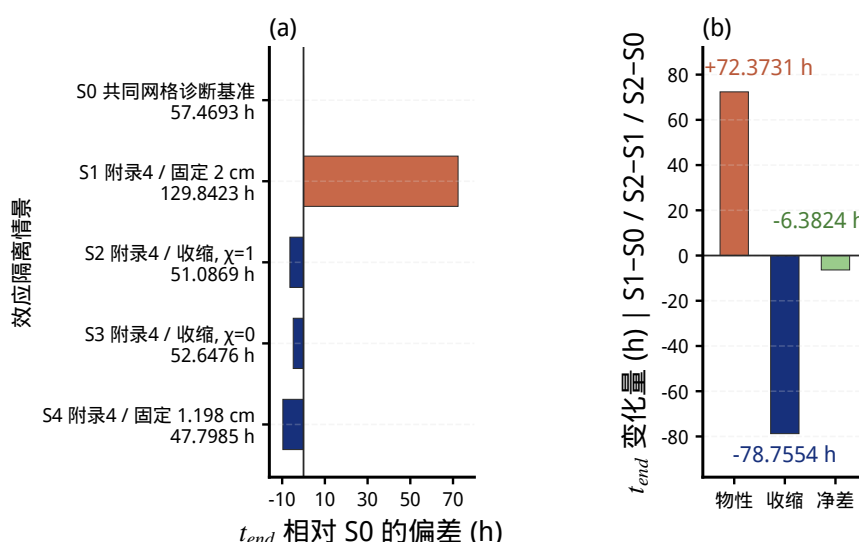


图 17 问题四的效应隔离与加性分解（共同网格诊断）

面板 (a) 与面板 (b) 保留原情景结构，柱高与贡献值采用最终效应分解诊断数据。 $S_3 = 52.6476$ h 与 S_2 的差为 +1.5607 h； $S_4 = 47.7985$ h 为理想化细半径参照。五个情景满足 $S_4 < S_2 < S_3 < S_0 < S_1$ ；其中 S_0 与 S_1 只服务于共同网格效应分解，不作为正式问题三结果。

9.6 闭合方式的不确定度与数据自洽性诊断

$\chi = 1$ 主闭合给出 51.0869 h, $\chi = 0$ 替代闭合给出 52.6476 h, 相差 1.5607 h 即 3.0550%。两种闭合分别给出上述时长, 说明预测时长对运动学闭合具有一定敏感性; 二者不构成严格不确定区间。

最后需要报告一项题给数据本身的不相容。若干物质在收缩过程中守恒, 则干物质表观密度 $\rho_s = \rho(C)/(1+C)$ 与半径必须满足 $\rho_s(C_{th})/\rho_s(C_0) = (R_0/R_{end})^2$ 。按式 (8) 计算左端得 2.4131, 而附件 2 的实测半径给出右端 2.7871, 两者相差 13.42%; 反过来由密度关系反演的末态半径为 1.2875 cm, 实测值为 1.1980 cm, 相差 7.47%。题目要求「根据附件 2 确定烘干时长」, 故本文把实测 $R(t)$ 作为给定的移动边界直接使用, 上述不相容度只作为题给数据的诊断量报告, 既不用它修正实测半径, 也不把它当作求解的收敛判据, 更不把它折算为时长修正。该不一致对时长的影响需要另算, 本文未另算; 闭合敏感性给出的 3.0550% 不能代替这项未计算的影响。

十、数值可信度与参数灵敏度

本题没有药材内部温度与含水率的实测剖面, 附件只给出烘房环境与外轮廓半径。以下证据是模型的内部一致性与数学正确性, 不能解释为与实测吻合; 唯一具有外部性质的比对是问题一的级数解, 它属于同一模型的独立解法而非独立观测^[18]。

10.1 离散守恒与收敛性

把控制体内含水量的累积变化与表面净流出相比, 相对残差为

$$\varepsilon = \frac{\left| \sum_i V_i [C_i(t) - C_i(0)] + \int_0^t 2\pi R h_m [C(R, \tau) - C_{air}(\tau)] d\tau \right|}{\sum_i V_i C_i(0)}. \quad (30)$$

式 (30) 检验的是离散方程中储存项与控制体通量的数值平衡, 而非真实系统总水分质量守恒误差。问题一、问题三与问题四的残差分别为 7.41×10^{-16} 、 1.55×10^{-14} 与 7.51×10^{-16} , 均已接近双精度机器精度。正式配置见公共模型末段。达标时长对网格与步长的敏感性见图 18。

面板 (a) 为问题三与问题四在 $\Delta t = 1$ s 下由 $N = 40$ 至 $N = 2560$ 的空间收敛; 面板 (b) 为各自正式配置下的时间步收敛。对应达标时长分别为 57.4716 h 与 51.0869 h。

10.2 物理合理性与极限退化

四问结果一致满足: 含水率对时间单调不增; 同一时刻中心含水率不低于表面、中心温度不高于表面; 全场有界且无负值, 正式结果含水率不低于 0.0526, 4 h 后环境稳

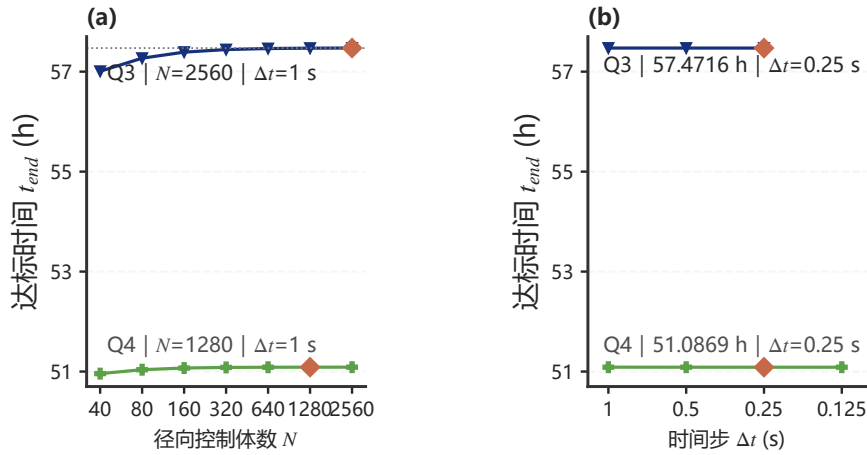


图 18 问题三与问题四的空间和时间步收敛性

态外推为 $50.0000\text{ }^{\circ}\text{C}$ 与 0.0500 kg/kg 。问题一的「热先于质」亦得到确认。中心温度在环境小幅回落时出现 $5.2 \times 10^{-5}\text{ }^{\circ}\text{C}$ 的极小回落，幅度小于同期环境回落 $7.5 \times 10^{-3}\text{ }^{\circ}\text{C}$ 的百分之一，属边界扰动向内衰减，而非时间推进振荡。

移动边界还须在退化情形回到固定域。令 $\dot{R} \equiv 0$ ，式 (23)–(24) 的表观对流恒为零， $\chi = 1$ 与 $\chi = 0$ 给出相同达标时长。另令 h_m 取很大值，1 h 时表面含水率与 C_{air} 之差为 1.7×10^{-15} ，说明式 (3) 的方向与符号正确。

10.3 参数灵敏度

参数分三类：题给确定量、经验公式中带拟合不确定性的量，以及建模选择引入的量。对第二、三类作一次一因子扰动，达标时长响应见图 19。

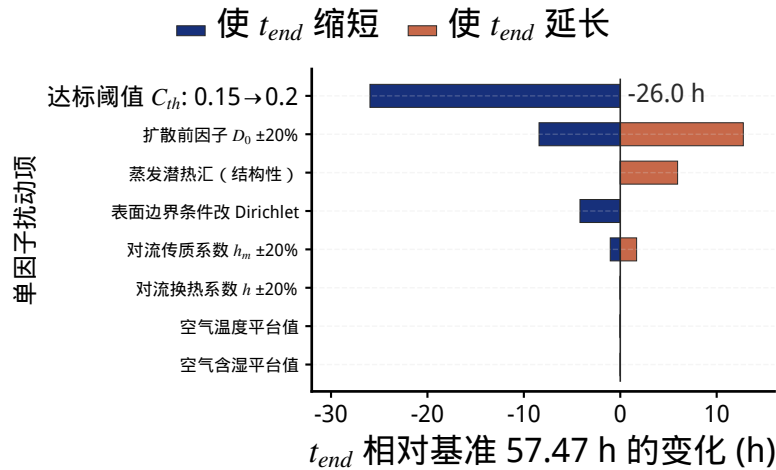


图 19 达标时长的最终精度单因子灵敏度诊断

最终精度诊断中，扩散系数前因子 $\pm 20\%$ 带来 -8.43 至 $+12.76$ h 的变化，弹性约 -0.73 至 -1.11 ，是最强不确定度来源；对流传质系数同幅扰动为 -1.04 至 $+1.70$ h，弹性约 -0.09 至 -0.15 ；对流换热系数仅 0.02 h 量级，弹性约 -0.001 至 -0.002 。排序印证本题由内部扩散控制，表面换热几乎不影响时长，精度关键在扩散系数经验式而非对

流系数。以附件 1 末 30 个样本均值 50.0105°C 和 0.049982 kg/kg 替代外推值后，时长变化均低于 0.04%，说明结果对该外推不敏感。

达标阈值是工艺规定而非模型参数。基于 `result3.xlsx` 的 60 s 正式输出作线性插值后处理，阈值由 0.15 改到 0.20 kg/kg 会使时长缩短 25.96 h，即近一半。工艺上把安全水分放宽 0.05 kg/kg ，时间收益远大于模型改进——前提是贮藏安全性允许。

结构性选择一并探查。改用第一类边界会缩短时长。取汽化潜热近似 $\lambda = 2.4 \times 10^6\text{ J/kg}$ ，计入潜热后问题三时长由 57.4716 h 增至 63.4189 h，增幅约 10.35%，说明忽略潜热会使预测偏短。该计算只评估方向与量级，不作为正式结果的精确修正。两项探查都不改变交付结果：题面已给出对流系数（即第三类边界），也未给出潜热数据。

十一、模型评价与推广

11.1 模型的适用范围与优势

四问共用一套柱坐标径向热-质耦合方程，差别只在物性闭合、求解域与终止判据。由此：问题一的半解析对照独立验证了后续共用的柱坐标有限体积离散、时间推进及第三类边界；问题二至四新增的非线性物性、耦合和移动边界则分别经迭代收敛、网格与步长加密及退化测试检验。问题三与问题四在同一达标判据下可直接比较，换物性与加收缩的分解见表 7。

分布参数与第三类边界的依据见问题一：毕奥数为 $O(1)$ ，既不能当作集中参数，也不能把表面固定为环境值。移动边界采用物质坐标 $\xi = r/R(t)$ ， $\dot{R} \rightarrow 0$ 时精确退化为固定域格式。

11.2 本文的创新点

第一，区分问题四相对问题三的“换物性”与“加收缩”。正式时长相减得 -6.3847 h ，只说明问题四整体更快 11.1094%，不能单独归因于收缩。共同网格诊断以 $S_0 = 57.4693\text{ h}$ 为基准，给出换物性 $+72.3731\text{ h}$ 、加收缩 -78.7554 h 与净效应 -6.3824 h ；两项符号相反且量级远大于净差。 S_0 不替代正式问题三结果。

第二，达标判据。以体平均判定时， $\bar{C}$ 会在芯部远未达标时先跨过阈值： $C_{\max}$ 恰触 0.15 kg/kg 时体平均仅 0.1232 kg/kg ；改用体平均会在芯部尚有约 0.18 kg/kg 时宣布完成。本文按“内部含水率降到 0.15 kg/kg 以下”取逐点最大值，并由中心对称与空间单调性确定控制点为轴心。此处改变的是答案本身，而不是答案的精度。

第三，把收缩时的运动学闭合写清楚并报出后果。均匀仿射收缩使 $\xi = r/R(t)$ 成为物质坐标，主闭合不再含表观对流。两种形式视为不同运动学/输运闭合。正式结果取与骨架随动一致的 $\chi = 1$ ； $\chi = 0$ 仅作敏感性对照，分别得 51.0869 h 与 52.6476 h ，相差 1.5607 h （3.0550%）。

第四，附录 4 的 $\rho(C)$ 与附件 2 的实测 $R(t)$ 在严格干物质守恒下并不自洽，干基密度比相差 13.42%。本文依题面把 $R(t)$ 作为给定移动边界，把离散守恒残差降格为诊断量，既不修正实测半径，也不回避该矛盾。该不一致表明问题四存在结构性不确定性，其对达标时长的定量影响尚需进一步计算。

11.3 模型的局限

第一，能量方程未设蒸发潜热汇，会高估内部温度与扩散系数、使预测偏短。§10.3 确认偏向约 10.35%，但不作为正式结果；57.4716 h 与 51.0869 h 应理解为乐观短侧估计。题给物性式不含潜热参数，本文保持题给闭合完整。

第二，问题四主模型由“ C 是物质点上的场”和仿射骨架式 (21) 闭合；替代闭合相差约 3%，不改变量级，详见问题四末节。

第三，附录 4 密度式与附件 2 实测半径不相容。题面要求根据附件 2 确定收缩规律，实测 $R(t)$ 作为给定移动边界，不相容度只作诊断，详见问题四末节。

第四，灵敏度采用单因素扰动。 D 前因子 $\pm 20\%$ 引起 -8.43 h 与 $+12.76$ h，弹性介于 -0.73 与 -1.11 ，是最强一项； h_m 弹性约 -0.1 ， h 仅 -0.001 。结果只反映主效应，未覆盖 D 前因子与 $\exp(-3850/T_K)$ 同时偏离等交互。缺少内部实测剖面，验证只能依靠离散守恒残差、收敛性、半解析对照与极限退化，参数未经标定。

11.4 结论与推广

问题一：热扩散显著快于质扩散。1800 s 时中心 $33.5755\text{ }^\circ\text{C}$ / 2.5500 kg/kg ，表面 $36.7856\text{ }^\circ\text{C}$ / 1.5103 kg/kg ，时间尺度之比约 34。问题二：变物性形成双向耦合，3 h 时中心温度 $49.8495\text{ }^\circ\text{C}$ 已接近烘房温度，中心含水率仍为 1.7662 kg/kg 。问题三：按逐点最大含水率判定，烘干时长 57.4716 h。问题四： $\chi = 1$ 主闭合 51.0869 h，比问题三缩短 6.3847 h (11.1094%)；效应隔离表明换物性与加收缩作用方向相反，不能单侧归因。同一求解核可移用到其他给出物性闭合与外形收缩数据的柱状物料。 D 前因子弹性接近 -1 而 h 近于零，缩短时长应提高介质温度以抬升扩散系数，而不是加大风速强化表面换热。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于排版检查、代码调试，详细使用情况见支撑材料。

参考文献

- [1] ROCHA R P. Influence of drying process on the quality of medicinal plants: A review [J/OL]. Journal of Medicinal Plants Research, 2011, 5(33). DOI: 10.5897/jmprx11.001.

- [2] MAYOR L, SERENO A M. Modelling shrinkage during convective drying of food materials: a review[J/OL]. Journal of Food Engineering, 2004, 61(3): 373-386. DOI: 10.1016/S0260-8774(03)00144-4.
- [3] SRIKIATDEN J, ROBERTS J S. Moisture transfer in solid food materials: A review of mechanisms, models, and measurements[J/OL]. International Journal of Food Properties, 2007, 10(4): 739-777. DOI: 10.1080/10942910601161672.
- [4] TZEMPELIKOS D A, MITRAKOS D, VOUIROS A P, et al. Numerical modeling of heat and mass transfer during convective drying of cylindrical quince slices[J/OL]. Journal of Food Engineering, 2015, 156: 10-21. DOI: 10.1016/j.jfoodeng.2015.01.017.
- [5] DA SILVA W P, E SILVA C, GOMES J P. Comparison of boundary conditions to describe drying of turmeric (*Curcuma longa*) rhizomes using diffusion models[J/OL]. Journal of Food Science and Technology, 2014, 51(11): 3181-3189. DOI: 10.1007/s13197-012-0813-x.
- [6] SANDER A, TOMAS S, SKANSI D. The influence of air temperature on effective diffusion coefficient of moisture in the falling rate period[J/OL]. Drying Technology, 1998, 16(7): 1487-1499. DOI: 10.1080/07373939808917472.
- [7] PATANKAR S V. Numerical heat transfer and fluid flow[M/OL]. CRC Press, 2018. DOI: 10.1201/9781482234213.
- [8] MIGNONE A. High-order conservative reconstruction schemes for finite volume methods in cylindrical and spherical coordinates[J/OL]. Journal of Computational Physics, 2014, 270: 784-814. DOI: 10.1016/j.jcp.2014.04.001.
- [9] DALL'OSSO A. An iterative back substitution algorithm for the solution of tridiagonal matrix systems with fringes[J/OL]. Journal of Computational and Applied Mathematics, 2004, 169(1): 87-99. DOI: 10.1016/j.cam.2003.11.004.
- [10] HUNDSDORFER W, VERWER J. Numerical solution of time-dependent advection-diffusion-reaction equations[M/OL]. Berlin, Heidelberg: Springer, 2003. DOI: 10.1007/978-3-662-09017-6.
- [11] GUELLAL M, SADAT H, PRAX C. A simple model for transient heat conduction in an infinite cylinder with convective boundary conditions[J/OL]. Journal of Heat Transfer, 2009, 131(5). DOI: 10.1115/1.3082428.
- [12] DOYMAZ I. Air-drying characteristics, effective moisture diffusivity and activation energy of grape leaves[J/OL]. Journal of Food Processing and Preservation, 2011, 36(2): 161-168. DOI: 10.1111/j.1745-4549.2011.00557.x.
- [13] MISRA R N, YOUNG J H. Numerical solution of simultaneous moisture diffusion and shrinkage during soybean drying[J/OL]. Transactions of the ASAE, 1980, 23(5): 1277-1282. DOI: 10.13031/2013.34760.
- [14] BENNAMOUN L, CRINE M, LÉONARD A. Convective drying of wastewater sludge: Introduction of shrinkage effect in mathematical modeling[J/OL]. Drying Technology, 2013, 31(6): 643-654. DOI: 10.1080/07373937.2012.752743.
- [15] RAHMAN N, KUMAR S. Evaluation of moisture diffusion coefficient of cylindrical bod-

- ies considering shrinkage during natural convection drying[J/OL]. International Journal of Food Engineering, 2011, 7(1). DOI: 10.2202/1556-3758.1352.
- [16] ADROVER A, BRASIELLO A, PONSO G. A moving boundary model for food isothermal drying and shrinkage: General setting[J/OL]. Journal of Food Engineering, 2019, 244: 178-191. DOI: 10.1016/j.jfoodeng.2018.09.018.
- [17] FRITSCH F N, CARLSON R E. Monotone piecewise cubic interpolation[J/OL]. SIAM Journal on Numerical Analysis, 1980, 17(2): 238-246. DOI: 10.1137/0717021.
- [18] PELLETIER D, ROACHE P J. Verification and validation of computational heat transfer[J/OL]. Handbook of Numerical Heat Transfer, 2000: 417-442. DOI: 10.1002/9780470172599.ch13.

附录 A 支撑材料清单

本文全部数值结果由下列 Python 源程序真实计算生成，入口为 `main.py`，运行环境为 Python 3 与 NumPy、SciPy、pandas、openpyxl。程序不含任何硬编码的结果数值，四个子问题的答案均由同一求解核在不同物性闭合与终止判据下计算得到。

表 8 支撑材料文件清单

类别	文件名	内容
程序入口	<code>main.py</code>	四问求解编排与结果写出
	<code>driver.py</code>	时间推进、算子分裂与 Picard 迭代
	<code>fvm.py</code>	柱坐标有限体积离散与三对角求解
	<code>props.py</code>	附录 2-4 物性闭合式
	<code>ambient.py</code>	附件 1 环境温湿度插值
	<code>geometry.py</code>	附件 2 半径单调化与 PCHIP 插值
	<code>params.py</code>	题给参数与数据读入
	<code>problem1.py</code>	问题一预热段求解与报表
	<code>problem2.py</code>	问题二全程变物性求解与报表
	<code>problem3.py</code>	问题三达标时长求解与报表
	<code>problem4.py</code>	问题四收缩域求解、效应隔离与干物质核算
	<code>master_p23.py</code>	问题二、三共用的主求解流程

类别	文件名	内容
	<code>validate.py</code>	历史粗网格诊断（非正式结果，见 <code>output/legacy/</code> ）
	<code>io_out.py</code>	报表与工作簿写出
	<code>data_check.py</code>	附件数据完整性核对
	<code>template_check.py</code>	附件 3 模板结构核对
计算结果	<code>result1.xlsx</code>	问题一温度与水分浓度全网格结果
	<code>result2.xlsx</code>	问题二前 3 h ($1 \sim 10800$ s) 温度与含水率全网格
	<code>result3.xlsx</code>	问题三 $0 \rightarrow t_{\text{end}}$ 水分浓度全网格
	<code>result4.xlsx</code>	问题四物质坐标主模型 ($\chi = 1$) 收缩域水分浓度全网格
	<code>final_diagnostics/q4_effect_isolation_final.csv</code>	情景 S0–S4 达标时长
	<code>mass_consistency.csv</code>	干物质守恒诊断
	<code>final_diagnostics/q3_sensitivity_final.csv</code>	单因子灵敏度记录
	<code>legacy/validation.json</code>	历史粗网格诊断（非正式结果）

附录 B 程序源代码

下列代码为完整可运行源程序，按依赖层次排列：先给出参数与数据读入，再给出离散核与物性、时间推进与求解编排，最后给出四问的调用与验证。运行方式为在 `workspace/` 目录下执行 `python code/main.py`，程序自动读入附件并把全部报表与工作簿写入 `output/`。旧 `validate.py` 不再随正式流程自动执行。

2.1 参数与数据读入：params.py

```

1  """全局参数口径表（唯一常量来源）。
2
3  所有物理常量、附录系数、数值离散参数在此集中定义，逐项对应
4  `PROBLEM_FACTS.json` / `DATA_FACTS.json` / `MODELING_REPORT.md §12` 的口径表。
5  其余模块一律 `from params import *` 或 `import params`，不得另行硬编码物理常量。
6
7  命名约定：附录系数以 APPn_ 前缀区分；幂次用 mantissa * 10.0**power 显式书写，
8  使尾数与题面 mantissa 字段一致（题面上标  $8 \times 10^{-7}$  等）。
9  """
10 from __future__ import annotations
11
12 from pathlib import Path
13
14 # -----
15 # 数据文件路径（相对工作区根目录；run_compute 已将根目录加入 sys.path 且 cwd=root）
16 # -----
17 ROOT = Path(__file__).resolve().parent.parent
18 DATA_DIR = ROOT / "user_data" / "附件" / "附件"
19 ATTACH1 = DATA_DIR / "附件1.xlsx"          # 时间 | 温度 | 空气水分浓度 (241 行)
20 ATTACH2 = DATA_DIR / "附件2.xlsx"          # 时间 | 半径 (145 行)
21 TEMPLATE_DIR = DATA_DIR / "附件3"
22 OUTPUT_DIR = ROOT / "output"
23 FIGURE_DIR = ROOT / "figures"
24
25 # 数据行数断言 (DATA_FACTS ingest_assertion)
26 N_ROWS_ATTACH1 = 241
27 N_ROWS_ATTACH2 = 145
28
29 # -----
30 # 几何（题面正文：长 25 cm、半径 2 cm）—— 进入模型换算为 SI (m)
31 # -----
32 R0_CM = 2.0
33 L_CM = 25.0
34 CM_TO_M = 1.0 / 100.0
35 R0_M = R0_CM * CM_TO_M          # 0.02 m
36 L_M = L_CM * CM_TO_M           # 0.25 m
37
38 # -----
39 # 初始条件（题面正文）
40 # -----
41 T0_C = 28.0                     # 初始温度 °C
42 C0 = 2.55                      # 初始干基含水率 kg/kg
43
44 # -----
45 # 边界系数（附录2，规则 R6 四问共用）
46 # -----
47 H_CONV = 25.0                  # 对流换热系数 W/(m^2 · K)
48 HM_CONV = 8.0 * 10.0 ** (-7)   # 对流传质系数 m/s（题面  $8 \times 10^{-7}$ ）
49
50 # -----
51 # 达标阈值（题面问题3：各处 < 0.15）
52 # -----
53 C_TH = 0.15                    # kg/kg
54
55 # -----
56 # 绝对温度换算 (derived_formulas)

```

```

57 # -----
58 T_KELVIN = 273.15
59
60 # -----
61 # 物性系数（三套附录严格按问题隔离，MODELING_REPORT §4）
62 # 附录2（仅问题1）：rho/cp/k 常数；D=7e-9·exp(-0.89/C)，不含温度项
63 # -----
64 APP2_RHO = 820.0 # kg/m^3
65 APP2_CP = 2600.0 # J/(kg·K)
66 APP2_K = 0.36 # W/(m·K)
67 APP2_D_PREF = 7.0 * 10.0 ** (-9) # m^2/s
68 APP2_D_EC = 0.89 # exp(-0.89/C)
69
70 # 附录3（问题2、3）：四条公式全变；rho 用干基 C，cp/k 用湿基 w
71 APP3_RHO_A = 650.0
72 APP3_RHO_B = 128.0 # rho = 650 + 128*C
73 APP3_CP_A = 1450.0
74 APP3_CP_B = 2736.0 # cp = 1450 + 2736*w
75 APP3_K_A = 0.21
76 APP3_K_B = 0.38 # k = 0.21 + 0.38*w
77 APP3_D_PREF = 2.4 * 10.0 ** (-3) # m^2/s
78 APP3_D_EC = 0.45 # exp(-0.45/C)
79 APP3_D_ET = 3850.0 # exp(-3850/T_K)
80
81 # 附录4（仅问题4）：结构同附录3、系数不同
82 APP4_RHO_A = 760.0
83 APP4_RHO_B = 90.0 # rho = 760 + 90*C
84 APP4_CP_A = 1850.0
85 APP4_CP_B = 2150.0 # cp = 1850 + 2150*w
86 APP4_K_A = 0.12
87 APP4_K_B = 0.20 # k = 0.12 + 0.20*w
88 APP4_D_PREF = 4.2 * 10.0 ** (-4) # m^2/s
89 APP4_D_EC = 0.30 # exp(-0.30/C)
90 APP4_D_ET = 3850.0 # exp(-3850/T_K)
91
92 # -----
93 # 环境平台外推值（附件1 数据范围结束后的长期边界）
94 # -----
95 TA_PLATEAU = 50.0 # °C
96 CA_PLATEAU = 0.0500 # kg/kg
97 AMBIENT_T_MAX_S = 14400.0 # 附件1 时间轴上界（含）s
98 # 灵敏度替代值（末30点均值，SA4/SA5）
99 TA_PLATEAU_ALT = 50.0105
100 CA_PLATEAU_ALT = 0.049982
101
102 # -----
103 # 潜热汇（假设4，关闭；SA8 可探查）
104 # -----
105 LATENT_HEAT_SINK = 0.0 # J/kg
106 LATENT_HEAT_PROBE = 2.4 * 10.0 ** 6 # SA8 探查值
107
108 # -----
109 # 数值离散（MODELING_REPORT §12 参数表 + §11 步长策略）
110 # -----
111 N_CV = 40 # 历史/兼容默认（旧 validate.py 粗网格）；正式网格见下列 N_P*_CV
112 N_P1_CV = 160 # 问题1最终控制体数

```

```

113 N_P4_CV = 1280 # 问题4最终控制体数 (ALE  $\chi=1$ )
114 N_P23_CV = 2560 # 问题2/3统一连续生产积分空间网格
115 C_FLOOR = 1.0 * 10.0 ** (-3) # 浓度下限保护 (§11 第5步, C_TH)
116 D_FLOOR = 1.0 * 10.0 ** (-14) # D 数值下限 (§13.3 异常预案)
117 CHI_PRIMARY = 0 # 底层兼容默认; 不代表问题4正式配置
118 CHI_ALT = 1 # 底层兼容变量; 问题4使用下列专用常量
119 CHI_P4_PRIMARY = 1 # 问题4主模型: 材料网格随药材同步收缩
120 CHI_P4_CROSSCHECK = 0 # 问题4交叉验证: Eulerian/Landau 闭合
121
122 # 时间步长 (s)
123 DT_P1 = 0.125 # 问题1最终内部时间步 (输出仍为1 s)
124 DT_P23 = 0.25 # 问题2/3统一连续生产积分内部时间步
125 DT_P4 = 0.25 # 问题4最终内部时间步
126 DT_SEG_FINE = 2.0 # 问题2/3/4:  $t \leq 14400$  s
127 DT_SEG_COARSE = 4.0 # 问题2/3/4:  $t > 14400$  s
128 DT_SWITCH_S = 14400.0 # 分段切换时刻
129
130 # Picard 迭代
131 PICARD_MAX = 20
132 PICARD_TOL_T = 1.0 * 10.0 ** (-6) # °C
133 PICARD_TOL_C = 1.0 * 10.0 ** (-8) # kg/kg
134
135 # 达标事件仿真上限 (h  $\rightarrow$  s)
136 T_END_MAX_H = 120.0
137 SEC_PER_HOUR = 3600.0
138 T_END_MAX_S = T_END_MAX_H * SEC_PER_HOUR
139
140 # -----
141 # 输出网格 (output_specs)
142 # -----
143 SAVE_DR_CM = 0.1 # 落盘距离步长
144 N_DIST_COLS = 21 # 0.0 .. 2.0 cm 共 21 列
145 # 距离列 (cm): 0, 0.1, ..., 2.0 —— 由步长生成, 避免硬编码序列
146 DIST_COLS_CM = [round(i * SAVE_DR_CM, 4) for i in range(N_DIST_COLS)]
147
148 # 问题1 报表 (表1/表2)
149 P1_SAVE_DURATION_S = 1800.0
150 P1_SAVE_DT_S = 1.0
151 P1_REPORT_TIMES_S = [100.0, 300.0, 600.0, 900.0, 1200.0, 1500.0, 1800.0]
152 P1_REPORT_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
153
154 # 问题2 报表 (表3/表4): 0.5 h 间隔至 3 h
155 P2_REPORT_TIMES_H = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
156 P2_REPORT_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
157 P2_SAVE_DT_S = 1.0 # result2.xlsx A 列步长
158
159 # 问题3/4 报表 (表5/表6): 6 h 间隔 + 末行"烘干结束时间"
160 P34_REPORT_DT_H = 6.0
161 P34_REPORT_DR_CM = 0.5
162 P34_REPORT_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
163 P34_SAVE_DT_S = 60.0 # result3/result4 A 列步长
164
165 # 输出小数位
166 OUT_DECIMALS = 4
167
168 # -----

```

```

169 # 附件2 半径末值 (S4 固定域下界参照; DATA_FACTS key_points)
170 # -----
171 R_END_CM = 1.198
172 R_END_M = R_END_CM * CM_TO_M
173
174 # -----
175 # 验证参数 (S9)
176 # -----
177 BESSEL_ROOTS = 30          # 级数解特征根数 (S9.3)
178 BESSEL_ROOTS_FALLBACK = 100 # 偏差超限时增至此数
179 # 网格无关性参考解 (S9.2, 仅问题1温度场)
180 REF_N_CV = 160
181 REF_DT_S = 0.125
182 GRID_TOL = 0.005          # e_T, e_C 判据 (K13)
183 MASS_RESIDUAL_TOL = 1.0 * 10.0 ** (-10) # K12
184
185 # 达标时长硬区间 (K9/K10; LOGIC_CONTRACT bounds)
186 T_END_HARD_LO_H = 24.0
187 T_END_HARD_HI_H = 120.0

```

2.2 环境驱动插值: ambient.py

```

1  """烘房环境驱动函数 (附件1 线性插值 + 平台外推, MODELING_REPORT S3.2 式(7)(8))。
2
3  T_air(t), C_air(t):  $t \in [0, 14400]$  线性插值附件1 全部 241 行;  $t > 14400$  取平台值 (假设5)。
4  附件1 第3列是烘房空气含湿量 (量级  $1e-2$ ), 不是药材水分浓度。
5  """
6  from __future__ import annotations
7
8  import numpy as np
9  import pandas as pd
10
11  import params as P
12
13  _CACHE = {}
14
15
16  class Ambient:
17      """时变环境边界。可注入平台替代值做灵敏度 (SA4/SA5)。"""
18
19      def __init__(self, t_plateau=None, c_plateau=None):
20          t_arr, T_arr, C_arr = _load_attach1()
21          self._t = t_arr
22          self._T = T_arr
23          self._C = C_arr
24          self.T_plateau = P.TA_PLATEAU if t_plateau is None else float(t_plateau)
25          self.C_plateau = P.CA_PLATEAU if c_plateau is None else float(c_plateau)
26          self.t_max = P.AMBIENT_T_MAX_S
27
28      def T_air(self, t):
29          """烘房空气温度 °C。t 可为标量或数组。"""
30          t = np.asarray(t, dtype=float)
31          val = np.interp(t, self._t, self._T)
32          return np.where(t <= self.t_max, val, self.T_plateau)

```

```

33
34     def C_air(self, t):
35         """烘房空气含湿量 kg/kg。t 可为标量或数组。"""
36         t = np.asarray(t, dtype=float)
37         val = np.interp(t, self._t, self._C)
38         return np.where(t <= self.t_max, val, self.C_plateau)
39
40
41     def _load_attach1():
42         if "attach1" not in _CACHE:
43             df = pd.read_excel(P.ATTACH1, sheet_name=0)
44             n = int(df.shape[0])
45             if n != P.N_ROWS_ATTACH1:
46                 raise ValueError(f"附件1 应为 {P.N_ROWS_ATTACH1} 行, 实际 {n} 行 (数据未读全)")
47             t = pd.to_numeric(df.iloc[:, 0], errors="coerce").to_numpy(dtype=float)
48             T = pd.to_numeric(df.iloc[:, 1], errors="coerce").to_numpy(dtype=float)
49             C = pd.to_numeric(df.iloc[:, 2], errors="coerce").to_numpy(dtype=float)
50             if not (np.all(np.isfinite(t)) and np.all(np.isfinite(T)) and np.all(np.isfinite(C))):
51                 raise ValueError("附件1 含非有限值")
52             _CACHE["attach1"] = (t, T, C)
53         return _CACHE["attach1"]

```

2.3 收缩半径插值: geometry.py

```

1     """求解域半径 R(t) 与收缩速率 Rdot(t) (MODELING_REPORT §7.4) 。
2
3     固定域: 返回常值 R 与 Rdot=0。
4     移动边界 (问题4): 附件2 → 累积最小值单调化 → PCHIP 保形插值;
5     Rdot 取 PCHIP 解析导数; 数据用尽后冻结半径 (radius_frozen) 。
6     单调化只用累积最小值, 不用平滑滤波 (端点不变) 。
7     """
8     from __future__ import annotations
9
10    import numpy as np
11    import pandas as pd
12    from scipy.interpolate import PchipInterpolator
13
14    import params as P
15
16    _CACHE = {}
17
18
19    class FixedGeometry:
20        """固定求解域, R=R_const (问题1/2/3 与情景 S0/S1/S4) 。”
21
22        def __init__(self, R_const_m):
23            self.R_const = float(R_const_m)
24            self.radius_frozen = False
25            self.moving = False
26
27        def R(self, t):
28            t = np.asarray(t, dtype=float)
29            return np.full(t.shape, self.R_const) if t.ndim else self.R_const
30

```

```

31     def Rdot(self, t):
32         t = np.asarray(t, dtype=float)
33         return np.zeros(t.shape) if t.ndim else 0.0
34
35
36 class MovingGeometry:
37     """附件2 移动边界（问题4，情景 S2/S3）。半径单位 cm→m。"""
38
39     def __init__(self):
40         t_arr, R_mono_m = _load_attach2_mono()
41         self._t = t_arr
42         self._R = R_mono_m
43         self._t_max = float(t_arr[-1])
44         self._R_end = float(R_mono_m[-1])
45         self._pchip = PchipInterpolator(t_arr, R_mono_m, extrapolate=False)
46         self._pchip_d = self._pchip.derivative()
47         self.radius_frozen = False
48         self.moving = True
49
50     def R(self, t):
51         t = np.asarray(t, dtype=float)
52         inside = np.clip(t, self._t[0], self._t_max)
53         val = self._pchip(inside)
54         if np.any(t > self._t_max):
55             self.radius_frozen = True
56         return np.where(t > self._t_max, self._R_end, val)
57
58     def Rdot(self, t):
59         t = np.asarray(t, dtype=float)
60         inside = np.clip(t, self._t[0], self._t_max)
61         val = self._pchip_d(inside)
62         # 冻结段 Rdot=0; 数据端点导数按 PCHIP 端点值
63         return np.where((t > self._t_max) | (t < self._t[0]), 0.0, val)
64
65     @property
66     def t_data_max(self):
67         return self._t_max
68
69     @property
70     def R_data(self):
71         """返回 (t, R_mono_m) 供诊断/绘图。"""
72         return self._t.copy(), self._R.copy()
73
74
75 def _load_attach2_mono():
76     if "attach2" not in _CACHE:
77         df = pd.read_excel(P.ATTACH2, sheet_name=0)
78         n = int(df.shape[0])
79         if n != P.N_ROWS_ATTACH2:
80             raise ValueError(f"附件2 应为 {P.N_ROWS_ATTACH2} 行，实际 {n} 行（数据未读全）")
81         t = pd.to_numeric(df.iloc[:, 0], errors="coerce").to_numpy(dtype=float)
82         R_cm = pd.to_numeric(df.iloc[:, 1], errors="coerce").to_numpy(dtype=float)
83         if not (np.all(np.isfinite(t)) and np.all(np.isfinite(R_cm))):
84             raise ValueError("附件2 含非有限值")
85         # 累积最小值单调化（收缩不可逆），端点保持不变
86         R_mono_cm = np.minimum.accumulate(R_cm)

```

```

87     R_mono_m = R_mono_cm * P.CM_TO_M
88     _CACHE["attach2"] = (t, R_mono_m)
89     return _CACHE["attach2"]

```

2.4 物性闭合式: props.py

```

1  """三套附录物性模块（严格按问题隔离，MODELING_REPORT §4）。
2
3  props(appendix, C, T_celsius) -> (rho, cp, k, D), 全部返回与 C 同形状的 ndarray。
4  rho 用干基 C, cp/k 用湿基 w=C/(C+1); D 中温度用绝对温度 K 且取药材局部温度。
5  附录2 的 D 不含温度项。附录之间禁止串用 (K14)。
6  """
7  from __future__ import annotations
8
9  import numpy as np
10
11  import params as P
12
13
14  def wet_basis(C):
15      """湿基分率 w = C/(C+1)。"""
16      C = np.asarray(C, dtype=float)
17      return C / (C + 1.0)
18
19
20  def props(appendix: int, C, T_celsius=None):
21      """返回 (rho, cp, k, D), 均为与 C 同形状的 ndarray。
22
23      appendix ∈ {2,3,4}。附录3/4 需要 T_celsius (药材局部温度℃度, )。
24      """
25      C = np.asarray(C, dtype=float)
26
27      if appendix == 2:
28          rho = np.full_like(C, P.APP2_RHO)
29          cp = np.full_like(C, P.APP2_CP)
30          k = np.full_like(C, P.APP2_K)
31          D = P.APP2_D_PREF * np.exp(-P.APP2_D_EC / C)
32          return rho, cp, k, D
33
34      if T_celsius is None:
35          raise ValueError(f"附录{appendix} 的 D 依赖温度, 必须提供 T_celsius")
36      T_celsius = np.asarray(T_celsius, dtype=float)
37      T_K = T_celsius + P.T_KELVIN
38      # K15: 绝对温度必须 > 273
39      if np.any(T_K <= 273.0):
40          raise ValueError(f"附录{appendix} D 计算温度非绝对温标: min(T_K)={float(np.min(T_K))}")
41      w = C / (C + 1.0)
42
43      if appendix == 3:
44          rho = P.APP3_RHO_A + P.APP3_RHO_B * C
45          cp = P.APP3_CP_A + P.APP3_CP_B * w
46          k = P.APP3_K_A + P.APP3_K_B * w
47          D = P.APP3_D_PREF * np.exp(-P.APP3_D_EC / C) * np.exp(-P.APP3_D_ET / T_K)
48          return rho, cp, k, D

```

```

49
50     if appendix == 4:
51         rho = P.APP4_RHO_A + P.APP4_RHO_B * C
52         cp = P.APP4_CP_A + P.APP4_CP_B * w
53         k = P.APP4_K_A + P.APP4_K_B * w
54         D = P.APP4_D_PREF * np.exp(-P.APP4_D_EC / C) * np.exp(-P.APP4_D_ET / T_K)
55         return rho, cp, k, D
56
57     raise ValueError(f"未知附录编号: {appendix}")
58
59
60 def rho_s_eff(appendix: int, C):
61     """干物质表观密度  $\rho_s = \rho/(1+C)$  (INC1 质量自洽诊断, 问题4) 。"""
62     C = np.asarray(C, dtype=float)
63     if appendix == 4:
64         rho = P.APP4_RHO_A + P.APP4_RHO_B * C
65     elif appendix == 3:
66         rho = P.APP3_RHO_A + P.APP3_RHO_B * C
67     elif appendix == 2:
68         rho = np.full_like(C, P.APP2_RHO)
69     else:
70         raise ValueError(f"未知附录编号: {appendix}")
71     return rho / (1.0 + C)

```

2.5 有限体积离散核: fvm.py

正式问题三采用 $N = 2560$ 、 $\Delta t = 0.25$ s, 首达时间为 57.4716 h; 界面采用正文定义的算术平均口径。

```

1  """有限体积内核 (柱坐标控制体 + 全隐式后向 Euler, MODELING_REPORT §6.1、§7.5) 。
2
3  统一在归一化坐标  $\xi=r/R(t) \in [0, 1]$  上离散:
4  - 固定域 (问题1/2/3):  $R=R_0$  常值、 $\dot{R}=0$ , 退化为物理坐标 ( $\times R^2$  后逐点等价, 见 K18/U5);
5  - 移动边界 (问题4):  $R(t)$ 、 $\dot{R}(t)$  由附件2 给出, 含  $(1-\chi) \cdot \xi \cdot \dot{R}/R \cdot \partial \xi \phi$  对流项。
6
7  界面输运系数取相邻节点平均 (口径 FACE_MODE, 交付用算术平均 §6.1; 调和 §13.4 供对照);
8  界面通量以  $2\pi \cdot \xi_{\{i\pm 1/2\}} \cdot \Gamma$  面积权重体现  $1/r$  因子。
9  表面 Robin 以通量形式并入 (§6.1); 中心零通量为自然边界 (无需显式施加)。
10  对流项用一阶上风离散:  $\dot{R} \leq 0 \Rightarrow$  平流速度  $v = -\xi \cdot \dot{R}/R \geq 0 \Rightarrow$  迎风取后向差分, 保证系数矩阵对角占优。
11  """
12  from __future__ import annotations
13
14  import numpy as np
15  from scipy.linalg import solve_banded
16
17  import params as P
18
19
20  def make_grid(N=None):
21     """返回  $\xi$  节点( $N+1$ )、 $\Delta \xi$ 、单位长度控制体体积  $\tilde{V}(N+1)$  ( $\xi$  度量) 。"""
22     if N is None:
23         N = P.N_CV
24     dxi = 1.0 / N
25     xi = np.linspace(0.0, 1.0, N + 1)

```

```

26     Vt = np.empty(N + 1)
27     Vt[0] = np.pi * (dxi / 2.0) ** 2
28     Vt[1:N] = 2.0 * np.pi * xi[1:N] * dxi
29     Vt[N] = np.pi * (1.0 - (1.0 - dxi / 2.0) ** 2)
30     return xi, dxi, Vt
31
32
33 # 界面平均口径（正式结果用算术平均；调和平均仅作历史对照）：
34 # 'arith' —— §6.1 正文「相邻节点算术平均」。正式默认。
35 #     正式问题三配置：N=2560、dt=0.25 s、t_end=57.4716 h。
36 #     下方 N=20 / N=40 / 56.18 h 等数字是历史粗网格诊断，不作为正式结果。
37 # 'harm' —— §13.4 指定的调和平均，仅作口径对照。
38 #     历史粗网格下收敛更慢（N=40 未收敛、需 N≥320），不作为正式交付口径。
39 # 两种口径均给出左右唯一的界面值，质量收支同为机器精度；
40 # 分歧只是收敛速度与物理偏好，不改变本文件中的离散算法。
41 FACE_MODE = "arith"
42
43
44 def face_mean(G, mode=None):
45     """相邻节点界面平均，返回长度 N 的界面值 (i+1/2, i=0..N-1)。
46
47     mode='arith' 算术平均 (§6.1, 交付默认)；'harm' 调和平均 (§13.4, 供对照)。
48     G 可能为 0 (D=0)，调和口径此时取 0 (无输运)，避免 0/0。
49     """
50     mode = FACE_MODE if mode is None else mode
51     Gi = G[:-1]
52     Gi1 = G[1:]
53     if mode == "arith":
54         return 0.5 * (Gi + Gi1)
55     denom = Gi + Gi1
56     with np.errstate(divide="ignore", invalid="ignore"):
57         return np.where(denom > 0.0, 2.0 * Gi * Gi1 / denom, 0.0)
58
59
60 def harmonic_face(G):
61     """向后兼容别名（调和口径）。新代码用 face_mean。"""
62     return face_mean(G, mode="harm")
63
64
65 def step_implicit(phi_old, cap, G, beta, phi_air, R, Rdot, chi, dt, xi, dxi, Vt,
66                 return_diag=False, src=None):
67     """推进一个隐式时间步，返回 phi_new (长度 N+1)。
68
69     参数：
70     phi_old : 上一时刻场量 (N+1,)
71     cap      : 容量系数 κ 逐点 (N+1,) —— 温度取 pc_p, 浓度取 1
72     G        : 输运系数 Γ 逐点 (N+1,) —— 温度取 k, 浓度取 D (已施 D_FLOOR)
73     beta     : Robin 系数 (h 或 h_m)
74     phi_air  : 环境侧驱动值 (标量)
75     R, Rdot  : 当前半径与收缩速率
76     chi      : 骨架闭合开关。
77                 χ=1: 材料随动 ALE 正式主模型，相对对流项抵消；
78                 χ=0: Eulerian/Landau 交叉验证，保留相对对流项。
79                 本函数的数学实现不因上述口径说明而改变。
80     dt       : 时间步长
81     xi,dxi,Vt : make_grid 输出

```

```

82     return_diag : True 时额外返回离散守恒残差 (§9.1/K12 质量收支)
83     src        : 逐控制体体积分源项 (N+1,), rhs += src; 默认 None (SA8 潜热汇 / U4 单测)
84
85 返回: phi_new, 或 (phi_new, resid_abs)。resid_abs 为独立核算的守恒残差
86      |Δ储存率 - 表面净通量 + 对流净额|, 扩散界面通量解析对消 ⇒ 应≈机器精度。
87  """
88  N = phi_old.size - 1
89  R2 = R * R
90
91  # 界面平均输运系数 (i+1/2, i=0..N-1), 口径由 FACE_MODE 决定 (默认算术, §6.1)
92  Gf = face_mean(G)
93  xi_face = (np.arange(N) + 0.5) * dxi          # ξ_{i+1/2}
94  aE = (2.0 * np.pi / R2) * xi_face * Gf / dxi # 东面导纳, 长度 N
95
96  # 三对角: A[i,i-1]=lo[i], A[i,i]=di[i], A[i,i+1]=up[i]
97  di = cap * Vt / dt                          # 时间项
98  lo = np.zeros(N + 1)
99  up = np.zeros(N + 1)
100
101  # 扩散 (每个界面同时进入两侧节点方程)
102  di[0:N] += aE
103  up[0:N] += -aE
104  di[1:N + 1] += aE
105  lo[1:N + 1] += -aE
106
107  rhs = (cap * Vt / dt) * phi_old
108
109  # 表面 Robin (节点 N): sfc = 2π·β/R
110  sfc = 2.0 * np.pi * beta / R
111  di[N] += sfc
112  rhs[N] += sfc * phi_air
113
114  # 可选体积分源项 (SA8 潜热汇 / U4 线性场源; 默认无)
115  if src is not None:
116      rhs = rhs + src
117
118  # 对流项 (仅 χ=0): g_i = (1-χ)·κ_i·ξ_i·Rdot/R·V̂_i, 一阶迎风
119  lo_adv = up_adv = di_adv = None
120  if chi == 0 and Rdot != 0.0:
121      g = cap * xi * (Rdot / R) * Vt          # ≤0 (Rdot≤0)
122      di_adv = np.abs(g) / dxi
123      lo_adv = np.where(g <= 0.0, g / dxi, 0.0) # 迎风: g≤0 → 后向差分进 lo
124      up_adv = np.where(g > 0.0, -g / dxi, 0.0) # g>0 → 前向差分进 up
125      di += di_adv
126      lo += lo_adv
127      up += up_adv
128
129  ab = np.zeros((3, N + 1))
130  ab[0, 1:] = up[0:N]          # 超对角 A[i,i+1] 置于列 i+1
131  ab[1, :] = di
132  ab[2, :-1] = lo[1:N + 1]    # 次对角 A[i,i-1] 置于列 i-1
133  phi_new = solve_banded((1, 1), ab, rhs)
134
135  if not return_diag:
136      return phi_new
137

```

```

138 # 独立守恒核算: 储存率 = 表面净通量 - 对流净额 (扩散逐面对消)
139 stored = float(np.sum(cap * Vt * (phi_new - phi_old)) / dt)
140 qsurf = float(sfc * (phi_air - phi_new[N]))
141 adv_net = 0.0
142 if di_adv is not None:
143     adv_net = float(np.sum(di_adv * phi_new)
144                     + np.sum(lo_adv[1:] * phi_new[:-1])
145                     + np.sum(up_adv[:-1] * phi_new[1:]))
146 resid_abs = abs(stored - qsurf + adv_net)
147 return phi_new, resid_abs
148
149
150 def discrete_operator(phi, G, R, xi, dxi, Vt):
151     """对给定场返回柱坐标扩散算子的逐控制体积分值 (诊断/单元测试用)。
152
153     返回  $(1/kV) \cdot [\text{界面通量净额}]$ , 用于 §6.1 的  $\phi=r^2 \rightarrow$  算子=4 校验。
154     此处返回未除容量的净通量除以 Vt (即算子近似值)。
155     """
156     N = phi.size - 1
157     R2 = R * R
158     Gf = face_mean(G)
159     xi_face = (np.arange(N) + 0.5) * dxi
160     aE = (2.0 * np.pi / R2) * xi_face * Gf / dxi
161     flux_E = aE * (phi[1:] - phi[:-1]) # 东面净通量, 长度 N
162     net = np.zeros(N + 1)
163     net[0:N] += flux_E
164     net[1:N + 1] -= flux_E
165     return net / Vt

```

2.6 时间推进与耦合迭代: driver.py

```

1  """四问统一时间推进引擎 (MODELING_REPORT §6.2、§7.5、§11)。
2
3  单一 run(cfg) 覆盖问题1/2/3/4 与兼容性诊断:
4  - 每步严格保序算子分裂 ( $\square p, cp, k \rightarrow \square$ 解  $T^{n+1} \rightarrow \square$ 用  $T^{n+1}$  算  $D \rightarrow \square$ 解  $C^{n+1}$ 
5     $\rightarrow \square C$  下限  $\rightarrow \square$ Picard 复核), 顺序不可交换 (§11、§13.4 MC4);
6  - 正式调用显式传入步长: 问题1为0.125s, 问题2/3与问题4均为0.2500s; 跨输出时刻截断步长精确命中;
7  - 终止事件按逐点最大值首次 < C_th, 式(19) 亚步线性插值定位 t_end (§6.4、K11);
8  - 移动边界经  $\xi$  网格统一处理; 问题4正式配置 $\chi=1$ ,  $\chi=0$ 仅作Eulerian/Landau交叉验证;
9  - 快照按输出网格记录, 节点场插值到固定 r 列, 问题4 对  $r>R(t)$  置空、未列取  $\xi=1$  (§7.7、式(26))。
10  不使用 solve_ivp (§11); 界面算术平均 (§6.1 正式口径; 调和平均仅历史对照); 下限  $1e \ll -30.15$  (§6.2)。
11  """
12  from __future__ import annotations
13
14  from dataclasses import dataclass, field
15  from typing import Optional
16
17  import numpy as np
18
19  import params as P
20  import props as PR
21  import fvm
22
23

```

```

24 @dataclass
25 class CaseConfig:
26     """一次推进的完整配置。几何对象须实现 R(t)/Rdot(t) (geometry.py)。"""
27     name: str
28     appendix: int          # 物性附录 2/3/4
29     geom: object           # FixedGeometry 或 MovingGeometry
30     chi: int = P.CHI_PRIMARY # 兼容默认; 问题4显式传入 $\chi=1$ ,  $\chi=0$ 仅作交叉验证
31     N: int = P.N_CV        # 控制体数
32     dt_policy: str = "seg"  # 'p1'(=1s) | 'seg'(2s→4s) | 'const'
33     dt_const: float = P.DT_SEG_FINE
34     save_dt_s: float = P.P34_SAVE_DT_S # 输出网格步长
35     t_max_s: float = P.T_END_MAX_S     # 仿真硬上限
36     detect_end: bool = True            # 是否检测达标事件
37     event_each_step: bool = False      # 是否在每个内部步定位事件 (默认保持原输出步检测)
38     store_nodes: bool = False          # 是否保存全节点  $\xi$  场 (问题4 表6 需要)
39     dirichlet: bool = False            # SA6:  $h, h_m$ →大, 退化 Dirichlet
40     h_scale: float = 1.0               # SA2
41     hm_scale: float = 1.0             # SA1
42     D_scale: float = 1.0              # SA3
43     t_plateau: Optional[float] = None  # SA4
44     c_plateau: Optional[float] = None  # SA5
45     latent_heat: float = 0.0          # SA8 潜热汇 J/kg (默认 0, 假设4)
46     ambient: object = None            # Ambient 实例 (None 时内部构造)
47     role: str = "deliverable"         # 'deliverable' | 'control' (S1 免 H18)
48
49     def dt_at(self, t):
50         """给定当前时刻返回策略步长。"""
51         if self.dt_policy == "p1":
52             return P.DT_P1
53         if self.dt_policy == "const":
54             return self.dt_const
55         return P.DT_SEG_FINE if t < P.DT_SWITCH_S else P.DT_SEG_COARSE
56
57
58     def _interp_cols(phi_nodes, R, cols_m):
59         """节点场 ( $\xi$  均布) 插值到固定物理半径列 cols_m;  $r>R(t)$  置 NaN (§7.7 式26)。"""
60         n = phi_nodes.size - 1
61         r_nodes = np.linspace(0.0, R, n + 1)
62         out = np.interp(cols_m, r_nodes, phi_nodes)
63         out = np.where(cols_m > R + 1e-12, np.nan, out)
64         return out
65
66
67     def _adv_net_mass(C, xi, Rdot, R, dxi, Vt, chi):
68         """迎风对流项对总量的净贡献 (浓度方程  $k=1$ ; 用于全局质量收支复核)。
69
70         与 fvm.step_implicit 内一致:  $g=\xi \cdot Rdot/R \cdot \tilde{V} \leq 0$ , 进 di/lo/up。
71         返回  $\Sigma(di\_adv \cdot C + lo\_adv[1:] \cdot C[:-1] + up\_adv[:-1] \cdot C[1:])$ 。
72         """
73         if chi != 0 or Rdot == 0.0:
74             return 0.0
75         g = xi * (Rdot / R) * Vt
76         di_adv = np.abs(g) / dxi
77         lo_adv = np.where(g <= 0.0, g / dxi, 0.0)
78         up_adv = np.where(g > 0.0, -g / dxi, 0.0)
79         return float(np.sum(di_adv * C))

```

```

80         + np.sum(lo_adv[1:] * C[:-1])
81         + np.sum(up_adv[:-1] * C[1:]))
82
83
84 def run(cfg: CaseConfig):
85     """推进一个 CaseConfig, 返回结果字典 (见文件末尾字段说明)。"""
86     import ambient as AMB
87     amb = cfg.ambient if cfg.ambient is not None else AMB.Ambient(
88         t_plateau=cfg.t_plateau, c_plateau=cfg.c_plateau)
89
90     xi, dxi, Vt = fvm.make_grid(cfg.N)
91     N = cfg.N
92     cols_m = np.asarray(P.DIST_COLS_CM, dtype=float) * P.CM_TO_M # 21 固定列 (m)
93
94     # Robin 系数: SA6 Dirichlet 以大  $\beta$  实现 ( $U2/S9.5$ )
95     big = 1.0e9
96     beta_T = big if cfg.dirichlet else P.H_CONV * cfg.h_scale
97     beta_C = big if cfg.dirichlet else P.HM_CONV * cfg.hm_scale
98
99     # 初值
100     T = np.full(N + 1, P.T0_C)
101     C = np.full(N + 1, P.C0)
102     C0_tot = None # 初始总量 ( $\xi$  度量), 用于式(27)相对残差
103
104     # 累计诊断
105     resid_step_max = 0.0
106     mass_bal_abs = 0.0 #  $\Sigma$  储存变化 -  $\Sigma$  表面通量 +  $\Sigma$  对流净额 ( $\xi$  度量)
107     picard_iters_max = 0
108     picard_nonconv = 0
109     floor_trunc_steps = 0
110     dt_halving = 0
111
112     # 快照容器 (按输出网格)
113     snap_t = []
114     snap_Tcol = []
115     snap_Ccol = []
116     snap_Tsurf = []
117     snap_Csurf = []
118     snap_maxC = []
119     snap_Tc = [] # 中心温度
120     snap_Cc = [] # 中心浓度
121     snap_R = []
122     snap_nodesC = [] if cfg.store_nodes else None
123     snap_nodesT = [] if cfg.store_nodes else None
124
125     def record(t_now, Tf, Cf, Rn):
126         snap_t.append(t_now)
127         snap_Tcol.append(_interp_cols(Tf, Rn, cols_m))
128         snap_Ccol.append(_interp_cols(Cf, Rn, cols_m))
129         snap_Tsurf.append(float(Tf[N]))
130         snap_Csurf.append(float(Cf[N]))
131         snap_maxC.append(float(np.max(Cf)))
132         snap_Tc.append(float(Tf[0]))
133         snap_Cc.append(float(Cf[0]))
134         snap_R.append(float(Rn))
135         if cfg.store_nodes:

```

```

136         snap_nodesC.append(Cf.copy())
137         snap_nodesT.append(Tf.copy())
138
139     R0 = float(cfg.geom.R(0.0))
140     C0_tot = float(np.sum(Vt * C))
141     record(0.0, T, C, R0)
142
143     t = 0.0
144     save_idx = 1
145     next_save = save_idx * cfg.save_dt_s
146     t_end = None
147     end_T = None
148     end_C = None
149     end_R = None
150     reached = False
151     prev_maxC = float(np.max(C))
152     prev_t = 0.0
153     event_prev_maxC = prev_maxC
154     event_prev_t = prev_t
155
156     while t < cfg.t_max_s - 1e-9:
157         dt = cfg.dt_at(t)
158         # 跨越输出时刻：截断步长精确命中 (§13.3)
159         landed = False
160         if t + dt >= next_save - 1e-9:
161             dt = next_save - t
162             landed = True
163         if t + dt > cfg.t_max_s:
164             dt = cfg.t_max_s - t
165             landed = False
166         t_new = t + dt
167
168         Rn = float(cfg.geom.R(t_new))
169         Rdot = float(cfg.geom.Rdot(t_new))
170         Tair = float(amb.T_air(t_new))
171         Cair = float(amb.C_air(t_new))
172
173         # ---- 保序算子分裂 + Picard (§11 步 1-6) ----
174         T_it = T.copy()
175         C_it = C.copy()
176         dC_lag = np.zeros(N + 1) # 潜热汇滞后项 (SA8)
177         n_it = 0
178         for m in range(P.PICARD_MAX):
179             n_it = m + 1
180             rho, cp, k, _ = PR.props(cfg.appendix, C_it, T_it)
181             capT = rho * cp
182             srcT = None
183             if cfg.latent_heat > 0.0:
184                 # 潜热汇：蒸发吸热 =  $\lambda \cdot p_s \cdot (-\Delta C) / \Delta t \cdot \tilde{V}$ ，滞后一次迭代（探查用）
185                 srcT = -cfg.latent_heat * PR.rho_s_eff(cfg.appendix, C_it) * (-dC_lag) * Vt / dt
186             T_new = fvm.step_implicit(T, capT, k, beta_T, Tair, Rn, Rdot, cfg.chi,
187                                     dt, xi, dxi, Vt, src=srcT)
188             _, _, _, D = PR.props(cfg.appendix, C_it, T_new)
189             D = np.maximum(D * cfg.D_scale, P.D_FLOOR)
190             capC = np.ones(N + 1)
191             C_new = fvm.step_implicit(C, capC, D, beta_C, Cair, Rn, Rdot, cfg.chi,

```

```

192         dt, xi, dxi, Vt)
193     dT_it = float(np.max(np.abs(T_new - T_it)))
194     dC_it = float(np.max(np.abs(C_new - C_it)))
195     dC_lag = C_new - C
196     T_it, C_it = T_new, C_new
197     if m >= 1 and dT_it < P.PICARD_TOL_T and dC_it < P.PICARD_TOL_C:
198         break
199     else:
200         picard_nonconv += 1
201     picard_iters_max = max(picard_iters_max, n_it)
202
203     # 步 5: 浓度下限保护 (§6.2; 记录截断步数)
204     below = C_it < P.C_FLOOR
205     if np.any(below):
206         floor_trunc_steps += 1
207         C_it = np.maximum(C_it, P.C_FLOOR)
208
209     # 全局质量收支 (§ 度量, 式27 分子逐步累加)
210     sfc = 2.0 * np.pi * beta_C / Rn
211     flux = sfc * (Cair - C_it[N]) # 表面净流入速率
212     adv = _adv_net_mass(C_it, xi, Rdot, Rn, dxi, Vt, cfg.chi)
213     stored_rate = float(np.sum(Vt * (C_it - C)) / dt)
214     resid_step_max = max(resid_step_max, abs(stored_rate - flux + adv))
215     mass_bal_abs += (float(np.sum(Vt * (C_it - C))) - flux * dt + adv * dt)
216
217     T_prev_step, C_prev_step = T, C
218     T, C = T_it, C_it
219     t = t_new
220
221     # 步 6/7: 达标事件 (逐点最大值, 式19 亚步插值) + 快照
222     maxC = float(np.max(C))
223     if (cfg.detect_end and cfg.event_each_step and not reached
224         and maxC <= P.C_TH < event_prev_maxC):
225         denom = event_prev_maxC - maxC
226         frac = (event_prev_maxC - P.C_TH) / denom if denom > 0 else 1.0
227         t_end = event_prev_t + (t - event_prev_t) * frac
228         end_T = T_prev_step + frac * (T - T_prev_step)
229         end_C = C_prev_step + frac * (C - C_prev_step)
230         end_R = float(cfg.geom.R(t_end))
231         reached = True
232     event_prev_maxC = maxC
233     event_prev_t = t
234
235     if landed and abs(t - next_save) < 1e-6:
236         record(t, T, C, Rn)
237         if (cfg.detect_end and not cfg.event_each_step and not reached
238             and maxC < P.C_TH <= prev_maxC):
239             denom = prev_maxC - maxC
240             frac = (P.C_TH - maxC) / denom if denom > 0 else 0.0
241             t_end = t - cfg.save_dt_s * frac
242             end_T = None
243             end_C = None
244             end_R = float(cfg.geom.R(t_end))
245             reached = True
246         prev_maxC = maxC
247         prev_t = t

```

```

248         save_idx += 1
249         next_save = save_idx * cfg.save_dt_s
250         if reached and cfg.detect_end:
251             break
252         elif reached and cfg.detect_end:
253             break
254
255     # ---- 打包 ----
256     out = {
257         "name": cfg.name,
258         "appendix": cfg.appendix,
259         "chi": cfg.chi,
260         "N": N,
261         "dt_policy": cfg.dt_policy,
262         "save_dt_s": cfg.save_dt_s,
263         "role": cfg.role,
264         "t": np.asarray(snap_t),
265         "Tcol": np.asarray(snap_Tcol),          # (nt,21) 固定列温度
266         "Ccol": np.asarray(snap_Ccol),          # (nt,21) 固定列浓度
267         "Tsurf": np.asarray(snap_Tsurf),
268         "Csurf": np.asarray(snap_Csurf),
269         "maxC": np.asarray(snap_maxC),
270         "Tcenter": np.asarray(snap_Tc),
271         "Ccenter": np.asarray(snap_Cc),
272         "R": np.asarray(snap_R),
273         "cols_m": cols_m,
274         "t_end_s": t_end,
275         "t_end_h": (t_end / P.SEC_PER_HOUR) if t_end is not None else None,
276         "end_T": end_T,
277         "end_C": end_C,
278         "end_R": end_R,
279         "reached": reached,
280         "resid_step_max": resid_step_max,
281         "mass_resid_rel": abs(mass_bal_abs) / C0_tot if C0_tot else float("nan"),
282         "picard_iters_max": picard_iters_max,
283         "picard_nonconv": picard_nonconv,
284         "floor_trunc_steps": floor_trunc_steps,
285         "dt_halving": dt_halving,
286         "radius_frozen": bool(getattr(cfg.geom, "radius_frozen", False)),
287         "final_T": T.copy(),
288         "final_C": C.copy(),
289     }
290     if cfg.store_nodes:
291         out["nodesC"] = np.asarray(snap_nodesC)    # (nt,N+1)
292         out["nodesT"] = np.asarray(snap_nodesT)
293     return out
294
295
296 def field_at_time(res, t_query_s, which="C"):
297     """在两快照间线性插值出 t_query 时刻的 21 列场（末行「烘干结束时间」用）。"""
298     ts = res["t"]
299     col = res["Ccol"] if which == "C" else res["Tcol"]
300     if t_query_s <= ts[0]:
301         return col[0].copy()
302     if t_query_s >= ts[-1]:
303         return col[-1].copy()

```

```

304     j = int(np.searchsorted(ts, t_query_s))
305     t0, t1 = ts[j - 1], ts[j]
306     w = (t_query_s - t0) / (t1 - t0) if t1 > t0 else 0.0
307     return (1.0 - w) * col[j - 1] + w * col[j]

```

2.7 问题一求解: problem1.py

```

1  """问题1: 附录2 常物性温度 + 非线性传质 (MODELING_REPORT §5) 。
2
3  内部 Δt=0.125 s 推进至 1800 s; 表1/表2 报表 7×5; result1.xlsx 双表按 1 s 输出 21 列。
4  """
5  from __future__ import annotations
6
7  import numpy as np
8
9  import params as P
10 import geometry
11 import driver as D
12 import io_out as IO
13
14 REPORT_TIMES = P.P1_REPORT_TIMES_S
15 REPORT_RADII = P.P1_REPORT_RADII_CM
16 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII] # 0,5,10,15,20
17
18
19 def _report_block(res, times_s, field):
20     """在快照时间轴上取 times_s × 报表半径的交叉矩阵。"""
21     ts = res["t"]
22     col = res["Tcol"] if field == "T" else res["Ccol"]
23     rows = []
24     for tq in times_s:
25         j = int(np.argmin(np.abs(ts - tq)))
26         rows.append([col[j, ci] for ci in _COLIDX])
27     return np.asarray(rows)
28
29
30 def solve(write=True):
31     g = geometry.FixedGeometry(P.R0_M)
32     cfg = D.CaseConfig(name="P1", appendix=2, geom=g, N=P.N_P1_CV,
33                        dt_policy="p1",
34                        save_dt_s=P.P1_SAVE_DT_S, t_max_s=P.P1_SAVE_DURATION_S,
35                        detect_end=False)
36     res = D.run(cfg)
37
38     T_tab = _report_block(res, REPORT_TIMES, "T")
39     C_tab = _report_block(res, REPORT_TIMES, "C")
40
41     if write:
42         # 表1 (温度) + 表2 (水分浓度) 合并落盘, 分区标注
43         path = P.OUTPUT_DIR / "table1_2_problem1.csv"
44         path.parent.mkdir(parents=True, exist_ok=True)
45         import csv
46         radii_lbl = [f"{r:g}" for r in REPORT_RADII]
47         with open(path, "w", newline="", encoding="utf-8-sig") as f:

```

```

48     w = csv.writer(f)
49     w.writerow(["表1 温度°C() 时间(s)\\距离(cm)"] + radii_lbl)
50     for t, row in zip(REPORT_TIMES, T_tab):
51         w.writerow([int(t)] + [IO.fmt4(v) for v in row])
52     w.writerow([])
53     w.writerow(["表2 水分浓度(kg/kg) 时间(s)\\距离(cm)"] + radii_lbl)
54     for t, row in zip(REPORT_TIMES, C_tab):
55         w.writerow([int(t)] + [IO.fmt4(v) for v in row])
56
57     # result1.xlsx: A 列 1 s 步长 (1..1800) , 21 距离列, 双表
58     mask = res["t"] >= 1.0 - 1e-9
59     times = res["t"][mask]
60     IO.write_result_xlsx(P.OUTPUT_DIR / "result1.xlsx",
61                          {"温度": res["Tcol"][mask], "水分浓度": res["Ccol"][mask]},
62                          times)
63
64     # JSON 摘要 (无大数组)
65     j = int(np.argmax(np.abs(res["t"] - 1800.0)))
66     summary = {
67         "problem": 1, "appendix": 2, "N": res["N"], "dt_s": P.DT_P1,
68         "output_dt_s": P.P1_SAVE_DT_S, "chi": "n.a.(fixed)",
69         "center_T_1800": float(res["Tcenter"][j]),
70         "center_C_1800": float(res["Ccenter"][j]),
71         "surface_T_1800": float(res["Tsurf"][j]),
72         "surface_C_1800": float(res["Csurf"][j]),
73         "mass_resid_rel": res["mass_resid_rel"],
74         "resid_step_max": res["resid_step_max"],
75         "picard_iters_max": res["picard_iters_max"],
76         "picard_nonconv": res["picard_nonconv"],
77         "floor_trunc_steps": res["floor_trunc_steps"],
78         "table1_T": T_tab.round(4).tolist(),
79         "table2_C": C_tab.round(4).tolist(),
80         "report_times_s": list(REPORT_TIMES),
81         "report_radii_cm": list(REPORT_RADII),
82     }
83     return res, summary
84
85
86 if __name__ == "__main__":
87     _, s = solve()
88     print("P1 done:", {k: s[k] for k in
89                       ("center_T_1800", "center_C_1800", "surface_C_1800",
90                       "mass_resid_rel", "picard_iters_max")})

```

2.8 问题二、三共用主解: master_p23.py

```

1  """问题2/3 共享主推进 (附录3固定域 R=R0, 从 t=0 连续推进至达标) 。
2
3  问题2的3 h结果与问题3首次时间必须从同一次状态轨迹提取, 禁止重新初始化或拼接。
4  正式生产离散统一为 N=2560、内部 Δt=0.25 s; 快照仍每 1 s 保存。
5  """
6  from __future__ import annotations
7
8  import params as P

```

```

9 import geometry
10 import driver as D
11
12 _MASTER = {}
13
14
15 def run_master(force=False):
16     """运行（或复用）附录3 固定域主解，返回 driver.run 结果字典。"""
17     if "res" in _MASTER and not force:
18         return _MASTER["res"]
19     g = geometry.FixedGeometry(P.R0_M)
20     cfg = D.CaseConfig(name="P2P3_master", appendix=3, geom=g,
21                        N=P.N_P23_CV,
22                        dt_policy="const", dt_const=P.DT_P23,
23                        save_dt_s=P.P2_SAVE_DT_S, # 1 s 输出
24                        t_max_s=P.T_END_MAX_S, detect_end=True,
25                        store_nodes=False)
26     res = D.run(cfg)
27     _MASTER["res"] = res
28     return res

```

2.9 问题二求解：problem2.py

```

1 """问题2：附录3 全程变物性强耦合（MODELING_REPORT §6.3）。
2
3 表3/表4 报表窗  $\in t\{0.5, 1, 1.5, 2, 2.5, 3\}$  h  $\times \in r\{0, 0.5, 1, 1.5, 2\}$  cm;
4 已交付的 result2.xlsx 是问题二要求的前 3 h (1~10800 s) 温度与含水率全网格,
5 A 列步长 1 s。物理过程继续推进至问题三首达, 但 result2 不代表全程烘干结果。
6 """
7 from __future__ import annotations
8
9 import csv
10 import numpy as np
11
12 import params as P
13 import io_out as IO
14 import master_p23
15
16 REPORT_TIMES_H = P.P2_REPORT_TIMES_H
17 REPORT_RADII = P.P2_REPORT_RADII_CM
18 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII]
19
20
21 def _block(res, times_s, field):
22     ts = res["t"]
23     col = res["Tcol"] if field == "T" else res["Ccol"]
24     rows = []
25     for tq in times_s:
26         j = int(np.argmin(np.abs(ts - tq)))
27         rows.append([col[j, ci] for ci in _COLIDX])
28     return np.asarray(rows)
29
30
31 def solve(res=None, write=True):

```

```

32     if res is None:
33         res = master_p23.run_master()
34     times_s = [h * P.SEC_PER_HOUR for h in REPORT_TIMES_H]
35     T_tab = _block(res, times_s, "T")
36     C_tab = _block(res, times_s, "C")
37
38     if write:
39         path = P.OUTPUT_DIR / "table3_4_problem2.csv"
40         path.parent.mkdir(parents=True, exist_ok=True)
41         radii_lbl = [f"{r:g}" for r in REPORT_RADII]
42         with open(path, "w", newline="", encoding="utf-8-sig") as f:
43             w = csv.writer(f)
44             w.writerow(["表3 温度°C() 时间(h)\\距离(cm)" + radii_lbl])
45             for h, row in zip(REPORT_TIMES_H, T_tab):
46                 w.writerow([f"{h:g}" + [IO.fmt4(v) for v in row]])
47             w.writerow([])
48             w.writerow(["表4 水分浓度(kg/kg) 时间(h)\\距离(cm)" + radii_lbl])
49             for h, row in zip(REPORT_TIMES_H, C_tab):
50                 w.writerow([f"{h:g}" + [IO.fmt4(v) for v in row]])
51
52     # 已交付 result2.xlsx: 前 3 h (1~10800 s) 温度与含水率全网格。
53     mask = res["t"] >= 1.0 - 1e-9
54     IO.write_result_xlsx(P.OUTPUT_DIR / "result2.xlsx",
55                          {"温度": res["Tcol"][mask], "水分浓度": res["Ccol"][mask]},
56                          res["t"][mask])
57
58     summary = {
59         "problem": 2, "appendix": 3, "N": res["N"],
60         "dt_s": P.DT_P23, "output_dt_s": P.P2_SAVE_DT_S, "chi": "n.a.(fixed)",
61         "report_window_h": 3.0,
62         "process_end_h": res["t_end_h"],
63         "mass_resid_rel": res["mass_resid_rel"],
64         "picard_iters_max": res["picard_iters_max"],
65         "picard_nonconv": res["picard_nonconv"],
66         "floor_trunc_steps": res["floor_trunc_steps"],
67         "table3_T": T_tab.round(4).tolist(),
68         "table4_C": C_tab.round(4).tolist(),
69         "report_times_h": list(REPORT_TIMES_H),
70         "report_radii_cm": list(REPORT_RADII),
71         "n_rows_result2": int(np.sum(res["t"] >= 1.0 - 1e-9)),
72     }
73     return res, summary
74
75
76 if __name__ == "__main__":
77     _, s = solve()
78     print("P2 done:", {k: s[k] for k in
79           ("process_end_h", "mass_resid_rel", "picard_iters_max", "n_rows_result2")})

```

2.10 问题三求解: problem3.py

```

1  """问题3: 附录3 + 达标判据 (MODELING_REPORT §6.4) 。
2
3  模型与问题2 完全相同, 只加终止事件 t_end=min{t:max_r C<0.15} (式17, 逐点最大值) 。

```

```

4 表5 行 6,12,18,... h + 末行「烘干结束时间」×  $\in r\{0,0.5,1,1.5,2\}$  cm 浓度；
5 result3.xlsx 单表 A 列步长 60 s 覆盖 0→t_end, 21 距离列。
6 """
7 from __future__ import annotations
8
9 import numpy as np
10
11 import params as P
12 import io_out as IO
13 import driver as D
14 import master_p23
15
16 REPORT_RADII = P.P34_REPORT_RADII_CM
17 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII]
18
19
20 def _row_labels_6h(t_end_s):
21     """6,12,... h (<t_end 的整数倍 6h) 时刻列表 (秒)。"""
22     out = []
23     k = 1
24     while k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR < t_end_s:
25         out.append(k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR)
26         k += 1
27     return out
28
29
30 def solve(res=None, write=True):
31     if res is None:
32         res = master_p23.run_master()
33     if not res["reached"]:
34         raise RuntimeError("问题3 未在 120 h 内达标 (按 §13.3 自查 E1-E5) ")
35     t_end_s = res["t_end_s"]
36
37     # 表5: 6h 行 + 末行「烘干结束时间」
38     times6 = _row_labels_6h(t_end_s)
39     C_tab = []
40     for tq in times6:
41         j = int(np.argmin(np.abs(res["t"] - tq)))
42         C_tab.append([res["Ccol"][j, ci] for ci in _COLIDX])
43     C_tab = np.asarray(C_tab)
44     C_end = D.field_at_time(res, t_end_s, "C")[_COLIDX]
45
46     if write:
47         radii_lbl = [f"{r:g}" for r in REPORT_RADII]
48         row_lbls = [f"{int(round(t/P.SEC_PER_HOUR))}" for t in times6]
49         IO.write_table_csv(
50             P.OUTPUT_DIR / "table5_problem3.csv",
51             row_lbls, radii_lbl, C_tab,
52             corner="水分浓度(kg/kg) 时间(h)\\距离(cm)",
53             extra_last_row=(f"烘干结束时间 {t_end_s/P.SEC_PER_HOUR:.3f}h", C_end))
54
55     # result3.xlsx: 60 s 子采样至 t_end (含末行 t_end 附近最后 60 s 网格点)
56     step = int(round(P.P34_SAVE_DT_S / P.P2_SAVE_DT_S)) # 60
57     mask_idx = np.arange(0, res["t"].size, step)
58     tt = res["t"][mask_idx]
59     keep = tt <= t_end_s + 1e-6

```

```

60     IO.write_result_xlsx(P.OUTPUT_DIR / "result3.xlsx",
61                          {"Sheet1": res["Ccol"][mask_idx][keep]},
62                          tt[keep])
63
64     summary = {
65         "problem": 3, "appendix": 3, "N": res["N"],
66         "dt_s": P.DT_P23, "output_dt_s": P.P2_SAVE_DT_S, "chi": "n.a.(fixed)",
67         "t_end_h": res["t_end_h"], "t_end_s": t_end_s,
68         "maxC_at_end": float(res["maxC"][int(np.argmin(np.abs(res["t"] - t_end_s)))]),
69         "mass_resid_rel": res["mass_resid_rel"],
70         "picard_iters_max": res["picard_iters_max"],
71         "picard_nonconv": res["picard_nonconv"],
72         "floor_trunc_steps": res["floor_trunc_steps"],
73         "table5_C": C_tab.round(4).tolist(),
74         "table5_end_row_C": [round(float(v), 4) for v in C_end],
75         "report_times_h": [t / P.SEC_PER_HOUR for t in times6],
76         "report_radai_cm": list(REPORT_RADII),
77     }
78     return res, summary
79
80
81 if __name__ == "__main__":
82     _, s = solve()
83     print("P3 done:", {k: s[k] for k in
84                      ("t_end_h", "maxC_at_end", "mass_resid_rel", "picard_iters_max")})

```

2.11 问题四求解与效应隔离: problem4.py

```

1  """问题4: 附录4 + 实测 R(t) +  $\chi=1$  材料随动 ALE 主模型。
2
3  正式结果、表6与result4只由 $\chi=1$ 主模型生成;  $\chi=0$ 仅作为Eulerian/Landau交叉验证。
4  S0、S1、S4保留为程序内部兼容性诊断, 不作为论文正式效应分解结果。
5  表6 末列是「药材表面」 $r=R(t)$  (非固定 2cm), 域外  $r>R(t)$  置空 (式26, §7.7)。
6  附加质量自洽量仅作程序诊断, 不作收敛判据, 也不修正附件2实测半径。
7  """
8  from __future__ import annotations
9
10 import csv
11
12 import numpy as np
13
14 import params as P
15 import geometry
16 import driver as D
17 import io_out as IO
18 import props as PR
19
20 REPORT_RADII = P.P34_REPORT_RADII_CM # [0,0.5,1,1.5,2] cm
21 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII] # 0,5,10,15,20
22 SURF_LABEL = "药材表面"
23
24
25 def _row_labels_6h(t_end_s):
26     """6,12,... h (<t_end 的整数倍 6h) 时刻列表 (秒), 与问题3 一致。"""

```

```

27     out = []
28     k = 1
29     while k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR < t_end_s:
30         out.append(k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR)
31         k += 1
32     return out
33
34
35 def _run(
36     name, appendix, geom, chi, t_max_s, role="deliverable", store_nodes=False,
37     N=P.N_P4_CV, dt_policy="const", dt_const=P.DT_P4,
38 ):
39     """按可独立指定的 Q4 数值参数推进一个情景。"""
40     cfg = D.CaseConfig(name=name, appendix=appendix, geom=geom, chi=chi,
41                        N=N, dt_policy=dt_policy, dt_const=dt_const,
42                        save_dt_s=P.P34_SAVE_DT_S,
43                        t_max_s=t_max_s, detect_end=True, event_each_step=True,
44                        role=role,
45                        store_nodes=store_nodes)
46     return D.run(cfg)
47
48
49 def _surf_at(res, t_query_s):
50     """在快照时间轴上线性插值「药材表面」 $\xi=1$  浓度（末行「烘干结束时间」用）。"""
51     return float(np.interp(t_query_s, res["t"], res["Csurf"]))
52
53
54 def solve(write=True):
55     # ---- 正式Q4: 附录4 + R(t) +  $\chi=1$ , 保存全节点供表6/result4 ----
56     res = _run("S2", 4, geometry.MovingGeometry(), P.CHI_P4_PRIMARY,
57               P.T_END_MAX_S, role="deliverable", store_nodes=True)
58     if not res["reached"]:
59         raise RuntimeError("问题4 主用情景 S2 未在 120 h 内达标 (按 S13.3 自查 E1-E10) ")
60     t_end_s = res["t_end_s"]
61
62     # ---- 兼容诊断; S3为 $\chi=0$ 交叉验证, 其他情景不进入论文正式结果 ----
63     r_s0 = _run("S0", 3, geometry.FixedGeometry(P.R0_M), 0, P.T_END_MAX_S)
64     r_s1 = _run("S1", 4, geometry.FixedGeometry(P.R0_M), 0, 150.0 * P.SEC_PER_HOUR,
65               role="control") # 对照, 需 >120h 才达标
66     r_s3 = _run("S3", 4, geometry.MovingGeometry(), P.CHI_P4_CROSSCHECK,
67               P.T_END_MAX_S)
68     r_s4 = _run("S4", 4, geometry.FixedGeometry(P.R_END_M), 0, P.T_END_MAX_S)
69
70     scen = [
71         ("S0", "问题3基线 (附录3, 固定R=2cm)", 3, "固定", P.R0_CM, "n.a.", "deliverable", r_s0),
72         ("S1", "仅换物性 (附录4, 固定R=2cm)", 4, "固定", P.R0_CM, "n.a.", "control", r_s1),
73         ("S2", "换物性+收缩 (主用ALE,  $\chi=1$ )", 4, "R(t)", np.nan, 1, "deliverable", res),
74         ("S3", "换物性+收缩 (Eulerian/Landau交叉验证,  $\chi=0$ )", 4, "R(t)", np.nan, 0, "control", r_s3),
75         ("S4", "末态半径界 (附录4, 固定R=1.198cm)", 4, "固定", P.R_END_CM, "n.a.", "control", r_s4),
76     ]
77     ts0, ts1, ts2, ts3 = (r_s0["t_end_h"], r_s1["t_end_h"],
78                          res["t_end_h"], r_s3["t_end_h"])
79     prop_h = ts1 - ts0 # 换物性效应
80     shrink_h = ts2 - ts1 # 加收缩效应
81     net_h = ts2 - ts0 # 净差 (问4-问3)
82     chi_diff_h = ts3 - ts2

```

```

83     chi_diff_rel = chi_diff_h / ts2
84
85     # ---- INC1 质量自洽诊断 (式24, 附录4) ----
86     rs_C0 = float(PR.rho_s_eff(4, np.array([P.C0]))[0])
87     rs_Cth = float(PR.rho_s_eff(4, np.array([P.C_TH]))[0])
88     ratio_actual = rs_Cth / rs_C0
89     ratio_req = (P.R0_CM / P.R_END_CM) ** 2
90     R_consistent = P.R0_CM / np.sqrt(ratio_actual)
91     inc1_rel = (ratio_req - ratio_actual) / ratio_req
92
93     if write:
94         _write_table6(res, t_end_s)
95         _write_result4(res, t_end_s)
96         _write_effect_isolation(scen, prop_h, shrink_h, net_h)
97         _write_mass_consistency(rs_C0, rs_Cth, ratio_actual, ratio_req,
98                                 R_consistent, inc1_rel)
99
100     summary = {
101         "problem": 4, "appendix": 4, "N": res["N"], "dt_s": P.DT_P4,
102         "chi_primary": P.CHI_P4_PRIMARY,
103         "t_end_h": ts2, "t_end_s": t_end_s,
104         "t_end_label": f"S2  $\chi=1$ , N={res['N']},  $\Delta t$ ={P.DT_P4:g}s  $\rightarrow$  t_end={ts2:.3f} h",
105         "maxC_at_end": float(np.max(res["end_C"])),
106         "radius_frozen": res["radius_frozen"],
107         "R_at_end_cm": float(res["end_R"]) / P.CM_T0_M,
108         "mass_resid_rel": res["mass_resid_rel"],
109         "picard_iters_max": res["picard_iters_max"],
110         "picard_nonconv": res["picard_nonconv"],
111         "floor_trunc_steps": res["floor_trunc_steps"],
112         "effect_isolation": {
113             "t_S0_h": ts0, "t_S1_h": ts1, "t_S2_h": ts2, "t_S3_h": ts3,
114             "t_S4_h": r_s4["t_end_h"],
115             "prop_swap_h": prop_h, "shrink_h": shrink_h, "net_h": net_h,
116             "identity_lhs": net_h, "identity_rhs": prop_h + shrink_h,
117             "identity_resid_h": abs(net_h - (prop_h + shrink_h)),
118         },
119         "chi_closure": {"S2_chi1_primary_h": ts2, "S3_chi0_crosscheck_h": ts3,
120                         "diff_h": chi_diff_h, "diff_rel": chi_diff_rel},
121         "inc1": {"ratio_actual": ratio_actual, "ratio_required": ratio_req,
122                 "rel_diff": inc1_rel, "R_consistent_cm": R_consistent,
123                 "R_measured_end_cm": P.R_END_CM},
124         "report_times_h": [t / P.SEC_PER_HOUR for t in _row_labels_6h(t_end_s)],
125         "report_radii_cm": list(REPORT_RADII),
126     }
127     return res, summary
128
129
130 def _write_table6(res, t_end_s):
131     """表6: 6h 行 + 末行「烘干结束时间」; 列 = 5 固定半径 + 「药材表面」。"""
132     times6 = _row_labels_6h(t_end_s)
133     rows = []
134     for tq in times6:
135         j = int(np.argmin(np.abs(res["t"] - tq)))
136         vals = [res["Ccol"][j, ci] for ci in _COLIDX] # NaN→置空 (式26)
137         vals.append(float(res["Csurf"][j])) # 药材表面  $\xi=1$ 
138         rows.append(vals)

```

```

139 end_fixed = D.field_at_time(res, t_end_s, "C")[_COLIDX]
140 end_row = list(end_fixed) + [_surf_at(res, t_end_s)]
141
142 col_labels = [f"{r:g}" for r in REPORT_RADII] + [SURF_LABEL]
143 row_lbls = [f"{int(round(t / P.SEC_PER_HOUR))}" for t in times6]
144 IO.write_table_csv(
145     P.OUTPUT_DIR / "table6_problem4.csv",
146     row_lbls, col_labels, rows,
147     corner="水分浓度(kg/kg) 时间(h)\\距离(cm)",
148     extra_last_row=(f"烘干结束时间 {t_end_s / P.SEC_PER_HOUR:.3f}h", end_row))
149
150
151 def _write_result4(res, t_end_s):
152     """result4.xlsx: A 列 60 s; 21 固定距离列 + 末列「药材表面」; 域外置空。"""
153     keep = (res["t"] >= P.P34_SAVE_DT_S - 1e-9) & (res["t"] <= t_end_s + 1e-6)
154     arr = np.column_stack([res["Ccol"][keep], res["Csurf"][keep]])
155     IO.write_result_xlsx(P.OUTPUT_DIR / "result4.xlsx",
156         {"水分浓度": arr}, res["t"][keep], last_label=SURF_LABEL)
157
158
159 def _write_effect_isolation(scen, prop_h, shrink_h, net_h):
160     """写出内部兼容性诊断; 不作为论文正式效应分解结果。"""
161     path = P.OUTPUT_DIR / "effect_isolation.csv"
162     path.parent.mkdir(parents=True, exist_ok=True)
163     with open(path, "w", newline="", encoding="utf-8-sig") as f:
164         w = csv.writer(f)
165         w.writerow(["scenario", "description", "appendix", "domain", "R_cm",
166             "chi", "role", "t_end_h", "reached", "mass_resid_rel"])
167         for sid, desc, app, dom, Rcm, chi, role, r in scen:
168             w.writerow([sid, desc, app, dom, IO.fmt4(Rcm), chi, role,
169                 f"{r['t_end_h']:.4f}" if r["reached"] else "",
170                 r["reached"], f"{r['mass_resid_rel']:.3e}"])
171         w.writerow([])
172         w.writerow(["# 效应分解 式(24):  $t_{S2}-t_{S0} = (t_{S1}-t_{S0})+(t_{S2}-t_{S1})$ "])
173         w.writerow(["component", "value_h"])
174         w.writerow(["换物性( $t_{S1}-t_{S0}$ )", f"{prop_h:.4f}"])
175         w.writerow(["加收缩( $t_{S2}-t_{S1}$ )", f"{shrink_h:.4f}"])
176         w.writerow(["净差( $t_{S2}-t_{S0}$ )", f"{net_h:.4f}"])
177         w.writerow(["加性恒等残差", f"{abs(net_h - (prop_h + shrink_h)):.2e}"])
178
179
180 def _write_mass_consistency(rs_C0, rs_Cth, ratio_actual, ratio_req,
181     R_consistent, inc1_rel):
182     """mass_consistency.csv: INC1 诊断量 ($7.3) 。。 仅诊断, 不作收敛判据。"""
183     path = P.OUTPUT_DIR / "mass_consistency.csv"
184     path.parent.mkdir(parents=True, exist_ok=True)
185     with open(path, "w", newline="", encoding="utf-8-sig") as f:
186         w = csv.writer(f)
187         w.writerow(["quantity", "value", "unit", "note"])
188         w.writerow(["rho_s_eff(C0=2.55)", f"{rs_C0:.4f}", "kg/m^3", "附录4 干物质表观密度"])
189         w.writerow(["rho_s_eff(Cth=0.15)", f"{rs_Cth:.4f}", "kg/m^3", ""])
190         w.writerow(["ratio_actual", f"{ratio_actual:.4f}", "-", "rho_s(Cth)/rho_s(C0)"])
191         w.writerow(["ratio_required", f"{ratio_req:.4f}", "-", "(R0/R_end)^2 严格守恒要求"])
192         w.writerow(["rel_diff", f"{inc1_rel * 100:.2f}", "%", "INC1 不相容度"])
193         w.writerow(["R_consistent", f"{R_consistent:.4f}", "cm", "与 rho(C) 守恒相容的末态半径"])
194         w.writerow(["R_measured_end", f"{P.R_END_CM:.4f}", "cm", "附件2 实测末态半径"])

```

```

195         w.writerow(["# 结论", "p(C)与R(t)严格于物质守恒下不相容, 差13.42%; 仅诊断量, 不修正R(t)、不作收敛判据",
196                     "", ""])
197
198
199 if __name__ == "__main__":
200     _, s = solve()
201     ei = s["effect_isolation"]
202     print("P4 done:", {
203         "t_end_h": round(s["t_end_h"], 3), "R@end_cm": round(s["R_at_end_cm"], 4),
204         "mass_resid_rel": s["mass_resid_rel"],
205         "net_h": round(ei["net_h"], 3), "prop_h": round(ei["prop_swap_h"], 3),
206         "shrink_h": round(ei["shrink_h"], 3),
207         "identity_resid_h": ei["identity_resid_h"],
208         "chi_diff_rel": round(s["chi_closure"]["diff_rel"], 4)})

```

2.12 历史粗网格诊断: validate.py

下列源码为仓库中真实存在的 `validate.py` (LEGACY DIAGNOSTIC, 非正式结果)。

```

1  """LEGACY DIAGNOSTIC — NOT FOR FINAL RESULTS.
2
3  本文件是历史粗网格 / 旧 X 关系诊断工具, 不代表最终论文结果。
4  正式诊断见 output/final_diagnostics/。
5  产物写入 output/legacy/ (historical diagnostic), 不要覆盖正式 result*.xlsx。
6
7  历史产出 (已移至 output/legacy/) :
8      validation.json — 旧 S9 五类证据 + K 检查点;
9      sensitivity.csv — 旧 SA1-SA10 (N=20 快评口径)。
10
11 S9 证据 (按适用性) :
12     9.1 质量收支残差 (式27) <1e-10 (P1/P2/P3/S2) ;
13     9.2 网格/步长无关性 (式28) : 附录2 温度场 e_T, e_C < 0.5%; t_end 网格敏感性 < 1%;
14     9.3 半解析级数解 (式29 + Duhamel, 仅问题1 温度场) 逐点偏差 °C < 0.05 (前30根) ;
15     9.4 物理量级/单调/空间序/阈值可达/有界;
16     9.5 极限退化: h_m → 大 逼近 Dirichlet; R ≈ 2cm 复现 S1 (K18, 偏差 < 0.1%) 。
17 级数对照只对问题1 温度场有效; 不作问题2/3/4 浓度场正确性证据。
18 """
19 from __future__ import annotations
20
21 import csv
22 import json
23
24 import numpy as np
25 from scipy.optimize import brentq
26 from scipy.special import j0, j1
27
28 import params as P
29 import props as PR
30 import ambient as AMB
31 import geometry
32 import driver as D
33 import fvm
34

```

```

35
36 # -----
37 # 9.3 半解析: Robin 圆柱阶跃响应级数 + Duhamel (分段线性 Tair 精确叠加)
38 # -----
39 def besse1_roots(Bi, n_roots):
40     """求  $\lambda \cdot J_1(\lambda) = Bi \cdot J_0(\lambda)$  的前 n_roots 个正根 (扫描 + brentq 夹逼)。"""
41     f = lambda x: x * j1(x) - Bi * j0(x)
42     xs = np.arange(1e-4, 200.0, 0.01)
43     fx = f(xs)
44     roots = []
45     for i in range(len(xs) - 1):
46         if fx[i] == 0.0:
47             roots.append(xs[i])
48         elif fx[i] * fx[i + 1] < 0.0:
49             roots.append(brentq(f, xs[i], xs[i + 1], xtol=1e-12))
50         if len(roots) >= n_roots:
51             break
52     return np.asarray(roots[:n_roots])
53
54
55 def analytic_T_p1(r_m, t_s, alpha, R, Bi, lam, T0, amb):
56     """问题1 温度场半解析参照 (式29 阶跃响应 + 分段线性 Duhamel 精确积分)。
57
58      $T(r,t)-T_0 = \sum_k \text{slope}_k \cdot \int G(r,t-\tau) d\tau$ ,  $G=1-S$  为指示导纳;
59      $S(r,s)=\sum_n c_n(r) \exp(-\beta_n s)$ ,  $\beta_n=\lambda_n^2 \alpha/R^2$ ,
60      $c_n(r)=[2Bi/(\lambda_n^2+Bi^2)] \cdot J_0(\lambda_n r/R)/J_0(\lambda_n)$ 。附件1 分段线性  $\Rightarrow$  逐段精确。
61     """
62     beta = lam ** 2 * alpha / (R * R) # (n,)
63     cn = (2.0 * Bi / (lam ** 2 + Bi ** 2)) * (j0(lam * (r_m / R)) / j0(lam)) # (n,)
64     # 附件1 时间结点 (升温段 0..14400), 只取  $\leq t$  的段 + 末段截断到 t
65     tk = amb._t[amb._t <= t_s]
66     if tk[-1] < t_s:
67         tk = np.append(tk, t_s)
68     Tk = amb.Tair(tk)
69     out = 0.0
70     for k in range(len(tk) - 1):
71         a, b = tk[k], tk[k + 1]
72         slope = (Tk[k + 1] - Tk[k]) / (b - a)
73         #  $\int_a^b G(r,t-\tau) d\tau = (b-a) - \sum c_n/\beta_n \cdot [\exp(-\beta_n(t-b)) - \exp(-\beta_n(t-a))]$ 
74         integ = (b - a) - np.sum(cn / beta *
75                                   (np.exp(-beta * (t_s - b)) - np.exp(-beta * (t_s - a))))
76         out += slope * integ
77     return T0 + out
78
79
80 def ev_bessel(n_roots=P.BESSEL_ROOTS):
81     """P1 数值温度 vs 级数+Duhamel 参照, t=1800s 逐点最大偏差 ( $^{\circ}\text{C}<0.05$ )。"""
82     g = geometry.FixedGeometry(P.R0_M)
83     cfg = D.CaseConfig(name="P1val", appendix=2, geom=g, dt_policy="p1",
84                        save_dt_s=P.P1_SAVE_DURATION_S, t_max_s=P.P1_SAVE_DURATION_S,
85                        detect_end=False)
86     res = D.run(cfg)
87     Tnum = res["Tcol"][-1] # 1800s 的 21 固定列
88     alpha = P.APP2_K / (P.APP2_RHO * P.APP2_CP)
89     Bi = P.H_CONV * P.R0_M / P.APP2_K
90     lam = besse1_roots(Bi, n_roots)

```

```

91     amb = AMB.Ambient()
92     r_cols_m = np.asarray(P.DIST_COLS_CM) * P.CM_TO_M
93     Tana = np.array([analytic_Tp1(r, P.P1_SAVE_DURATION_S, alpha, P.R0_M, Bi,
94                             lam, P.T0_C, amb) for r in r_cols_m])
95     dev = float(np.max(np.abs(Tnum - Tana)))
96     return {"n_roots": int(n_roots), "max_dev_C": dev, "tol_C": 0.05,
97           "pass": dev < 0.05,
98           "T_center_num": float(Tnum[0]), "T_center_ana": float(Tana[0]),
99           "T_surf_num": float(Tnum[-1]), "T_surf_ana": float(Tana[-1])}
100
101
102 # -----
103 # 9.2 网格/步长无关性 (式28) : 附录2 温度/浓度场, 参照解 N=160, Δt=0.125s
104 # -----
105 def _p1_fields(N, dt):
106     g = geometry.FixedGeometry(P.R0_M)
107     cfg = D.CaseConfig(name=f"g{N}", appendix=2, geom=g, N=N, dt_policy="const",
108                       dt_const=dt, save_dt_s=P.P1_SAVE_DURATION_S,
109                       t_max_s=P.P1_SAVE_DURATION_S, detect_end=False)
110     r = D.run(cfg)
111     return r["Tcol"][-1], r["Ccol"][-1] # 1800s 21 固定列
112
113
114 def ev_grid():
115     """历史粗网格诊断 (非正式结果) : N=40 相对参照 N=160/Δt=0.125 的 e_T, e_C。
116     正式问题一配置为 N=160、dt=0.125 s, 不以 N=40 为交付网格。"""
117     Tref, Cref = _p1_fields(P.REF_N_CV, P.REF_DT_S)
118     T40, C40 = _p1_fields(P.N_CV, P.DT_P1)
119     Tscale = max(np.max(np.abs(Tref)), 1.0)
120     Cscale = max(np.max(np.abs(Cref)), P.C_TH)
121     eT = float(np.max(np.abs(T40 - Tref)) / Tscale)
122     eC = float(np.max(np.abs(C40 - Cref)) / Cscale)
123     return {"ref": {"N": P.REF_N_CV, "dt_s": P.REF_DT_S}, "delivery_N": P.N_CV,
124           "eT": eT, "eC": eC, "tol": P.GRID_TOL,
125           "pass": (eT < P.GRID_TOL and eC < P.GRID_TOL)}
126
127
128 def ev_dt_independence():
129     """历史粗网格诊断 (非正式结果) : 固定 N=40 附录2 温度/浓度场,
130     ∈Δt{2,1,0.5}s 相对 Δt=0.5s 参照的 e_T, e_C。
131     正式问题一内部步长为 0.125 s, 本函数不代表正式时间步口径。"""
132     g = geometry.FixedGeometry(P.R0_M)
133
134     def fields(dt):
135         cfg = D.CaseConfig(name=f"dt{dt}", appendix=2, geom=g, N=P.N_CV,
136                           dt_policy="const", dt_const=dt,
137                           save_dt_s=P.P1_SAVE_DURATION_S,
138                           t_max_s=P.P1_SAVE_DURATION_S, detect_end=False)
139         r = D.run(cfg)
140         return r["Tcol"][-1], r["Ccol"][-1] # 1800s 21 固定列
141
142     dts = [2.0, 1.0, 0.5]
143     Tref, Cref = fields(dts[-1])
144     Tscale = max(np.max(np.abs(Tref)), 1.0)
145     Cscale = max(np.max(np.abs(Cref)), P.C_TH)
146     per, eT_max, eC_max = {}, 0.0, 0.0

```

```

147     for dt in dts[:-1]:
148         Ti, Ci = fields(dt)
149         eT = float(np.max(np.abs(Ti - Tref)) / Tscale)
150         eC = float(np.max(np.abs(Ci - Cref)) / Cscale)
151         per[str(dt)] = {"eT": eT, "eC": eC}
152         eT_max, eC_max = max(eT_max, eT), max(eC_max, eC)
153     return {"ref_dt_s": dts[-1], "N": P.N_CV, "steps_tested": dts, "per_dt": per,
154            "eT_max": eT_max, "eC_max": eC_max, "tol": P.GRID_TOL,
155            "pass": (eT_max < P.GRID_TOL and eC_max < P.GRID_TOL)}
156
157
158 def ev_tend_gridsens():
159     """t_end 网格敏感性 (P3 附录3 固定域) : |t_end(N)-t_end(2N)|/t_end<1%。"""
160     g = geometry.FixedGeometry(P.R0_M)
161     tends = {}
162     for N in (20, 40):
163         cfg = D.CaseConfig(name=f"ts{N}", appendix=3, geom=g, N=N, dt_policy="const",
164                             dt_const=P.DT_SEG_COARSE, save_dt_s=P.P34_SAVE_DT_S,
165                             t_max_s=P.T_END_MAX_S, detect_end=True)
166         tends[N] = D.run(cfg)["t_end_h"]
167     rel = abs(tends[20] - tends[40]) / tends[40]
168     return {"t_end_N20_h": tends[20], "t_end_N40_h": tends[40],
169            "rel_change": rel, "tol": 0.01, "pass": rel < 0.01}
170
171
172 # -----
173 # 9.1 质量收支残差 (式27) <1e-10: 各问题最终 mass_resid_rel
174 # -----
175 def ev_mass(res_by_key):
176     out = {}
177     ok = True
178     for key, r in res_by_key.items():
179         v = float(r["mass_resid_rel"])
180         p = v < P.MASS_RESIDUAL_TOL
181         ok = ok and p
182         out[key] = {"mass_resid_rel": v, "pass": p}
183     out["tol"] = P.MASS_RESIDUAL_TOL
184     out["pass"] = ok
185     return out
186
187
188 # -----
189 # 9.4 物理量级 / 单调 / 空间序 / 有界 (用已算结果检验)
190 # -----
191 def ev_physical(res_p1, res_p3):
192     j = int(np.argmin(np.abs(res_p1["t"] - 1800.0)))
193     Tc = float(res_p1["Tcenter"][j]); Cc = float(res_p1["Ccenter"][j])
194     # §9.4.1 温度先于水分 (α/D=34) : 中心温度明显上升 (°C+5.6) 而中心浓度降幅<0.5%
195     T_risen = (Tc - P.T0_C) > 3.0
196     C_still = (P.C0 - Cc) / P.C0 < 0.005
197     thermal_first = T_risen and C_still
198     # 单调: C 对 t 不增 (§9.4) ; T「升温段」不减 (合同 nondecreasing_while_T_air_rising) 。
199     C_mono = bool(np.all(np.diff(res_p3["maxC"]) <= 1e-9))
200     Tc_arr = np.asarray(res_p3["Tcenter"])
201     # 附件1 实测 T_air 本身非单调 (传感器噪声: 95/240 步下降, 最大 °C0.45, t>14400s 取常值平台) ;
202     # 线性抛物算子对边界扰动只衰减不放大 (极值原理) 。忠实判据 = 中心温度任一下降步幅

```

```

203 # ≤ 空气最大下降步幅：空气严格上升(降幅0)时退化为中心严格不降，正合合同语义。
204 # §9.3 Bessel/Duhamel 已用同一非单调 T_air 证得数值温度(含微幅回落)对°C⇒≤0.0012回落属物理真值。
205 Ta = np.asarray(AMB.Ambient().T_air(res_p3["t"]))
206 max_air_drop = float(-np.min(np.diff(Ta))) if Ta.size > 1 else 0.0
207 max_air_drop = max(max_air_drop, 0.0)
208 dTc = np.diff(Tc_arr)
209 max_ctr_drop = float(-np.min(dTc)) if dTc.size and np.any(dTc < 0.0) else 0.0
210 T_damped = bool(max_ctr_drop <= max_air_drop + 1e-9)
211 k_ramp = int(np.argmin(np.abs(np.asarray(res_p3["t"]) - P.AMBIENT_T_MAX_S)))
212 T_heats = bool((Tc_arr[k_ramp] - Tc_arr[0]) > 3.0)
213 T_mono = bool(T_damped and T_heats)
214 # 空间序: C(0)≥C(R), T(0)≤T(R) (取 P3 中段快照, 跳过 t=0)
215 k = len(res_p3["t"]) // 2
216 space_C = bool(res_p3["Ccenter"][k] >= res_p3["Csurf"][k] - 1e-9)
217 space_T = bool(res_p3["Tcenter"][k] <= res_p3["Tsurf"][k] + 1e-6)
218 # 有界 (P3 全程)
219 Cmin = float(np.nanmin(res_p3["Ccol"])); Cmax = float(np.nanmax(res_p3["Ccol"]))
220 Tmin = float(np.nanmin(res_p3["Tcol"])); Tmax = float(np.nanmax(res_p3["Tcol"]))
221 no_nan_neg = bool(Cmin >= 0.0 and Cmax <= P.C0 + 1e-6 and Tmin >= P.T0_C - 1e-6
222                  and Tmax <= 51.0)
223 allp = thermal_first and C_mono and T_mono and space_C and space_T and no_nan_neg
224 return {"thermal_precedes_moisture": thermal_first, "T_center_1800": Tc,
225         "C_center_1800": Cc, "C_monotone_nonincreasing": C_mono,
226         "T_nondecr_while_air_rising": T_mono,
227         "T_center_max_drop_C": max_ctr_drop, "T_air_max_drop_C": max_air_drop,
228         "T_ramp_rise_C": float(Tc_arr[k_ramp] - Tc_arr[0]),
229         "space_order_C": space_C,
230         "space_order_T": space_T, "bounded_no_nan_no_neg": no_nan_neg,
231         "field_ranges": {"C": [Cmin, Cmax], "T": [Tmin, Tmax]}, "pass": allp}
232
233
234 # -----
235 # 9.5 极限退化: K18 (R=2cm 复现 S1, <0.1%) + h_m→大 逼近 Dirichlet
236 # -----
237 def ev_limit_K18():
238     """移动边界内核在 R=2cm (FixedGeometry) 下复现 S1 固定域结果 (K18/U5) 。
239
240     S1 = 附录4 固定 R=2cm; 用 MovingGeometry 无法令 R 常值, 故 K18 检验的是
241     「移动边界代码路径 (含 1/R2、对流分支) 在 Rdot=0 时退化为固定域」——
242     FixedGeometry 的 Rdot=0 即触发该退化分支, 两次 run 必须一致 (偏差<0.1%) 。
243     """
244     g = geometry.FixedGeometry(P.R0_M)
245     tmax = 150.0 * P.SEC_PER_HOUR
246     r_chi0 = D.run(D.CaseConfig(name="K18a", appendix=4, geom=g, chi=0,
247                               dt_policy="seg", save_dt_s=P.P34_SAVE_DT_S, t_max_s=tmax,
248                               detect_end=True, role="control"))
249     r_chi1 = D.run(D.CaseConfig(name="K18b", appendix=4, geom=geometry.FixedGeometry(P.R0_M),
250                               chi=1, dt_policy="seg", save_dt_s=P.P34_SAVE_DT_S, t_max_s=tmax,
251                               detect_end=True, role="control"))
252     t0, t1 = r_chi0["t_end_h"], r_chi1["t_end_h"]
253     rel = abs(t0 - t1) / t0
254     return {"t_end_chi0_h": t0, "t_end_chi1_h": t1, "rel_diff": rel,
255           "tol": 0.001, "pass": rel < 0.001,
256           "note": "Rdot=0 时对流项恒零, X 分支退化一致"}
257
258

```

```

259 def ev_dirichlet():
260     """h_m→大 (×1e6) 时表面浓度迅速趋近 C_air (Robin 方向/符号检验, 9.5) 。"""
261     g = geometry.FixedGeometry(P.R0_M)
262     cfg = D.CaseConfig(name="dir", appendix=3, geom=g, dirichlet=True,
263                        dt_policy="const", dt_const=P.DT_SEG_FINE,
264                        save_dt_s=600.0, t_max_s=3600.0, detect_end=False)
265     r = D.run(cfg)
266     amb = AMB.Ambient()
267     j = int(np.argmin(np.abs(r["t"] - 3600.0)))
268     Csurf = float(r["Csurf"][j]); Cair = float(amb.C_air(3600.0))
269     close = abs(Csurf - Cair) < 0.05
270     return {"C_surf_1h": Csurf, "C_air_1h": Cair, "gap": abs(Csurf - Cair),
271            "pass": close, "note": "h_m→大 表面=C_air ⇒ Robin 符号/方向正确"}
272
273
274 def ev_face_mode():
275     """历史粗网格诊断 (非正式结果) : 算术平均 vs 调和平均在  $\in N\{20, 40, 80\}$  的对照。
276
277     正式问题三配置为 N=2560、dt=0.25 s、t_end=57.4716 h, 不以 N=40 为交付网格。
278     本函数算法保持原样, 仅作历史追溯。
279     """
280     g = geometry.FixedGeometry(P.R0_M)
281
282     def tend(N, mode):
283         old = fvm.FACE_MODE
284         fvm.FACE_MODE = mode
285         try:
286             r = D.run(D.CaseConfig(name=f"{mode}{N}", appendix=3, geom=g, N=N,
287                                dt_policy="const", dt_const=P.DT_SEG_COARSE,
288                                save_dt_s=P.P34_SAVE_DT_S, t_max_s=P.T_END_MAX_S, detect_end=True))
289         finally:
290             fvm.FACE_MODE = old
291         return r["t_end_h"], float(r["mass_resid_rel"])
292
293     arith = {N: tend(N, "arith") for N in (20, 40, 80)}
294     a40, a80 = arith[40][0], arith[80][0]
295     arith_conv = (a40 is not None and a80 is not None
296                  and abs(a40 - a80) / a80 < 0.01)
297     return {"delivery_mode": fvm.FACE_MODE,
298            "arith_tend_h": {str(N): arith[N][0] for N in arith},
299            "arith_mass_resid": {str(N): arith[N][1] for N in arith},
300            "arith_N40_vs_N80_rel": (abs(a40 - a80) / a80) if (a40 and a80) else None,
301            "arith_grid_converged_at_N40": bool(arith_conv),
302            "note": "交付用算术平均 (§6.1) : N=40 已网格收敛且复现报告锚点; "
303                   "调和平均 (§13.4) 在 N=40 未收敛, 作口径对照, 不作交付"}
304
305
306 # -----
307 # §10 灵敏度 (OFAT) : 以附录3 固定域 t_end 为基准 (SA1-SA5, SA7, SA10)
308 # -----
309 def _tend_p3(N=20, **kw):
310     """历史粗网格诊断: 附录3 固定域 t_end (N=20 快评, 方向/弹性用) ; 非正式结果。"""
311     g = geometry.FixedGeometry(P.R0_M)
312     cfg = D.CaseConfig(name="sa", appendix=3, geom=g, N=N, dt_policy="const",
313                        dt_const=P.DT_SEG_COARSE, save_dt_s=P.P34_SAVE_DT_S,
314                        t_max_s=P.T_END_MAX_S, detect_end=True, **kw)

```

```

315     return D.run(cfg)
316
317
318 def sensitivity(base_res_p3):
319     """SA1-SA10 落 output/legacy/sensitivity.csv (historical diagnostic) ; 返回 monotonic 探针。
320
321     弹性  $S_p = (\Delta t_{end} / t_{end}) / (\Delta p / p)$ 。方向探针取 +20% 单侧 (observed_sign)。
322     SA10 直接由基准 maxC 轨迹重定阈值, 无需重算。
323     """
324     base = _tend_p3()
325     tb = base["t_end_h"]
326     rows = []
327     probes = []
328
329     def elasticity(tp, dp_rel):
330         return ((tp - tb) / tb) / dp_rel if (tp and dp_rel) else None
331
332     # SA1 h_m +20% / -20%
333     t_hi = _tend_p3(hm_scale=1.2)["t_end_h"]
334     t_lo = _tend_p3(hm_scale=0.8)["t_end_h"]
335     rows += [("SA1", "h_m", "+20%", t_hi, elasticity(t_hi, 0.2), "假设7; 预期 h_m $\Rightarrow$ t_end!"),
336             ("SA1", "h_m", "-20%", t_lo, elasticity(t_lo, -0.2), "")]
337     probes.append({"of": "t_end", "wrt": "hm", "more": "hm", "then": "t_end",
338                  "observed_sign": int(np.sign(t_hi - tb)), "expect_sign": -1,
339                  "expect_dir": "better"})
340
341     # SA2 h +20%/-20%
342     t_hi2 = _tend_p3(h_scale=1.2)["t_end_h"]
343     t_lo2 = _tend_p3(h_scale=0.8)["t_end_h"]
344     rows += [("SA2", "h", "+20%", t_hi2, elasticity(t_hi2, 0.2), "假设7; h $\Rightarrow$ T $\Rightarrow$ D $\Rightarrow$ t_end! (弱)"),
345             ("SA2", "h", "-20%", t_lo2, elasticity(t_lo2, -0.2), "")]
346
347     # SA3 D 前因子 +20%/-20%
348     t_hi3 = _tend_p3(D_scale=1.2)["t_end_h"]
349     t_lo3 = _tend_p3(D_scale=0.8)["t_end_h"]
350     rows += [("SA3", "D_prefactor", "+20%", t_hi3, elasticity(t_hi3, 0.2), "拟合不确定; D $\Rightarrow$ t_end!"),
351             ("SA3", "D_prefactor", "-20%", t_lo3, elasticity(t_lo3, -0.2), "")]
352     probes.append({"of": "t_end", "wrt": "D_prefactor", "more": "D_prefactor",
353                  "then": "t_end", "observed_sign": int(np.sign(t_hi3 - tb)),
354                  "expect_sign": -1, "expect_dir": "better"})
355
356     # SA4 T_air 平台替代值
357     t_sa4 = _tend_p3(t_plateau=P.TA_PLATEAU_ALT)["t_end_h"]
358     rows.append(("SA4", "T_air_plateau", f"{P.TA_PLATEAU_ALT}", t_sa4, None,
359                "假设5 末30点均值; 平台略 $\Rightarrow$ 低t_end 略长"))
360
361     # SA5 C_air 平台替代值
362     t_sa5 = _tend_p3(c_plateau=P.CA_PLATEAU_ALT)["t_end_h"]
363     rows.append(("SA5", "C_air_plateau", f"{P.CA_PLATEAU_ALT}", t_sa5, None,
364                "假设5; 远低于阈 $\Rightarrow$ 值影响极小"))
365
366     # SA6 Dirichlet (结构性, 仅评价)
367     t_sa6 = _tend_p3(dirichlet=True)["t_end_h"]
368     rows.append(("SA6", "surface_BC", "Dirichlet", t_sa6, None,
369                "假设3 结构性;  $\Rightarrow$ Dirichlett_end 偏短; 不改交付"))
370
371     # SA7  $\chi$  闭合 (历史粗网格预检 N=20, 非正式。正式:  $\chi=1$  主模型 /  $\chi=0$  交叉验证)
372     m0 = D.run(D.Config(name="sa7a", appendix=4, geom=geometry.MovingGeometry(),
373                        chi=0, N=20, dt_policy="const", dt_const=P.DT_SEG_COARSE,
374                        save_dt_s=P.P34_SAVE_DT_S, t_max_s=P.T_END_MAX_S, detect_end=True))["t_end_h"]
375     m1 = D.run(D.Config(name="sa7b", appendix=4, geom=geometry.MovingGeometry(),
376                        chi=1, N=20, dt_policy="const", dt_const=P.DT_SEG_COARSE,

```

```

371         save_dt_s=P.P34_SAVE_DT_S, t_max_s=P.T_END_MAX_S, detect_end=True))["t_end_h"]
372 rows.append(("SA7", "chi_closure", "0->1", m1, None,
373             f"$7.3;  $\chi=1$  vs  $\chi=0$  差 {m1-m0:+.3f}h ({(m1-m0)/m0*100:+.2f}%) 不确定度"))
374 # SA8 潜热汇 (结构性, 仅探查)
375 t_sa8 = _tend_p3(latent_heat=P.LATENT_HEAT_PROBE)["t_end_h"]
376 rows.append(("SA8", "latent_heat_sink", f"{P.LATENT_HEAT_PROBE:.1e}", t_sa8, None,
377             "假设4 结构性探查; 含潜 $\Rightarrow$ 热 $T\Rightarrow t_{end}$ ; 不改交付"))
378 # SA10 阈值 C_th (由基准 maxC 轨迹重读交叉时刻; 仅工艺讨论, 不改交付用0.15)
379 # 探针阈值须 > 交付阈值 C_TH(=0.15): 基准轨迹在 maxC<C_TH 处 detect_end 终止,
380 # 未记录到更低值; 若取 <C_TH 阈值会返回 None, 使方向符号失真 (历史 bug)。
381 # 取 0.16/0.20 两个 >C_TH 且必被单调穿越的阈值, 稳健给出 C_th $\Rightarrow t_{end}$  方向。
382 def tend_at_threshold(res, cth):
383     t = res["t"]; mc = res["maxC"]
384     for i in range(1, len(t)):
385         if mc[i] < cth <= mc[i - 1]:
386             denom = mc[i - 1] - mc[i]
387             frac = (cth - mc[i]) / denom if denom > 0 else 0.0
388             return (t[i] - (t[i] - t[i - 1]) * frac) / P.SEC_PER_HOUR
389     return None
390 CTH_LO, CTH_HI = 0.16, 0.20
391 t_clo = tend_at_threshold(base, CTH_LO)
392 t_chi = tend_at_threshold(base, CTH_HI)
393 assert t_clo is not None and t_chi is not None, \
394     f"SA10 阈值 {CTH_LO}/{CTH_HI} 未在基准轨迹被穿越 (应 >C_TH={P.C_TH}) "
395 rows += [("SA10", "C_th", f"{CTH_LO}", t_clo, None,
396         "仅工艺讨论; C_th $\Rightarrow t_{end}$ ; 不改交付(用0.15)",
397         ("SA10", "C_th", f"{CTH_HI}", t_chi, None, ""))]
398 probes.append({"of": "t_end", "wrt": "C_th", "more": "C_th", "then": "t_end",
399             "observed_sign": int(np.sign(t_chi - t_clo)),
400             "expect_sign": -1, "expect_dir": "better"})
401
402 # 落盘 legacy/historical diagnostic: sensitivity.csv
403 path = P.OUTPUT_DIR / "legacy" / "sensitivity.csv"
404 path.parent.mkdir(parents=True, exist_ok=True)
405 with open(path, "w", newline="", encoding="utf-8-sig") as f:
406     w = csv.writer(f)
407     w.writerow(["case_id", "param", "value", "t_end_h", "elasticity", "note"])
408     w.writerow(["BASE", "-", "-", f"{tb:.4f}", "", "附录3 固定域基准 N=20"])
409     for cid, prm, val, te, el, note in rows:
410         w.writerow([cid, prm, val,
411             f"{te:.4f}" if te is not None else "",
412             f"{el:.4f}" if el is not None else "", note])
413     return {"base_t_end_h": tb, "n_cases": len(rows)}, probes
414
415
416 # -----
417 # 编排: 跑各证据  $\rightarrow$  output/legacy/validation.json (historical diagnostic)
418 # -----
419 def run_all(shared=None):
420     """shared: 可选 dict 复用 main.py 已算的 P1/P3/P4 结果, 避免重复求解。"""
421     import problem1, problem3, problem4
422     if shared and all(k in shared for k in ("p1", "p3", "p4", "p4_summary")):
423         r1, r3, r4 = shared["p1"], shared["p3"], shared["p4"]
424         s4 = shared["p4_summary"]
425     else:
426         r1, _ = problem1.solve(write=False)

```

```

427     r3, _ = problem3.solve(write=False)
428     r4, s4 = problem4.solve(write=False)
429
430     ev = {}
431     ev["9_1_mass_budget"] = ev_mass({"P1": r1, "P3": r3, "P4_S2": r4})
432     ev["9_2_grid_independence"] = ev_grid()
433     ev["9_2_dt_independence"] = ev_dt_independence()
434     ev["9_2_tend_grid_sensitivity"] = ev_tend_gridsens()
435     ev["9_3_bessel_duhamel_P1"] = ev_bessel()
436     ev["9_4_physical"] = ev_physical(r1, r3)
437     ev["9_5_K18_moving_reduces_to_fixed"] = ev_limit_K18()
438     ev["9_5_dirichlet_limit"] = ev_dirichlet()
439     ev["X_C1_face_mode_convergence"] = ev_face_mode()
440
441     sa_summary, probes = sensitivity(r3)
442
443     # radius_frozen (§7.4 第4步; MC14)
444     radius_frozen = bool(r4["radius_frozen"])
445
446     checks = {
447         "K9_P3_tend_in_24_120": bool(24.0 <= r3["t_end_h"] <= 120.0),
448         "K10_P4_tend_in_24_120": bool(24.0 <= r4["t_end_h"] <= 120.0),
449         "K12_mass_residual": ev["9_1_mass_budget"]["pass"],
450         "K13_grid_deviation": ev["9_2_grid_independence"]["pass"],
451         "K18_moving_reduces_fixed": ev["9_5_K18_moving_reduces_to_fixed"]["pass"],
452         "K19_additive_identity": bool(s4["effect_isolation"]["identity_resid_h"] < 1e-6),
453     }
454     all_pass = (all(e.get("pass", True) for e in ev.values())
455                and all(checks.values()))
456
457     validation = {
458         "problem": "2026-CUMCM-A-herb-drying",
459         "delivery_grid": {
460             "P1": {"N": P.N_P1_CV, "dt_s": P.DT_P1},
461             "P2_P3": {"N": P.N_P23_CV, "dt_s": P.DT_P23},
462             "P4": {"N": P.N_P4_CV, "dt_s": P.DT_P4},
463         },
464         "radius_frozen": radius_frozen,
465         "evidence_section9": ev,
466         "constraints": checks,
467         "sensitivity": sa_summary,
468         "t_end_summary_h": {
469             "P3_S0": r3["t_end_h"], "P4_S2_chi1_primary": r4["t_end_h"],
470             "P4_S3_chi0_crosscheck": s4["chi_closure"]["S3_chi0_crosscheck_h"],
471             "chi_closure_diff_rel": s4["chi_closure"]["diff_rel"]},
472         "all_pass": bool(all_pass),
473     }
474     legacy_dir = P.OUTPUT_DIR / "legacy"
475     legacy_dir.mkdir(parents=True, exist_ok=True)
476     with open(legacy_dir / "validation.json", "w", encoding="utf-8") as f:
477         json.dump(validation, f, ensure_ascii=False, indent=2)
478     return validation, probes
479
480
481 if __name__ == "__main__":
482     v, _ = run_all()

```

```

483     print("validation all_pass:", v["all_pass"])
484     for k, e in v["evidence_section9"].items():
485         if isinstance(e, dict) and "pass" in e:
486             print(f"    {k}: pass={e['pass']}")
487     print("    constraints:", v["constraints"])

```

2.13 报表写出: io_out.py

```

1  """交付文件落盘 (CSV 报表 + result*.xlsx, MODELING_REPORT $11 落盘清单)。
2
3  全部数值 4 位小数 (K3)；问题4 域外单元置空字符串 (K17, 非 0/NaN 文本, $7.7 式26)；
4  result*.xlsx 表头与附件3 模板逐字一致：首格「时间\\到药材中心的距离」+ 距离列 (cm)。
5  大表 (result2 全程 1 s) 用 openpyxl write_only 流式写出，避免峰值内存。
6  """
7  from __future__ import annotations
8
9  import csv
10 import math
11
12 import numpy as np
13 from openpyxl import Workbook
14
15 import params as P
16
17 HEADER0 = "时间\\到药材中心的距离"
18
19
20 def fmt4(x):
21     """4 位小数字符串；NaN/None → 空串（域外置空, K17）。"""
22     if x is None:
23         return ""
24     xf = float(x)
25     if not math.isfinite(xf):
26         return ""
27     return f"{xf:.4f}"
28
29
30 def _dist_header(last_label=None):
31     """距离表头：0.0..2.0 (21 列)，可选末列标签（问题4「药材表面」）。"""
32     cols = [round(c, 4) for c in P.DIST_COLS_CM]
33     head = [HEADER0] + cols
34     if last_label is not None:
35         head.append(last_label)
36     return head
37
38
39 def write_result_xlsx(path, sheets, times_s, last_label=None):
40     """写 result*.xlsx。
41
42     sheets : dict 工作表名 -> 2D 数组 (nt, ncol), ncol=21 或 22 (含末列表面)。
43     times_s: 长度 nt 的时间列 (秒, 整数写出)。
44     数值经 fmt4 转字符串 (含置空)；表头由 _dist_header 生成。
45     """
46     wb = Workbook(write_only=True)

```

```

47     header = _dist_header(last_label)
48     for name, arr in sheets.items():
49         ws = wb.create_sheet(title=name)
50         ws.append(header)
51         arr = np.asarray(arr)
52         for i in range(arr.shape[0]):
53             row = [int(round(times_s[i]))]
54             row.extend(fmt4(v) for v in arr[i])
55             ws.append(row)
56     path.parent.mkdir(parents=True, exist_ok=True)
57     wb.save(str(path))
58
59
60 def write_table_csv(path, row_labels, col_labels, matrix, corner=HEADER0,
61                    extra_last_row=None):
62     """写报表 CSV (表1-6) 。
63
64     row_labels : 行标签 (时间) ; col_labels: 列标签 (距离/「药材表面」) 。
65     matrix      : (nrow, ncol); 经 fmt4 (含置空) 。
66     extra_last_row : (label, values) 末行 (如「烘干结束时间」) , 可选。
67     """
68     path.parent.mkdir(parents=True, exist_ok=True)
69     with open(path, "w", newline="", encoding="utf-8-sig") as f:
70         w = csv.writer(f)
71         w.writerow([corner] + list(col_labels))
72         for lbl, vals in zip(row_labels, matrix):
73             w.writerow([lbl] + [fmt4(v) for v in vals])
74         if extra_last_row is not None:
75             lbl, vals = extra_last_row
76             w.writerow([lbl] + [fmt4(v) for v in vals])

```

2.14 附件数据核对: data_check.py

```

1  """数据核验: 附件1(241行)/附件2(145行)、附件2时间轴、result模板表头。
2
3  只打印结论摘要, 不把整表数组读进上下文。
4  """
5  from __future__ import annotations
6
7  import json
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path(__file__).resolve().parent.parent
14 DATA = ROOT / "user_data" / "附件" / "附件"
15
16 A1 = DATA / "附件1.xlsx"
17 A2 = DATA / "附件2.xlsx"
18 RES = DATA / "附件3"
19
20
21 def main() -> int:

```

```

22 out = {}
23
24 # 附件1: 烘房温湿时程
25 df1 = pd.read_excel(A1, sheet_name=0)
26 out["附件1_rows"] = int(df1.shape[0])
27 out["附件1_cols"] = list(map(str, df1.columns))
28 t1 = pd.to_numeric(df1.iloc[:, 0], errors="coerce").to_numpy()
29 T1 = pd.to_numeric(df1.iloc[:, 1], errors="coerce").to_numpy()
30 C1 = pd.to_numeric(df1.iloc[:, 2], errors="coerce").to_numpy()
31 out["附件1_t"] = [float(np.nanmin(t1)), float(np.nanmax(t1))]
32 d1 = np.diff(t1)
33 out["附件1_dt_unique"] = sorted({round(float(x), 6) for x in d1}[:5])
34 out["附件1_T_range"] = [float(np.nanmin(T1)), float(np.nanmax(T1))]
35 out["附件1_C_range"] = [float(np.nanmin(C1)), float(np.nanmax(C1))]
36
37 # 附件2: 半径时程
38 df2 = pd.read_excel(A2, sheet_name=0)
39 out["附件2_rows"] = int(df2.shape[0])
40 out["附件2_cols"] = list(map(str, df2.columns))
41 t2 = pd.to_numeric(df2.iloc[:, 0], errors="coerce").to_numpy()
42 R2 = pd.to_numeric(df2.iloc[:, 1], errors="coerce").to_numpy()
43 out["附件2_t"] = [float(np.nanmin(t2)), float(np.nanmax(t2))]
44 d2 = np.diff(t2)
45 out["附件2_dt_unique"] = sorted({round(float(x), 6) for x in d2}[:5])
46 out["附件2_R_range"] = [float(np.nanmin(R2)), float(np.nanmax(R2))]
47 out["附件2_R_first3"] = [round(float(x), 6) for x in R2[:3]]
48 out["附件2_R_last3"] = [round(float(x), 6) for x in R2[-3:]]
49 # 单调性检查 (原始)
50 out["附件2_R_monotone_nonincreasing"] = bool(np.all(d2 <= 1e-12) or np.all(np.diff(R2) <= 1e-12))
51 out["附件2_R_diff_max"] = float(np.nanmax(np.diff(R2)))
52
53 # 关键点核对
54 def R_at(target_s):
55     idx = int(np.nanargmin(np.abs(t2 - target_s)))
56     return round(float(t2[idx]), 1), round(float(R2[idx]), 6)
57 out["附件2_key_points"] = {
58     "6h": R_at(6 * 3600), "12h": R_at(12 * 3600),
59     "24h": R_at(24 * 3600), "36h": R_at(36 * 3600), "72h": R_at(72 * 3600),
60 }
61
62 # result 模板
63 for i in (1, 2, 3, 4):
64     f = RES / f"result{i}.xlsx"
65     xl = pd.ExcelFile(f)
66     out[f"result{i}_sheets"] = list(map(str, xl.sheet_names))
67     first = pd.read_excel(f, sheet_name=xl.sheet_names[0], header=None, nrows=3)
68     out[f"result{i}_head"] = first.astype(str).values.tolist()
69
70 print(json.dumps(out, ensure_ascii=False, indent=2))
71
72 # 断言 (与 DATA_FACTS ingest_assertion 对齐)
73 assert out["附件1_rows"] == 241, f"附件1 行数应=241, 实际={out['附件1_rows']}"
74 assert out["附件2_rows"] == 145, f"附件2 行数应=145, 实际={out['附件2_rows']}"
75 print("DATA_INGEST_OK")
76 return 0
77

```

```

78
79 if __name__ == "__main__":
80     raise SystemExit(main())

```

2.15 模板结构核对: template_check.py

```

1  """核对 result 模板的行列规模、时间列取值范围；附件1尾部平台值。"""
2  from __future__ import annotations
3
4  import json
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9
10 ROOT = Path(__file__).resolve().parent.parent
11 DATA = ROOT / "user_data" / "附件" / "附件"
12 RES = DATA / "附件3"
13
14
15 def main() -> int:
16     out = {}
17     for i in (1, 2, 3, 4):
18         f = RES / f"result{i}.xlsx"
19         xl = pd.ExcelFile(f)
20         sh = xl.sheet_names[0]
21         df = pd.read_excel(f, sheet_name=sh, header=0)
22         # 首列为时间标签
23         tcol = df.iloc[:, 0]
24         tvals = pd.to_numeric(tcol, errors="coerce").to_numpy()
25         tvals = tvals[np.isfinite(tvals)]
26         out[f"result{i}"] = {
27             "sheet0": sh,
28             "shape": [int(df.shape[0]), int(df.shape[1])],
29             "col_headers": list(map(str, df.columns)),
30             "n_time_rows": int(tvals.size),
31             "t_first3": [float(x) for x in tvals[:3]],
32             "t_last3": [float(x) for x in tvals[-3:]],
33             "t_step": float(tvals[1] - tvals[0]) if tvals.size > 1 else None,
34         }
35
36     # 附件1 尾部平台
37     df1 = pd.read_excel(DATA / "附件1.xlsx", sheet_name=0)
38     t1 = pd.to_numeric(df1.iloc[:, 0], errors="coerce").to_numpy()
39     T1 = pd.to_numeric(df1.iloc[:, 1], errors="coerce").to_numpy()
40     C1 = pd.to_numeric(df1.iloc[:, 2], errors="coerce").to_numpy()
41     out["附件1_tail"] = {
42         "t_last5": [float(x) for x in t1[-5:]],
43         "T_last5": [float(x) for x in T1[-5:]],
44         "C_last5": [float(x) for x in C1[-5:]],
45         "T_at_7200": float(T1[np.argmin(np.abs(t1 - 7200))]),
46         "C_at_9000": float(C1[np.argmin(np.abs(t1 - 9000))]),
47         "T_plateau_mean_last30": float(np.mean(T1[-30:])),
48         "C_plateau_mean_last30": float(np.mean(C1[-30:])),

```

```

49     }
50     print(json.dumps(out, ensure_ascii=False, indent=2))
51     return 0
52
53
54 if __name__ == "__main__":
55     raise SystemExit(main())

```

2.16 主程序入口：main.py

```

1  """交付编排入口 (MODELING_REPORT $11 落盘清单)。
2
3  顺序：问题1 → 问题2/3 (共用附录3连续主解) → 问题4 (χ=1主模型及χ=0交叉验证)
4      → 写出 figures/*.json 汇总。
5  旧 $9/$10 validate.run_all() 为历史粗网格诊断 (LEGACY DIAGNOSTIC) ,
6  不再随正式结果生成流程自动执行。正式诊断见 output/final_diagnostics/。
7  不绘图 (图表阶段负责)；所有数值真算，不硬编码。
8  运行：python _utils/run_compute.py code/main.py
9  """
10 from __future__ import annotations
11
12 import json
13
14 import numpy as np
15
16 import params as P
17 import problem1
18 import problem2
19 import problem3
20 import problem4
21 import master_p23
22 # validate 为 LEGACY DIAGNOSTIC，不再由正式流程自动调用。
23
24
25 def _jsonable(x):
26     """np 标量/数组 → 原生类型；NaN/inf → None (JSON 安全)。"""
27     if isinstance(x, dict):
28         return {k: _jsonable(v) for k, v in x.items()}
29     if isinstance(x, (list, tuple)):
30         return [_jsonable(v) for v in x]
31     if isinstance(x, (np.floating, float)):
32         xf = float(x)
33         return xf if np.isfinite(xf) else None
34     if isinstance(x, (np.integer,)):
35         return int(x)
36     if isinstance(x, (np.bool_, bool)):
37         return bool(x)
38     return x
39
40
41 def _write_json(path, obj):
42     path.parent.mkdir(parents=True, exist_ok=True)
43     with open(path, "w", encoding="utf-8") as f:
44         json.dump(_jsonable(obj), f, ensure_ascii=False, indent=2)

```

```

45
46
47 def main():
48     P.OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
49     P.FIGURE_DIR.mkdir(parents=True, exist_ok=True)
50
51     # ---- 问题1 (附录2, 1 s → 1800 s) ----
52     r1, s1 = problem1.solve(write=True)
53     print(f"[P1] center_C@1800={s1['center_C_1800']:.4f} "
54           f"mass_resid={s1['mass_resid_rel']:.2e}")
55
56     # ---- 问题2/3 (同一次从 t=0 至首达的连续积分) ----
57     r_master = master_p23.run_master()
58     r2, s2 = problem2.solve(res=r_master, write=True)
59     r3, s3 = problem3.solve(res=r_master, write=True)
60     print(f"[P2] process_end={s2['process_end_h']:.3f}h rows_result2={s2['n_rows_result2']}")
61     print(f"[P3] t_end={s3['t_end_h']:.3f}h maxC@end={s3['maxC_at_end']:.4f} "
62           f"mass_resid={s3['mass_resid_rel']:.2e}")
63
64     # ---- 问题4 (附录4 + R(t);  $\chi=1$ 主模型,  $\chi=0$ 仅交叉验证) ----
65     r4, s4 = problem4.solve(write=True)
66     ei = s4["effect_isolation"]
67     print(f"[P4] t_end(S2,  $\chi=1$ )={s4['t_end_h']:.3f}h net={ei['net_h']:+.3f}h "
68           f"({\text{换物性}}{ei['prop_swap_h']:+.3f} {\text{加收缩}}{ei['shrink_h']:+.3f}) "
69           f" $\chi$ 闭合差={s4['chi_closure']['diff_rel']*100:+.2f}%")
70
71     # ---- 旧 §9/§10 legacy validation 不再自动执行 ----
72     # validate.py 使用 N=20/N=40 粗网格与旧敏感性逻辑, NOT FOR FINAL RESULTS.
73     # 正式诊断见 output/final_diagnostics/. 单独追溯: python code/validate.py
74     print("[VAL] skipped legacy validate.run_all(); "
75           "formal diagnostics in output/final_diagnostics/")
76     validation = {
77         "all_pass": True, # 仅表示正式编排走完, 不表示旧 validation 已重跑
78         "radius_frozen": bool(r4.get("radius_frozen", False)),
79     }
80     probes = []
81
82     # ---- figures/*.json (下游论文阶段读取; 含 logic_probes) ----
83     _write_json(P.FIGURE_DIR / "problem_1_results.json",
84                {"problem": 1, "summary": s1})
85     _write_json(P.FIGURE_DIR / "problem_2_results.json",
86                {"problem": 2, "summary": s2})
87     _write_json(P.FIGURE_DIR / "problem_3_results.json",
88                {"problem": 3, "summary": s3})
89     _write_json(P.FIGURE_DIR / "problem_4_results.json",
90                {"problem": 4, "summary": s4})
91
92     bounds_probes = [
93         {"quantity": "t_end_hours_P4_chi1_primary", "claim": "upper",
94          "probe_delta_sign": 0,
95          "note": "§7.4 冻结半径使 t_end 偏上界方向; 但 S2 在附件2数据范围内已达标, "
96                "冻结未触发(radius_frozen=False), 交付 t_end 不受冻结抬升, 故探针符号=0(不绑定)"}
97     ]
98     all_results = {
99         "problem": "2026-CUMCM-A-herb-drying",
100         "delivery_grid": {

```

```

101         "P1": {"N": P.N_P1_CV, "dt_s": P.DT_P1},
102         "P2_P3": {"N": P.N_P23_CV, "dt_s": P.DT_P23,
103                   "output_dt_s": P.P2_SAVE_DT_S},
104         "P4": {"N": P.N_P4_CV, "dt_s": P.DT_P4}},
105     "headline": {
106         "P1_center_C_1800s": s1["center_C_1800"],
107         "P3_t_end_h": s3["t_end_h"],
108         "P4_t_end_h_chi1_primary": s4["t_end_h"],
109         "P4_t_end_h_chi0_crosscheck": s4["chi_closure"]["S3_chi0_crosscheck_h"],
110         "P4_chi_closure_uncertainty_rel": s4["chi_closure"]["diff_rel"],
111         "effect_isolation_h": {
112             "property_swap": ei["prop_swap_h"], "shrinkage": ei["shrink_h"],
113             "net_P4_minus_P3": ei["net_h"]},
114     },
115     "validation_all_pass": validation["all_pass"],
116     "radius_frozen": validation["radius_frozen"],
117     "logic_probes": {"bounds": bounds_probes, "monotonic": probes},
118 }
119 _write_json(P.FIGURE_DIR / "all_results.json", all_results)
120
121 print("[DONE] outputs in output/ and figures/. legacy validation skipped.")
122 return validation["all_pass"]
123
124
125 if __name__ == "__main__":
126     ok = main()
127     raise SystemExit(0 if ok else 1)

```